

# INHALTSVERZEICHNIS

---

## I Thesis

|       |                                                                                 |    |
|-------|---------------------------------------------------------------------------------|----|
| 1     | Motivation                                                                      | 2  |
| 2     | Grundlagen                                                                      | 4  |
| 2.1   | Software-Qualitätsmetriken . . . . .                                            | 4  |
| 2.2   | Software-Visualisierung . . . . .                                               | 6  |
| 2.2.1 | 2.5D- und 3D-Software-Visualisierung . . . . .                                  | 6  |
| 2.3   | CodeCity: Grundstein für 2.5D-Visualisierung . . . . .                          | 8  |
| 2.4   | Treemap-Layouts . . . . .                                                       | 9  |
| 2.4.1 | Squarify-Algorithmus . . . . .                                                  | 12 |
| 3     | Problemstellung                                                                 | 20 |
| 3.1   | Das Treemap-Problem . . . . .                                                   | 23 |
| 4     | Verwandte Arbeiten                                                              | 27 |
| 4.1   | Vergleich von Visualisierungstechniken . . . . .                                | 27 |
| 4.2   | Das Treemap-Problem . . . . .                                                   | 31 |
| 5     | Layout Suche                                                                    | 36 |
| 5.1   | Methodik . . . . .                                                              | 36 |
| 5.2   | Durchführung der Suche . . . . .                                                | 38 |
| 5.3   | Dokumentation der Ergebnisse und Vorstellung der Layout-<br>Umsetzung . . . . . | 49 |
| 5.3.1 | Sunburst Layout . . . . .                                                       | 49 |
| 5.3.2 | Circle Packing Layout . . . . .                                                 | 49 |
| 5.3.3 | Noch eins mehr? . . . . .                                                       | 49 |
| 6     | Datenanalyse                                                                    | 50 |
| 6.1   | Auswahl der Projekte . . . . .                                                  | 50 |
| 6.1.1 | Die Auswahl der Projekte im Detail . . . . .                                    | 54 |
| 6.2   | Struktur von Softwareprojekten . . . . .                                        | 56 |
| 6.2.1 | Orderketten . . . . .                                                           | 57 |
| 7     | Erweiterung des Squarify Algorithmus                                            | 60 |
| 7.1   | Kriterien für Treemap Layouts . . . . .                                         | 60 |
| 7.2   | Zweifache Berechnung . . . . .                                                  | 61 |
| 7.2.1 | annäherungs wert . . . . .                                                      | 62 |
| 7.2.2 | Größenanpassung . . . . .                                                       | 62 |
| 7.3   | Scaling der Knoten . . . . .                                                    | 66 |
| 7.4   | Reihenfolge der Knoten im ersten Layoutschritt . . . . .                        | 68 |
| 7.4.1 | Reihenfolge der Knoten nach erstem Layoutschritt . . . . .                      | 68 |
| 7.5   | Mehrfache Berechnung . . . . .                                                  | 69 |
| 7.5.1 | Iterative Abstandsvergrößerung . . . . .                                        | 70 |
| 7.6   | Änderungen im Layout . . . . .                                                  | 70 |
| 7.6.1 | Optimierung von Ordnerketten . . . . .                                          | 70 |
| 7.6.2 | Beschriftung der Knoten . . . . .                                               | 74 |
| 7.6.3 | Abstände zwischen Geschwister Knoten . . . . .                                  | 75 |

|                     |    |
|---------------------|----|
| 7.7 Fazit . . . . . | 76 |
| 8 evaluation        | 80 |
| 9 Fazit             | 81 |
| <b>II Appendix</b>  |    |
| A Dateien           | 84 |
| Literatur           | 99 |

Teil I

THESIS

## MOTIVATION

---

Die Vermittlung von Softwarequalität an Personen ohne regelmäßigen Kontakt zum Quellcode stellt eine anhaltende Herausforderung dar [slrSoftwareVisualization]. Auch erfahrene Softwareentwickler stehen vor der Aufgabe, sich in kurzer Zeit einen systematischen Überblick über fremde oder umfangreiche Softwaresysteme zu verschaffen. Wichtige Fragestellungen sind dabei die Identifikation potenziell problematischer Bereiche, die Priorisierung von Analyseaufwänden sowie die gezielte Erkennung von Verbesserungspotenzialen. Darüber hinaus ist eine effektive Kommunikation über die Softwarestruktur und deren Qualität mit Stakeholdern erforderlich, die nicht direkt in die Softwareentwicklung involviert sind. Diese Anforderungen bedingen eine klare und verständliche Darstellung der Softwarearchitektur und deren Qualitätsaspekte.

Visualisierungen haben sich als zentrales Werkzeug zur Unterstützung dieser Aufgaben etabliert, da sie es ermöglichen, komplexe Softwarestrukturen zu abstrahieren und Muster sowie technische Schulden sichtbar zu machen, ohne den vollständigen Quelltext analysieren zu müssen [TCo8; WLo7; MFMo3]. Einfache Diagramme oder Dashboards erleichtern den Zugang zur Systemstruktur. Insbesondere immersive Visualisierungsformen, wie dreidimensionale Darstellungen, zeigen ein hohes Potenzial, Softwareinformationen intuitiv und mit hoher Informationsdichte zu vermitteln [TCo8].

2.5D-Visualisierungsmetaphern, beispielsweise die Stadtmetapher, haben sich sowohl in der Forschung als auch in industriellen Anwendungen als besonders wirkungsvoll erwiesen [WLo7; Mer+17; Gmbc]. Die Übertragung von Softwarestrukturen auf städtische Elemente wie Gebäude, Blöcke und Straßen ermöglicht ein räumliches Abbild, das verschiedene Code-Metriken integriert. Für das Layout solcher Städte werden häufig Treemap-Algorithmen eingesetzt [WLo7; TCo8; Gmbb]. Allerdings stoßen diese Methoden bei sehr großen Softwaresystemen bezüglich Übersichtlichkeit und Detailgenauigkeit an ihre Grenzen [LFo8]. Hinsichtlich Lesbarkeit, Informationsdichte und Nutzerfreundlichkeit ist bislang nicht abschließend geklärt, ob klassische Treemap-Layouts für 2.5D-Softwarevisualisierungen optimal geeignet sind oder ob Verbesserungsbedarf besteht [TCo8; LFo8].

Auch alternative Ansätze zur Visualisierung von Softwarequalität existieren [TCo8; LSo2; GYBo4]. Derzeit mangelt es jedoch an systematischen Vergleichen dieser unterschiedlichen Visualisierungsformen und Layoutalgorithmen. Insbesondere fehlen empirische Studien, die die Wirksamkeit der Methoden im praktischen Einsatz mit realen Anwendungsszenarien vergleichen [SLD20]. Zudem fokussieren die meisten bisherigen Untersuchungen hauptsächlich auf Softwareentwickler als Zielgruppe. Die Anforderungen



und Perspektiven von Personen ohne tiefgehende technische Vorkenntnisse wurden bislang nur unzureichend betrachtet [SLD20; TCo8].

Hieraus ergibt sich ein Bedarf an systematischen Vergleichen verschiedener Visualisierungsansätze und Layoutalgorithmen hinsichtlich ihrer Eignung und Verständlichkeit für diverse Nutzergruppen im Kontext der Softwarequalitätsdarstellung.

In diesem Abschnitt werden die grundlegenden Konzepte und Begriffe erläutert, die für das Verständnis dieser Arbeit notwendig sind. Zunächst werden Software-Qualitätsmetriken vorgestellt, da sie die Basisdaten für alle in dieser Arbeit genutzten Visualisierungen liefern. Anschließend wird das Thema Software-Visualisierung behandelt, wobei insbesondere auf die dreidimensionale Software-Visualisierung mit Hilfe von Treemaps eingegangen wird.

### 2.1 *Software-Qualitätsmetriken*

Software-Qualitätsmetriken sind zentrale Werkzeuge zur Messung und Bewertung der Qualität von Software. Sie ermöglichen es, verschiedene Eigenschaften der Software objektiv anhand von Kennzahlen zu analysieren. Eine *Softwaremetrik* ist dabei meist als mathematische Funktion zu verstehen, die eine spezifische Eigenschaft einer Software in einen Zahlenwert abbildet:

Eine Softwaremetrik, oder kurz Metrik, ist eine (meist mathematische) Funktion, die eine Eigenschaft von Software in einen Zahlenwert, auch Maßzahl genannt, abbildet. Hierdurch werden formale Vergleichs- und Bewertungsmöglichkeiten geschaffen.[WP04]

Auch in anderen Definitionen wird die grundsätzliche Funktion von Metriken hervorgehoben:

Softwaremetrik ist ein quantitatives Maß, das verwendet wird, um die Eigenschaften eines Softwareproduktes oder des Softwareentwicklungsprozesses zu bewerten.[Sofa]

Eine Softwarequalitätsmetrik ist eine Funktion, die eine Software-Einheit in einen Zahlenwert abbildet, welcher als Erfüllungsgrad einer Qualitätseigenschaft der Software-Einheit interpretierbar ist.[EH98]

Allen Definitionen ist gemeinsam, dass eine Metrik stets nur einen bestimmten Aspekt der Software abbildet. Genau darin liegt eine der wichtigsten Erkenntnisse und gleichzeitig eine zentrale Kritik: Einzelne Metriken geben ausschließlich Auskunft über spezifische Eigenschaften und erlauben keine umfassende Bewertung der gesamten Softwarequalität.

Zusammenfassend kann festgestellt werden, dass Software-Qualitätsmetriken als quantitative Indikatoren zur Kontrolle und Verbesserung der Software dienen. Sie sind jedoch stets darauf beschränkt, nur bestimmte Aspekte zu messen. Die Aussagekraft jeder einzelnen Metrik ist daher begrenzt:

Eine Metrik alleine kann nie eine vollständige Aussage über die Qualität der Software treffen. Metriken sollten immer in Kombination bewertet werden. Die Güte und Reife von Software kann nur in Kombination mit statischer Code Analyse, Code Reviews und funktionalen Tests final beurteilt werden.[Sch19]

Es existiert also keine universelle Metrik, die vollumfänglich Auskunft über die Qualität einer Software geben kann. Dies allein schon, da der Begriff *gute Software* situationsabhängig definiert werden muss. Selbst wenn fehlerfreie Ausführung als Ziel gilt, ist unklar, wie diese Eigenschaft in eine einzelne Metrik gefasst werden könnte.

Um ein möglichst vollständiges Qualitätsbild zu erhalten, ist es daher essenziell, verschiedene Metriken miteinander zu kombinieren. Die parallele Interpretation mehrerer Indikatoren erhöht die Aussagekraft und reduziert das Risiko von Fehleinschätzungen.

Die Aussagekraft von Softwaremetriken hängt maßgeblich davon ab, dass die Ziele der Software klar definiert sind [HF94]. Metriken müssen stets so gewählt werden, dass sie die Qualität einer Software bezüglich dieser Ziele abbilden. Beispielsweise ist eine Komplexitätsmetrik irrelevant, wenn primär die Ausführungsgeschwindigkeit optimiert werden soll. Auch kann die Erhebung und Auswertung von Metriken selbst mit Aufwand verbunden sein.

Ein weiterer Kritikpunkt ist, dass viele Metriken nicht ausreichend wissenschaftlich fundiert sind und der tatsächliche Nutzen nicht immer klar belegt werden kann [VK17]. Die Orientierung an Metriken sollte daher immer kritisch erfolgen und nicht zum Selbstzweck werden.

Zudem besteht die Gefahr, dass bei unpassend gewählten Metriken lediglich diese Werte optimiert werden, ohne dass sich dadurch die Gesamtqualität der Software verbessert. Entwickler könnten dazu verleitet werden, ihr Handeln auf das Erreichen guter Metrikwerte anstatt auf das eigentliche Ziel der Software auszurichten [Wor].

Nicht zuletzt sind Metriken kontextabhängig. Sie liefern Indizien, können aber Prozesse wie Code Reviews oder Tests keinesfalls ersetzen. Die Interpretation muss mit Blick auf den jeweiligen Kontext erfolgen. Bekannte Beispiele wie Lines-of-Code oder Anzahl von Commits werden oft falsch als Qualitätsindikatoren genutzt, obwohl sie weder die Komplexität noch die tatsächliche Arbeitsleistung adäquat abbilden [Wor].

Einfache Metriken können zudem keine komplexen Qualitätsprobleme identifizieren [VK17] und sind häufig stark abhängig von Programmiersprache und individuellem Programmierstil.

Die Norm ISO/IEC 25010 [Iso] beschreibt einen Rahmen, in dem die Qualität von Software anhand von acht Qualitätsmerkmalen, wie z.B. funktionaler Eignung, Zuverlässigkeit oder Benutzerfreundlichkeit, strukturiert wird. Diese Merkmale beziehen sich auf die Sicht von außen, also wie der Benutzer die Software erlebt:

Qualität wird als Abwesenheit von Fehlern im Systemverhalten verstanden, die Software verhält sich demnach so, wie der Benutzer es erwartet.[Wit18, S. 1]

Für die Ursachen bestimmter Qualitätsprobleme reicht die Betrachtung aus Nutzersicht allerdings oft nicht aus. Seit den 1970er Jahren entstand daher ein Fokus auf die Messung der *internen Qualität* beziehungsweise der *Code-Qualität*, etwa mittels Metriken wie der McCabe-Komplexität [Lud]. Diese ist insbesondere für Entwickler sowie Auftraggeber relevant und unterstützt die Sicherstellung von Wartbarkeit und Erweiterbarkeit der Software. Im Rahmen dieser Arbeit wird, soweit von Metriken die Rede ist, explizit auf Code-Qualitätsmetriken Bezug genommen.

Software-Qualitätsmetriken beziehen sich in der Regel auf spezifische Softwareeinheiten, beispielsweise Dateien, Module oder Klassen. Diese Einheiten sind meistens hierarchisch, etwa entlang von Datei- und Ordnerstrukturen, organisiert. Die Messwerte werden häufig auf der Ebene einzelner Dateien gesammelt und dann auf höhere Ebenen, zum Beispiel Ordner oder Module, aggregiert, sodass eine baumartige Hierarchie entsteht.

Die im Folgenden vorgestellten Ansätze zur Visualisierung hierarchischer Metrik-Daten sind nicht zwingend spezifisch für Softwaremetriken, sondern lassen sich auch auf andere Anwendungsfälle übertragen. Dennoch weist die Struktur von Software-Einheiten und deren Metrik-Daten gewisse Eigenheiten auf, was Einfluss auf die Gestaltung der Visualisierungen haben kann (siehe Abschnitt 6).

## 2.2 Software-Visualisierung

Die Darstellung von Softwarestrukturen zählt zu den entscheidenden Schritten, um komplexe Softwaresysteme für Menschen verständlich zu machen. Während numerische Metriken oder tabellarische Darstellungen für Fachleute in der Softwareentwicklung oft ausreichend sind, können diese Darstellungsformen für Personen ohne technischen Hintergrund abstrakt und schwer zugänglich bleiben. Es besteht daher das Ziel, Softwarequalität auch für nicht-technische Stakeholder intuitiv und erlebbar zu machen.

Softwarevisualisierung verfolgt das Ziel, komplexe Strukturen und Beziehungen in Softwaresystemen so darzustellen, dass sie schneller erfasst und verstanden werden können [Die]. Visuelle Repräsentationen unterstützen nicht nur Entwickler bei der Analyse und Fehlersuche, sondern fördern auch die Kommunikation zwischen technischen und nicht-technischen Stakeholdern [BD17].

### 2.2.1 2.5D- und 3D-Software-Visualisierung

Traditionell werden Softwaremetriken in numerischer oder zweidimensionaler Form präsentiert. Diese Formen liefern zwar einen guten Überblick, vermitteln jedoch wenig *Greifbarkeit* des Softwaresystems. 3D-Visualisierungen

bieten dagegen das Potenzial, immersive und erfahrbare Eindrücke zu schaffen, sodass Nutzer *in die Software eintauchen* können:

Despite the proven usefulness of 2D visualizations, they do not allow the viewer to be immersed in a visualization, and the feeling is that we are looking at things from ‘outside’. 3D visualizations on the other hand provide the potential to create such an immersive experience [WLo7, S. 1]

Anzumerken ist, dass die Autoren von CodeCity über 3D-Visualisierungen sprechen, jedoch in Wirklichkeit eine 2.5D-Visualisierung beschrieben wird. [Joh93] 2.5D ist laut Wikipedia [WP09] eine Darstellung, die zwar drei Dimensionen nutzt, aber die dritte Dimension “deutliche Einschränkungen” aufweist, was bei CodeCity der Fall ist, da die Darstellung drei Dimensionen nutzt, aber die dritte Dimension *nur* für die Extrusion der Gebäude genutzt wird und nicht zum Beispiel für die Platzierung der Gebäude, was eine deutliche Einschränkung in der dritten Dimension darstellt.

Die Idee, Softwarestrukturen dreidimensional darzustellen, ist nicht neu. Bereits 1995 stellte Steven P. Reiss einen ersten Ansatz zur 3D-Visualisierung von Software vor [Rei95]. Ursprünglich zielten diese Ansätze darauf ab, Entwicklern einen schnellen Überblick über Struktur und Aufbau umfangreicher Systeme, insbesondere beim Einarbeiten in unbekannte Software, zu verschaffen [YM98]. Die frühen Methoden fokussierten dabei meist die Darstellung von Architektur und Verbindungen, weniger jedoch die Visualisierung von Softwarequalitätsmetriken (siehe Abbildung 2.1).

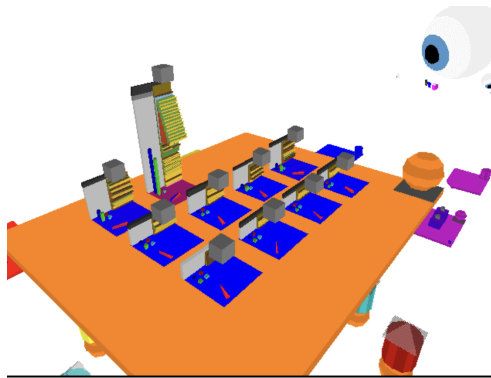


Abbildung 2.1: Beispiel für eine 3D-Visualisierung von Young und Munro [YM98, S. 6]

Für die Visualisierung von Softwarequalitätsmetriken lassen sich aus der Literatur Kriterien für eine gelungene Darstellung ableiten [YM98]:

- **Darstellung der Struktur:** Die Visualisierung soll Architektur und Aufbau der Software möglichst anschaulich und nachvollziehbar zeigen. Wichtige Aspekte dabei sind:
  - *Informationsgehalt:* Die Darstellung soll möglichst viele relevante Informationen kompakt vermitteln.

- *Visuelle Komplexität*: Trotz hoher Informationsdichte soll die Visualisierung übersichtlich und nicht überfordernd wirken. Dies ist als Gegenspieler zum Informationsgehalt zu verstehen.
  - *Skalierbarkeit*: Auch große Softwaresysteme müssen klar und verständlich dargestellt werden können. Die Autoren von *Visualising Software in virtual reality* [YM98] sagen sogar, dass Mechanismen notwendig wären, um Komplexität und Informationsgehalt bei zunehmender Größe manuell zu steuern und je nach Software-System anpassen zu können.
  - *Stabilität gegenüber Änderungen*: Die Visualisierung soll bei Software-Änderungen konsistent bleiben, um Vergleichbarkeit über Versionen hinweg zu gewährleisten.
  - *Visuelle Metaphern*: Der Einsatz eingängiger Metaphern erleichtert das Verständnis komplexer Softwarestrukturen.
- **Abstraktion**: Unwesentliche Details sollten ausgeblendet werden, um ein verständliches, abstrahiertes Modell zu erzeugen.
  - **Navigation**: Die Orientierung innerhalb der Visualisierung muss auch bei sehr großen Systemen einfach und intuitiv möglich sein.
  - **Korrelation mit dem Code**: Es sollte eine klare Zuordnung zwischen Visualisierungselementen und Quellcodebestandteilen existieren.
  - **Automatisierung**: Die Generierung der Visualisierung sollte vollständig automatisierbar sein, sodass keine manuelle Nachbearbeitung notwendig ist.

### 2.3 CodeCity: Grundstein für 2.5D-Visualisierung

Eine prägende Arbeit für die 2.5D-Visualisierung von Softwarestrukturen und -metriken ist das Konzept *CodeCity* von Richard Wetzel und Michele Lanza [WL07]. Sie stellten erstmals einen Ansatz vor, der Software-Struktur auf Modulebene nutzt, um den Grundriss für eine 2.5D-Visualisierung zu schaffen. Dabei werden Softwareeinheiten als Quader dargestellt. Wie von Young und Munro [YM98] gefordert, nutzen sie eine gute visuelle Metapher, um die Software darzustellen: die Stadtmetapher. Sie ist zentrales Gestaltungsmittel von CodeCity: Klassen erscheinen als Gebäude, Pakete als Stadtbezirke. Den Objekten werden verschiedene Attribute (Dimensionen, Positionen, Farben, Farbsättigung, Transparenz) zugeordnet, was eine intuitive und greifbare Darstellung ermöglicht (siehe Abbildung 2.2).

Während frühere Arbeiten, wie die von Marcus et al. [MFM03], auf sehr feingranulare Codebestandteile, wie Attribute und Methoden, fokussierten, bietet CodeCity eine abstrahierende Sicht auf Klassen, Pakete und deren Struktur und ist dadurch vorallem für nicht-technische Stakeholder verständlicher und leichter zu überblicken.

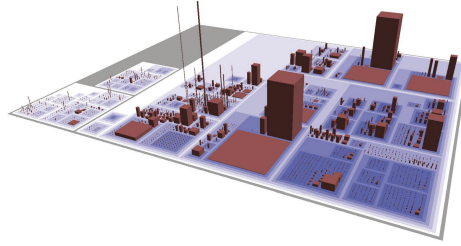


Abbildung 2.2: Beispiel für eine originale CodeCity-Visualisierung [WLo7, S. 2]

Der Grundriss (folgend auch Layout) der Städte beruht auf einer Variante der Treemap-Algorithmen (erklärt im folgenden Abschnitt 2.4). Es ist zu erkennen, dass größere Gebäude im unteren linken Bereich, kleinere weiter oben rechts angeordnet sind. Gebäude (Module) und Viertel (Pakete) sind immer quadratisch, dadurch bleibt viel ungenutzte *leere* Fläche.

Das Originalverfahren des CodeCity-Algorithmus ist nicht öffentlich verfügbar. In dieser Arbeit wird daher für vergleichende Analysen eine eigene Implementierung verwendet, so dass Abweichungen zur Originalvisualisierung möglich sind (außer wenn anders gekennzeichnet).

## 2.4 Treemap-Layouts

Hierarchische Daten werden häufig als Bäume dargestellt, jedoch zeigen sich klassische Darstellungen wie Baumdiagramme oder Venn-Diagramme insbesondere bei großen Datenmengen als ineffizient hinsichtlich der Flächennutzung und generell unübersichtlich [JS91]. Besonders für große, strukturreiche und mengenorientierte Daten stoßen traditionelle Visualisierungsmethoden an ihre Grenzen, da sie weder Informationen noch Hierarchien kompakt und intuitiv vermitteln können.

Um diese Herausforderungen zu adressieren, entwickelten Shneiderman und Johnson 1991 das Konzept der *Treemap* zur effizienten Darstellung hierarchischer Strukturen [JS91]. Das zentrale Prinzip von Treemaps besteht darin, jedem Knoten eines gewichteten Baums ein Rechteck zuzuweisen, dessen Fläche proportional zu dem Gewicht (beispielsweise Datenmenge, Marktwert, o. ä.) ist. Die Rechtecke aller Blätter füllen die Fläche des Wurzelrechtecks vollständig aus, sodass die Gesamtfläche der Kindknoten exakt der Fläche ihres Elternrechtecks entspricht. Mathematisch liegt der Visualisierung also ein gewichteter Baum zugrunde, wobei insbesondere die Blattknoten jeweils einen numerischen Wert besitzen.

Shneiderman und Johnson identifizieren für gelungene Treemaps folgende Schlüsselziele:

- **Effiziente Platznutzung:** Maximale Informationsdichte auf minimalem Raum.
- **Verständlichkeit:** Einfache Erfassbarkeit der dargestellten Hierarchie und Werte mit geringem kognitiven Aufwand. Die Struktur soll möglichst schnell erkennbar sein.



- **Ästhetik:** Ansprechende und übersichtliche Anordnung der Rechtecke.

Frühere Ansätze wie Listen, klassische Baumdiagramme (Abbildung 2.3) oder Venn-Diagramme (Abbildung 2.4) konnten diese Anforderungen nicht erfüllen. Sie leiden unter Problemen wie ineffizienter Flächenausnutzung und fehlender Möglichkeit, zusätzlich zu den Beziehungen auch quantitativ-metrische Informationen anschaulich zu vermitteln. So kann zum Beispiel in klassischen Baumdiagrammen bei großen Datenmengen mehr als die Hälfte des dargestellten Bereichs aus leerem, informationslosem Raum bestehen [JS91, S. 3]. Venn-Diagramme sind insbesondere aus Platzgründen für größere Strukturen ungeeignet: "The space required between regions would certainly preclude this Venn diagram representation from serious consideration for larger structures." [JS91, S. 5]

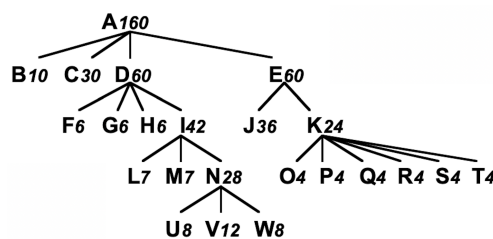


Abbildung 2.3: Beispiel für ein Baumdiagramm

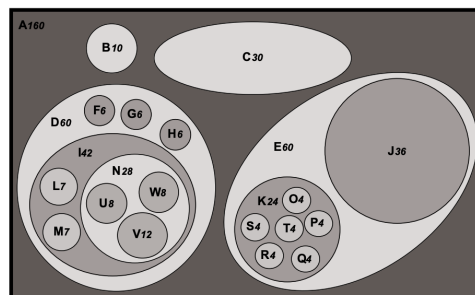


Abbildung 2.4: Beispiel für ein Venn-Diagramm

Um die genannten Schwierigkeiten zu lösen, formulieren Shneiderman und Johnson vier grundlegende Eigenschaften für Treemap-Layouts:

- Ist Knoten 1 ein Vorfahre von Knoten 2, so ist der Bereich von Knoten 2 vollständig innerhalb des Bereichs von Knoten 1 enthalten.
- Die Bereiche von Knoten schneiden sich, wenn einer ein Vorfahre des anderen ist.
- Jeder Knoten erhält eine Fläche, die streng proportional zu seinem Gewicht ist.
- Das Gewicht eines Knotens ist (mindestens) so groß wie die Summe der Gewichte seiner Kinder.



Sie präsentieren einen einfachen, rekursiven Algorithmus, der diese Bedingungen erfüllt: Der verfügbare Raum wird abwechselnd vertikal und horizontal entsprechend den Gewichten der Kindknoten aufgeteilt. Die Berechnung erfolgt von der Wurzel bis zu den Blättern und ist mit einer Laufzeit von  $O(n)$  effizient. Ein Beispiel für eine Treemap, die mit diesem Algorithmus erstellt wurde, ist in Abbildung 2.5 zu sehen.



Abbildung 2.5: Beispiel für eine Treemap

Trotz der weiten Verbreitung des Ansatzes weisen Shneiderman und Johnson auf ein zentrales Problem nicht explizit hin: Der notwendige Abstand zwischen Rechtecken wird realisiert, indem von jedem Rechteck an allen vier Seiten ein Rand abgezogen wird. Dadurch ist die resultierende Fläche oftmals nicht mehr proportional zu den zugehörigen Werten. Insbesondere sehr kleine oder langgestreckte Rechtecke können im Extremfall sogar komplett verschwinden. Dies verletzt die dritte Eigenschaft (strikte Proportionalität). In praktischen Anwendungsfällen, zum Beispiel wenn die Rechtecksfläche eine Kennzahl wie "Lines of Code" darstellen soll, können kleine Dateien oder Knoten durch die Ränder also komplett verloren gehen. Problematisch ist dies insbesondere, wenn eine weitere Dimension (wie beispielsweise die Testabdeckung) diese Elemente eigentlich hervorheben sollte, sie aber infolge der Skalierung durch Randabzug nicht mehr sichtbar sind.

Ein weiteres ästhetisches Manko liegt darin, dass die rekursive Unterteilung längliche oder stark verzerrte Rechtecksformen erzeugen kann. Dies widerspricht ebenfalls dem Anspruch übersichtlicher und ansprechender Visualisierung. Verschiedene Verbesserungsansätze wie etwa *Squarified Treemaps* begegnen diesem Problem und werden im Folgenden diskutiert (siehe Abschnitt 2.4.1).

Eine 3D-Treemap ist die rekursive Unterteilung eines Würfels (oder Quaders) in kleinere Volumina, wobei zum Wertvergleich das Volumen und nicht mehr die Fläche herangezogen wird. [Joh93] Gegenstand der vorliegenden Arbeit sind jedoch sogenannte *2.5D-Treemaps*, bei denen ein klassisches 2D-Treemap-Layout um eine zusätzliche dritte Dimension (zum Beispiel durch Extrusion nach oben) erweitert wird.

### 2.4.1 Squarify-Algorithmus

Der Squarify-Algorithmus ist ein Verfahren zur Anordnung von Rechtecken in Treemaps, bei dem die Seitenverhältnisse der Rechtecke möglichst ausgeglichen gestaltet werden sollen. Ziel ist es, annähernd quadratische Rechtecke zu erzeugen, um die Nachteile herkömmlicher Treemap-Layouts zu überwinden, bei denen oft sehr schmale und langgezogene Rechtecke entstehen. Bruls et al. betonen dazu: “another problem of standard treemaps [is] the emergence of thin, elongated rectangles” [BHVW00, S. 1]. Rechtecke mit möglichst ähnlichen Seitenlängen sind einerseits leichter wahrzunehmen, zu vergleichen sowie in ihrer Größe intuitiv abschätzbar.

Der Squarify-Algorithmus arbeitet rekursiv, indem er die zu verteilende Fläche schrittweise in Rechtecke aufteilt. Dabei wird stets angestrebt, das Verhältnis von Breite zu Länge der Rechtecke so nah wie möglich an einen Zielwert (idealerweise 1) zu bringen. Wie viele andere Treemap-Algorithmen erfolgt die Aufteilung vom Wurzelknoten her, indem die Fläche sukzessive in kleinere Teilflächen untergliedert wird, bis auf Blattknotenebene.

Im Folgenden wird der Algorithmus anhand eines Beispiels aus dem Originalartikel von Bruls et al. [BHVW00, S. 5] erläutert, wobei die Erklärung näher an einer praktischen Implementierung aus der d3-Bibliothek [D3ta] ausgerichtet ist.

Der Algorithmus füllt die zur Verfügung stehende Fläche stets in Reihen auf, wobei in jeder Reihe möglichst quadratische Rechtecke entstehen sollen. Das Einfügen erfolgt dabei iterativ: Das erste Rechteck wird eingefügt, dann das aktuelle Seitenverhältnis des Rechtecks berechnet. Dann wird das nächste Rechteck eingefügt. Wenn durch das Einfügen das Seitenverhältnis eines Rechtecks der Reihe schlechter wird, wird eine neue Reihe eröffnet.

In dieser Arbeit wird (abweichend von einigen Publikationen) die X-Koordinate als Breite und die Y-Koordinate als Länge bezeichnet, um Verwechslungen mit der Z-Komponente (Höhe im dreidimensionalen Raum) zu vermeiden.

Im Folgenden wird der Algorithmus zur Anordnung von Rechtecken anhand eines konkreten Beispiels erläutert. Es sollen Rechtecke mit den Größen 6, 6, 4, 3, 2, 2, 1 in ein übergeordnetes Rechteck mit den Abmessungen  $6 \times 4$  einsortiert werden.

Da das Rechteck, in welches eingefügt wird, breiter als hoch ist (Breite 6, Höhe 4), werden die Rechtecke einer vertikalen Reihe platziert. Diese vertikale Reihe, hat immer die Höhe 4 und kann nach rechts in der Breite variieren (siehe das linke große Rechteck in Abbildung 2.6). Diese Reihe wird solange ergänzt, bis das schlechteste Seitenverhältnis (also das ungünstigste der enthaltenen Rechtecke) schlechter ist, als das schlechteste Seitenverhältnis im vorigen Schritt.

Zunächst wird in Schritt 1 das erste Rechteck mit einer Fläche von 6 Einheiten in die noch leere Reihe eingefügt. Das Seitenverhältnis dieses Rechtecks beträgt  $1,5 \times 4$  (1,5 Einheiten breit und 4 Einheiten lang). Das schlechteste Seitenverhältnis aller Rechtecke in der aktuellen Reihe ist trivialerweise das des einzigen Rechtecks aktuell, also  $4 : 1,5 \approx 2,67$  (wir teilen per Definition immer den größeren Wert durch den kleineren, dadurch ist 1 der optimale

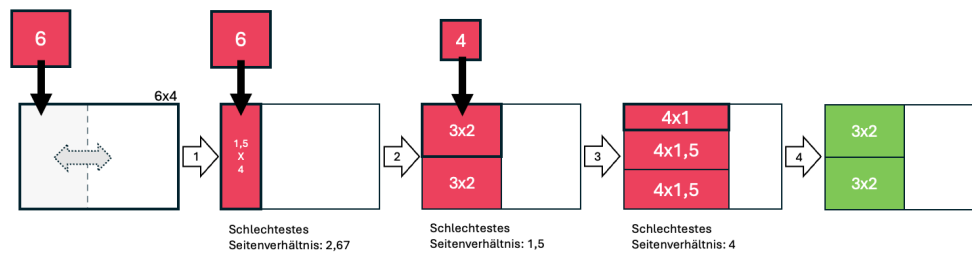


Abbildung 2.6: Beispiel für die erste Reihe des Squarify-Algorithmus. Die Reihe ist im linken großen Rechteck in hellgrau dargestellt, die kann nach rechts in der Breite variieren. Rot sind die Rechtecke, die in ihrer Breite und Höhe noch nicht festgelegt sind. Grün sind die Rechtecke, bei denen die Breite und Höhe neu festgelegt wurde.

Wert, an den man sich von oben annähert, dadurch werden die vergleiche einfacher). In Schritt zwei wird das nächste Rechteck mit einer Fläche von 6 Einheiten in die Reihe *von oben* über das erste Rechteck eingefügt. Dadurch wird die Reihe und auch das erste Rechteck breiter. In diesem Fall haben beide Rechtecke das gleiche Seitenverhältnis von  $3 \times 2$  (3 Einheiten breit und 2 Einheiten lang). Das schlechteste Seitenverhältnis in der Reihe ist nun als  $3 : 2 = 1,5$  zu berechnen. Das neue schlechteste Seitenverhältnis ist also besser als das vorherige, sodass die Reihe weiter gefüllt werden kann. Im nächsten Schritt soll ein Rechteck der Fläche 4 Einheiten eingefügt werden. Das hinzufügen dieses Rechtecks macht die Reihe erneut breiter. Es entsteht ein neues Schlechtestes Seitenverhältnis von  $4 : 1 = 4$ . Da das Verhältnis nun schlechter ist als das vorherige, wird die Reihe, so wie sie zuvor war, als abgeschlossen betrachtet. Das zuletzt eingefügte Rechteck (mit Fläche 4) wird daher nicht mehr in dieser Reihe, sondern in einer neuen Reihe einsortiert (siehe Schritt 4).

Nachdem nun die erste Reihe abgeschlossen ist, wird eine neue Reihe eröffnet (siehe Abbildung 2.7). Das durch die zuvor erstellte Reihe verbliebene, zu füllende Rechteck hat die Abmessungen  $3 \times 4$  (3 Einheiten breit und 4 Einheiten lang). Da die Reihe länger als breit ist. Durch das Einfügen des Rechtecks entsteht ein schlechtestes Seitenverhältnis von  $3 : 1,33 \approx 2,25$ . Das Einfügen des nächsten Rechtecks mit der Fläche 3 Einheiten verbessert das schlechteste Seitenverhältnis auf  $2,33 : 1,29 \approx 1,8$  und ist damit besser als das vorherige. Das Einfügen des nächsten Rechtecks mit der Fläche 2 verschlechtert das schlechteste Seitenverhältnis auf  $3 : 0,67 \approx 4,5$ , sodass die Reihe abgeschlossen wird (Schritt 8).

Um den Ablauf besser zu verstehen ist unter diesem [Link \[Meh25b\]](#) ein Video zu erreichen, das für den algorithmus das Füllen der ersten zwei Reihen zeigt. Es ist zu erkennen, wie das einfügen der Rechtecke die Reihen breiter macht und wie das letzte Rechteck in der Reihe entfernt wird, wenn es das schlechteste Seitenverhältnis verschlechtert.

Dieser Prozess wird so lange fortgesetzt, bis alle Rechtecke in Reihen einsortiert sind (siehe Abbildung 2.8).

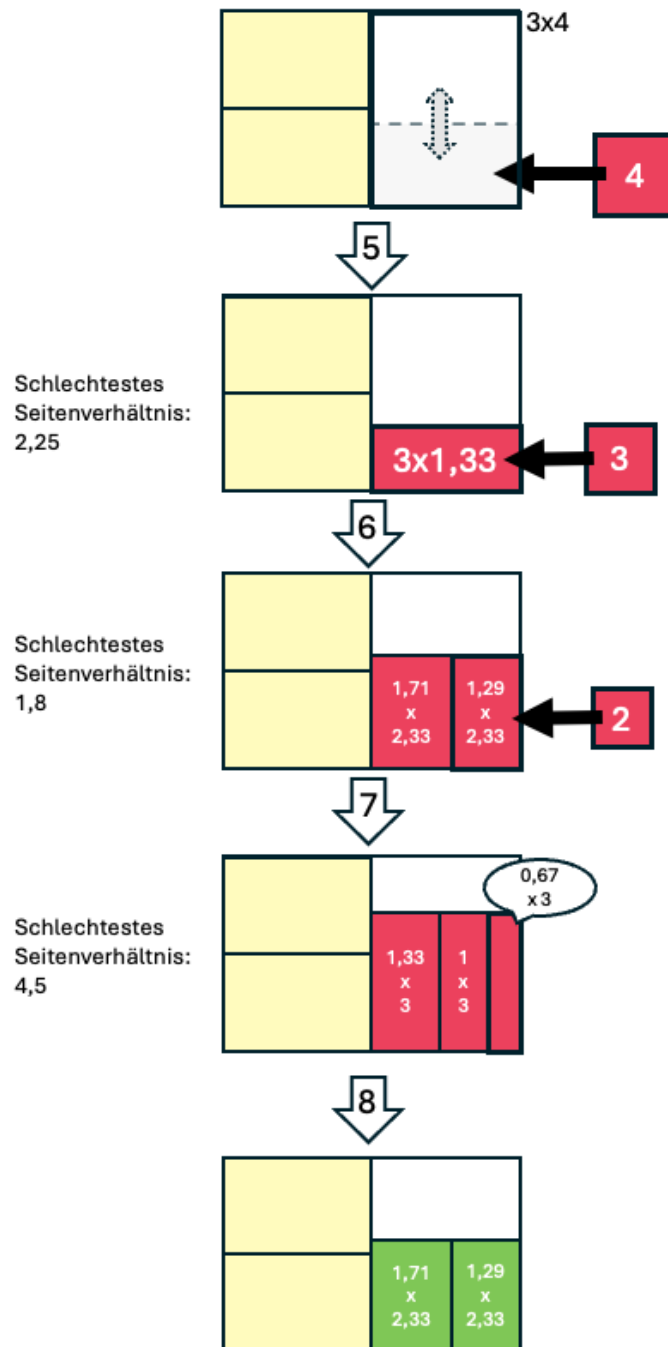


Abbildung 2.7: Beispiel für die zweite Reihe des Squarify-Algorithmus. Die Reihe ist im linken großen Rechteck in hellgrau dargestellt, die kann nach rechts in der Breite variieren. Rot sind die Rechtecke, die in ihrer Breite und Höhe noch nicht festgelegt sind. Gelb sind die Rechtecke die teil von zuvor abgeschlossenen Reihen sind. Grün sind die Rechtecke, bei denen die Breite und Höhe neu festgelegt wurde.

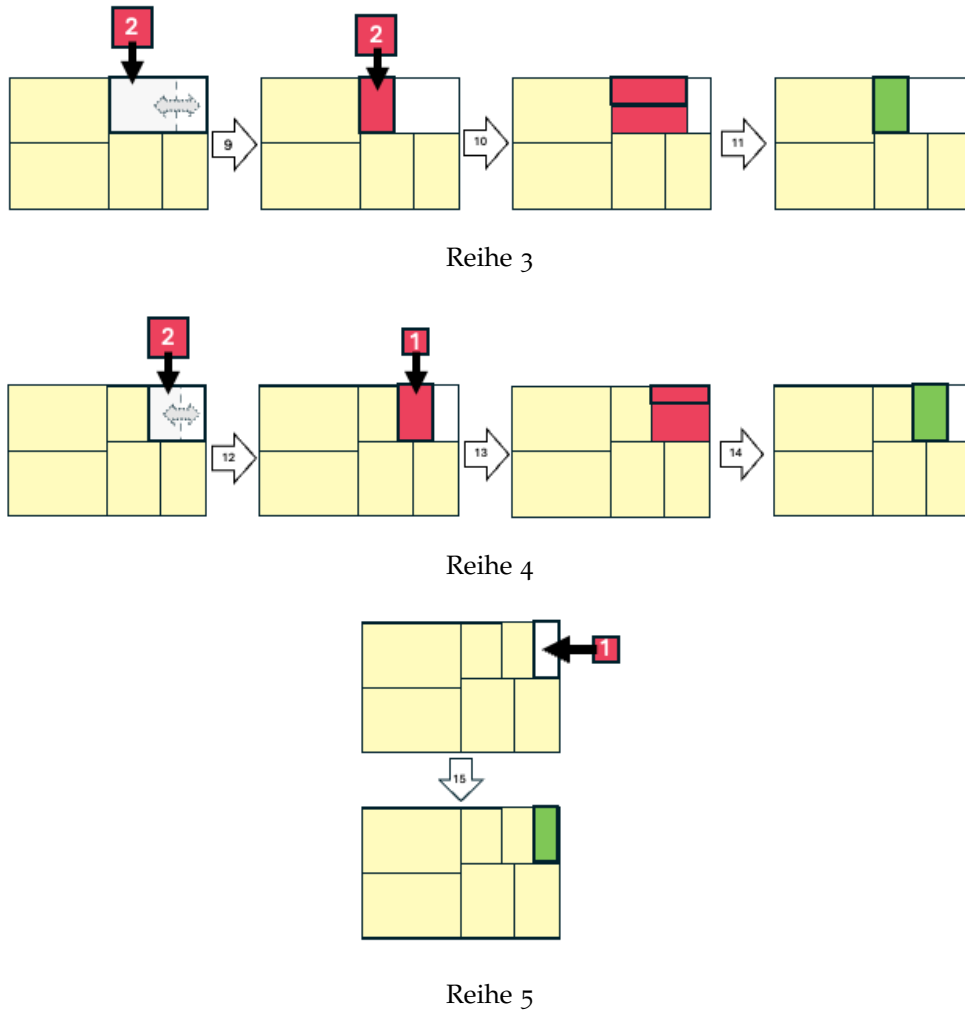


Abbildung 2.8: Beispiel für die letzten Reihen des Squarify-Algorithmus. Die Reihen sind im linken großen Rechteck in hellgrau dargestellt, die kann nach rechts in der Breite variieren. Rot sind die Rechtecke, die in ihrer Breite und Höhe noch nicht festgelegt sind. Gelb sind die Rechtecke die teil von zuvor abgeschlossenen Reihen sind. Grün sind die Rechtecke, bei denen die Breite und Höhe neu festgelegt wurde.

Der Schritt, das jeweils schlechteste Seitenverhältnis in einer Reihe zu bestimmen, lässt sich rechnerisch effizient gestalten. Für ein Rechteck mit Fläche  $w_i$  in einer Reihe mit konstanter Länge  $l$  und Breite  $w$  sowie Gesamtfläche  $sV$  innerhalb der Reihe kann das maximale Seitenverhältnis durch folgende Umformungen vereinfacht berechnet werden:

$$\frac{w_i}{l_i} = \frac{w_i \cdot l_i \cdot w^2}{l_i \cdot l_i \cdot w^2} \quad (2.1)$$

$$= \frac{w_i \cdot l_i \cdot w^2}{l_i^2 \cdot \left(\sum_{j=0}^n w_j\right)^2} \quad (2.2)$$

$$= \frac{w_i \cdot l_i \cdot w^2}{\left(l_i \cdot \sum_{j=0}^n w_j\right)^2} \quad (2.3)$$

$$= \frac{w_i \cdot l_i \cdot w^2}{\left(\sum_{j=0}^n l_i \cdot w_j\right)^2} \quad (2.4)$$

$$= \frac{w_i \cdot l_i \cdot w^2}{\left(\sum_{j=0}^n l_j \cdot w_j\right)^2} \quad \text{da } \forall i, j \in \{0, \dots, n\}, l_i = l_j \quad (2.5)$$

$$= \frac{V_i \cdot w^2}{sV^2} \quad (2.6)$$

Ebenso gilt analog  $\frac{l_i}{w_i} = \frac{sV^2}{V_i \cdot w^2}$ . Es genügt daher, für das jeweils größte und kleinste Rechteck in der Reihe den Wert zu berechnen und daraus das schlechteste Seitenverhältnis zu nehmen.

Während der Füllung einer Reihe bleibt  $w$  konstant und muss daher nur einmal berechnet werden;  $sV$  wird bei jedem neuen Rechteck in der Reihe aktualisiert.

Um den Algorithmus noch besser zu verstehen, hängen wir folgend den code an, so wie wir in in dieser Arbeit als Grundlage für die Erweiterungen verwendet haben. Der Code ist in TypeScript geschrieben. Als Vorlage für diese Implementierung diente die JavaScript-Bibliothek d3.js [D3tb].

```

1 function squarifyNode(parent: SquarifyNode) {
2   // nodes sind die einzusortierenden Knoten
3   const nodes: SquarifyNode[] = parent.children
4   let row: SquarifyRow,
5       x0 = parent.x0,
6       y0 = parent.y0,
7       i = 0,
8       j = 0,
9       numberOfChildren = nodes.length,
10      width: number,
11      length: number,
12      value = parent.value,
13      sumValue: number,
14      minValue: number,
15      maxValue: number,

```

```

16     newRatio: number,
17     minRatio: number,
18     alpha: number,
19     beta: number
20
21     // i ist der Index des aktuellen Knotens, der einsortiert wird
22     while (i < numberOfChildren) {
23         // Ein Schleifendurchlauf entspricht einer Reihe von Knoten
24         // Breite und Länge des noch zu belegenden Bereichs müssen nach
25             jeder Reihe neu berechnet werden
26         width = parent.x1 - x0
27         length = parent.y1 - y0
28
29         // Suche den nächsten Knoten mit Wert und aktualisiere den
30             sumValue
31         do {
32             sumValue = nodes[j++].value
33         } while (!sumValue && j < numberOfChildren)
34
35         // minValue ist der kleinste Wert in der Reihe
36         // maxValue ist der größte Wert in der Reihe
37         // sumValue ist die Summe der Werte der Knoten in der aktuellen
38             Reihe
39         minValue = maxValue = sumValue
40         // alpha ist der feste Faktor für die Berechnung des
41             Seitenverhältnisses
42         alpha = Math.max(length / width, width / length) / (value *
43             aimedRatio)
44         // beta ist der variable Faktor für die Berechnung des
45             Seitenverhältnisses
46         beta = sumValue * sumValue * alpha
47         // minRatio speichert das aktuelle schlechteste
48             Seitenverhältnis in der Reihe
49         minRatio = Math.max(maxValue / beta, beta / minValue)
50
51         // Füge weitere Knoten hinzu, solange sich das Seitenverhältnis
52             nicht verschlechtert
53         for (; j < numberOfChildren; ++j) {
54             const nodeValue = nodes[j].value
55             sumValue += nodeValue
56             if (nodeValue < minValue) {
57                 minValue = nodeValue
58             }
59             if (nodeValue > maxValue) {
60                 maxValue = nodeValue
61             }
62             beta = sumValue * sumValue * alpha
63             // Berechne das neue schlechteste Seitenverhältnis nach dem
64                 Hinzufügen des Knotens
65             newRatio = Math.max(maxValue / beta, beta / minValue)
66
67             if (newRatio > minRatio) {

```

```

59         // Wenn das Seitenverhältnis schlechter wird, entferne
           den letzten Knoten und beende die Schleife
60         sumValue -= nodeValue
61         break
62     }
63     minRatio = newRatio
64 }
65
66 // Fülle die Reihe mit den ausgewählten Knoten und berechne
   deren Position und Größe
67 row = { name: parent.name, value: sumValue, dice: width <
        length, children: nodes.slice(i, j) }
68 if (row.dice) {
69     treemapDice(row, x0, y0, parent.x1, value ? (y0 += (length
        * sumValue) / value) : y0)
70 } else {
71     treemapSlice(row, x0, y0, value ? (x0 += (width * sumValue)
        / value) : x0, parent.y1)
72 }
73
74 // Der noch zu belegende Bereich wird um die Größe der
   aktuellen Reihe verkleinert
75 value -= sumValue
76 i = j
77 }
78 }

```

Listing 2.1: Vereinfachte und kommentierte Grundlage für den Algorithmus der in dieser Arbeit hauptsächlich verwendet wird

Obwohl sich das ursprüngliche Paper von Bruls et al. [BHVW00] vor allem auf das Seitenverhältnis 1 als Idealwert konzentriert, erlauben viele Implementierungen, wie auch die implementierungen von d3.js [D3ta] auch die Annäherung an andere Zielwerte, beispielsweise an den Goldenen Schnitt [Lu+17]. In Abbildung 2.9 ist ein Beispiel für ein Layout zu sehen, dass durch den Squarify-Algorithmus generiert wurde, wobei die Rechtecke möglichst quadratisch werden sollen, so wie beim Original-Algorithmus von Bruls et al. [BHVW00]. In Abbildung 2.10 sind die selben Werte zu sehen. Day layout wurde auch durch den Squarify-Algorithmus generiert, jedoch mit dem Ziel, dass die Rechtecke ein Seitenverhältnis von 5 anstreben. Es ist ein deutlicher Unterschied in den Darstellungen zu erkennen: Die Rechtecke sind deutlich langgestreckter, als im ersten Beispiel.





Abbildung 2.9: Beispiel für ein Squarify-Layout mit Annäherung an quadratische Rechtecke (durchschnittliches Seitenverhältnis 1,42)



Abbildung 2.10: Beispiel für ein Squarify-Layout mit Annäherung an den Wert 5 (durchschnittliches Seitenverhältnis 2,79)

## PROBLEMSTELLUNG

---

Die zentrale Herausforderung bei der Entwicklung von Stadtmetaphern zur Softwarevisualisierung besteht in der Festlegung eines geeigneten Layouts für die Visualisierung. Im Abschnitt 2.3 wurde das ursprüngliche Layout-Konzept der CodeCity-Analogie vorgestellt. Ein gemeinsames Grundprinzip dieser und verwandter Arbeiten ist, dass zunächst ein 2D-Layout erzeugt wird, das anschließend durch eine zusätzliche Metrik in die dritte Dimension extrudiert und schließlich durch weitere Metriken (z.B. Farbe, Transparenz) angereichert wird. In dieser Arbeit werden wir uns genau auf dieses Grundprinzip konzentrieren und untersuchen, wie sich verschiedene Layout-Algorithmen auf die Visualisierung von Code-Qualitätsmetriken auswirken. Die daraus entstehende Leitfrage lautet:

Welches Layout eignet sich am besten zur Visualisierung von Code-Qualitätsmetriken? Gibt es Ansätze, die Übersichtlichkeit und Verständlichkeit im Vergleich zu bestehenden Lösungen (insbesondere der CodeCity-Metapher) verbessern?

Daraus ergeben sich für diese Arbeit folgende konkrete Forschungsfragen:

1. Wie lassen sich bestehende Treemap-Layout-Algorithmen an das spezifische Anwendungsproblem dieser Arbeit anpassen? Welche Vor- und Nachteile resultieren daraus?
2. Lassen sich durch Analyse verwandter Arbeiten und bestehender Tools alternative Layouts identifizieren, die eine bessere Grundlage für die Visualisierung von Code-Qualitätsmetriken bieten?
  - a) Beispielhaft: Übertrifft das Sunburst-Layout das beste Treemap-Layout im Hinblick auf die Darstellung von Code-Qualitätsmetriken?
  - b) Beispielhaft: Bietet das Stack-Layout Vorteile gegenüber klassischen Treemap-Layouts für diesen Anwendungszweck?

Um die angestrebten Visualisierungen eindeutiger zu evaluieren und in ihrem Ziel klarer zu definieren, orientieren wir uns an den fünf Dimensionen der Softwarevisualisierung nach Marcus Adrian et al. [MFMo3, S. 2]:

- **Tasks:** Warum wird die Visualisierung benötigt?
- **Audience:** Wer nutzt die Visualisierung?
- **Target:** Welche Datenbasis/Quelle wird abgebildet?
- **Representation:** Wie wird die Information dargestellt?
- **Medium:** Wo findet die Darstellung statt?

### 1. Warum ist die Visualisierung von Code-Qualitätsmetriken wichtig?

Die Visualisierung von Code-Qualitätsmetriken ermöglicht die Bewertung und das bessere Verständnis der Softwarequalität. Sie dient dazu, einen schnellen Überblick über komplexe Codebasen zu erhalten, Hotspots zu identifizieren und vertiefende Analysen effizient einzuleiten. Besonders wichtig ist es dabei, Zusammenhänge zwischen verschiedenen Metriken auf einen Blick erfassbar zu machen, was in rein tabellarischer oder isolierter Darstellung kaum möglich ist. Das Hauptziel besteht darin, das abstrakte Konzept der Codequalität *greifbar* und anschaulich zu machen.

### 2. Wer ist die Zielgruppe der Visualisierung?

Die Visualisierung richtet sich primär an Personen ohne tiefgehende Kenntnisse der Codebasis und ohne Programmierkenntnisse. Trotzdem kann die Visualisierung aber auch Entwicklerinnen und Entwicklern helfen, die sich neu in ein Projekt einarbeiten oder Teams, die systematisch die Softwarequalität verbessern wollen, in der Kommunikation unterstützen.

### 3. Was ist die Datenquelle?

Grundlage sind hierarchische Code-Qualitätsmetriken, die automatisiert aus dem Source Code extrahiert werden. Wobei die Metriken auf File-Basis erstellt werden. Dieser vorschlag stammt ursprünglich von Code City. Er bietet eine gute granularität, um die Struktur und die Qualität von Software zu beurteilen [WLo7]. Jeder Knoten der Hierarchie (Node) besitzt dabei einen Namen und entweder eine Liste von Kindknoten (children) oder einen Wert (value), der die Metrik repräsentiert (siehe Listing 3.1). Diese Struktur ermöglicht es, komplexe Softwareprojekte in übersichtliche Hierarchien zu zerlegen, die dann visualisiert werden können.

```
"node": {
  "name": string,
  "children": List[Node] | "value": number
}
```

Listing 3.1: Schema einer Node

Genauer werden wir in Abschnitt 6 auf die Eingabedaten eingehen.

### 4. In welchem Medium soll die Visualisierung erfolgen?

Die Visualisierung ist für die digitale Darstellung auf herkömmlichen Bildschirmmedien konzipiert. Im vorliegenden Prototyp wird eine Umsetzung im Webbrowser mittels TypeScript und Three.js realisiert. Die Übertragung der Methoden auf andere Programmierumgebungen ist möglich.

### 5. Wie erfolgt die Darstellung?

Die Darstellungen in der es in dieser Arbeit geht ist eine 2.5D-Darstellung, die als Grundlage ein 2D-Layout hat. Dieses 2D-Layout kann beliebig aussehen, mit Formen, Platzierungen, Abständen, etc. Dabei haben alle Layouts ein gemeinsames Schema (siehe Listing 3.2).

```
"node": {
  "name": string,
  "x": number,
  "y": number,
  "width": number,
  "length": number,
  "children": List[Node] | "value": number
}
```

Listing 3.2: Schema einer Node

Dieses Schema ermöglicht es, die Position und Größe jedes Knotens im 2D-Raum zu definieren. Die dritte Dimension wird durch eine zusätzliche Metrik extrudiert. Dabei muss diese Extrusion nicht zwingend linear sein. Zum Beispiel muss ein Kreis nicht zwingend zu einem Zylinder extrudiert werden, sondern kann auch zu einem Kegel oder einer Kugel extrudiert werden. Dadurch erhalten wir eine eindeutige Definition und Anforderung an 2D-Layouts und sind gleichzeitig flexibel in der Darstellung der dritten Dimension.

Im Mittelpunkt dieser Arbeit steht das zweidimensionale Layout der Knoten. Die Gestaltung dieses Layouts wird jedoch eingeschränkt, dadurch, dass Fläche, Höhe und Farbgebung (auch Schattierung, Struktur oder Transparenz) zur Darstellung verschiedener Metriken dienen. Maßnahmen wie das Einfärben von verschiedenen Ordnern oder Modulen, zur besseren visuellen Unterscheidung fallen für die Visualisierung in dieser Arbeit weg (bis auf wenige Ausnahmen - siehe Abschnitt 7.6.3). Stattdessen wird der Fokus auf die geschickte Platzierung, Wahl von Abständen und Beschriftung gelegt.

Aus dem hier definierten Rahmen und den in den Grundlagen beschriebenen Grundlegenden Anforderungen (Abschnitt 2.2.1) leiten sich für die, im Rahmen dieser Arbeit entwickelte, Visualisierung folgende Anforderungen ab:

- **Informationsgehalt und effiziente Nutzung des Platzes:** Möglichst viel relevante Information soll bei minimalem Flächenverbrauch dargestellt werden.
- **Niedrige visuelle Komplexität und hohe Verständlichkeit:** Das Layout muss klar und intuitiv lesbar bleiben, um Überforderung zu vermeiden.
- **Skalierbarkeit:** Auch umfangreiche Softwaresysteme müssen übersichtlich visualisierbar sein.

- **Korrelation mit dem Code:** Eine gute Zuordnung zwischen Visualisierung und Source Code muss gewährleistet sein, um Rückschlüsse auf die Softwarestruktur zu ermöglichen.
- **Stabilität gegenüber Änderungen:** Nachrangig, aber erwünscht ist, dass die Visualisierung auf Änderungen im Code robust reagiert, um zeitliche Vergleiche zu ermöglichen, ohne dass der Betrachter sich neu orientieren muss.

Wir gehen hier bewusst nicht auf die Interaktionsmöglichkeiten des Nutzers mit dem Layout ein, da dies den Rahmen dieser Arbeit sprengen würde und die Vergleichbarkeit der Layouts erschwert, da die Interaktion stark vom Layout selbst abhängt. Wir wollen hier zunächst nur die Layouts vergleichen und nicht die Interaktion mit den Layouts.

### 3.1 Das Treemap-Problem

Durch die nähere Erläuterung des Treemap-Algorithmus (siehe Abschnitt 2.4) wurde gezeigt, dass klassische Treemap-Algorithmen grundlegende Schwächen aufweisen, insbesondere wenn Abstände (*Margins*) zwischen den Knoten dargestellt werden sollen. Das Grundproblem, das von Johnson und Shneiderman [JS91] adressiert wurde, ist bereits ohne Abstände NP-Hard [BHVW00, S. 3], und zusätzliche Abstände verkomplizieren die Problematik noch weiter.

Wesentliche Schwierigkeiten bei der Integration von Abständen in das Layout ergeben sich durch:

- Die abgezogenen Flächen für Abstände führen dazu, dass die verbleibende Knotenfläche nicht mehr proportional zum Wert des Knotens ist. Dies ist nicht nur eine Schwierigkeit, sondern tatsächlich unmöglich zu erfüllen. Ein Ordner, der genau ein Kindknoten hat, kann nicht gleichzeitig eine Fläche proportional zu seinem Wert haben, während sein Kindknoten eine Fläche proportional zu seinem Wert hat, da der Ordner durch die Abstände zwingend eine größere Fläche als sein Kindknoten haben muss, während er aber eigentlich den selben Wert hat.
- Bei kleinen Knoten können durch Abzüge von Minimalabständen einzelne Knoten ganz verschwinden, wenn ihre minimale Länge oder Breite kleiner als der geforderte Abstand wird. Knoten sind also nicht *zu klein* um gesehen zu werden, sondern sie verschwinden wirklich komplett.

In den Abbildungen 3.1, 3.2 und 3.3 sind verschiedene Treemap-Layouts zu sehen, die mithilfe des Squarify-Algorithmus (siehe Abschnitt 2.4.1) erzeugt wurden. Die zugrundeliegenden Strukturen und Metriken wurden dabei händisch erstellt, um typische Probleme bei der Flächenproportionalität und Sichtbarkeit in Treemaps zu verdeutlichen (siehe Anhang A.1).

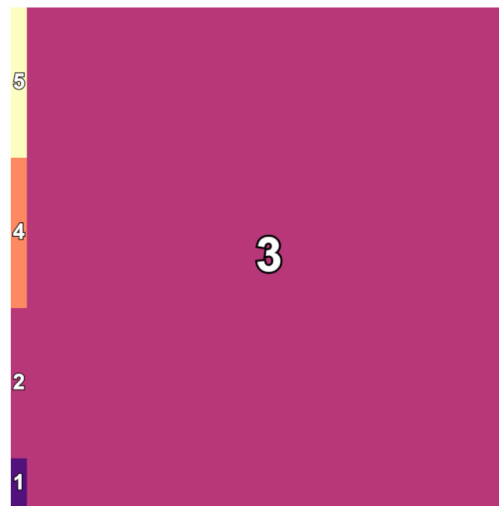


Abbildung 3.1: Treemap-Layout, generiert mit dem Squarify-Algorithmus (Abstand 0).

In Abbildung 3.1 sind alle Knoten sichtbar, und die Flächen aller Knoten sind proportional zu dem jeweiligen Wert. Dadurch ergibt sich eine exakte Flächenproportionalität zwischen Wert und Darstellung.

In Abbildung 3.2 ist zwar weiterhin jeder Knoten sichtbar, jedoch sind die Flächen nicht mehr exakt proportional zu den Werten der Knoten. Beispielsweise besitzt der große Knoten 3 mit dem Wert 3000 nur noch eine Fläche von etwa 2600, was ein Flächen-Wert-Verhältnis von ungefähr 0,9 ergibt. Der kleine Knoten oben links (Knoten 5) mit dem Wert 30 weist hingegen nur noch eine Fläche von etwa 5 auf, entsprechend einem Verhältnis von ca. 0,2. Knoten 2 hat eine Fläche von etwa 17, er erscheint also, obwohl er den selben Wert hat wie Knoten 5, deutlich größer. Dies verdeutlicht, dass die Abstände die Flächenproportionalität erheblich beeinträchtigen.

In Abbildung 3.3, bei einem Abstand von 10, werden einige Knoten, zum Beispiel wie etwa der zuvor oben links gelegene Knoten 5 mit Wert 30, gar nicht mehr dargestellt, da ihre berechnete Breite kleiner als der vorgegebene Abstand ist. Damit sind relevante Informationen in der Visualisierung nicht länger sichtbar.

Diese Beispiele verdeutlichen die Auswirkungen gewählter Abstandsparameter auf Proportionalität und Lesbarkeit von Treemaps.

Das zentrale Dilemma entsteht dadurch, dass die Kernannahme der Treemap-Algorithmen verletzt wird: Die benötigte Fläche aller Knoten einer Hierarchieebene muss vor deren Anordnung bekannt sein. Wird jedoch Fläche für Abstände benötigt, ist vorab unklar, wieviel Raum tatsächlich für jeden Knoten zur Verfügung steht. Die effektive *Belegungsfläche* hängt erheblich von der jeweiligen Anordnung der Knoten ab.

Diese Problematik illustriert Abbildung 3.4. Zwei Rechtecke identischer Fläche (16 Einheiten) benötigen im Layout mit je einem Abstand von 1 eine unterschiedlich große Gesamtfläche, je nachdem, ob sie quadratisch (z.B.

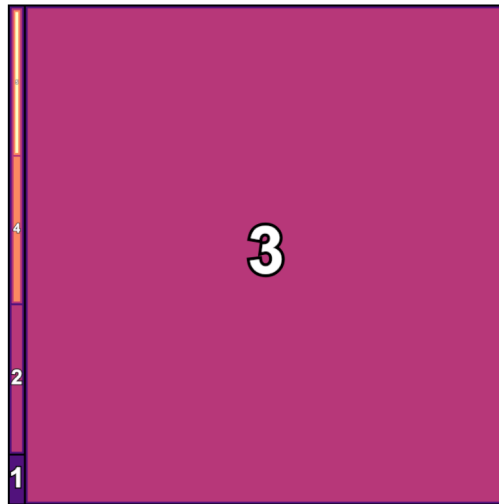


Abbildung 3.2: Treemap-Layout, generiert mit dem Squarify-Algorithmus (Abstand 5).

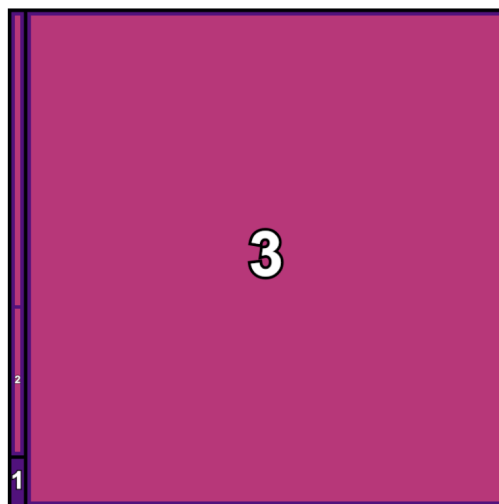


Abbildung 3.3: Treemap-Layout, generiert mit dem Squarify-Algorithmus (Abstand 10).

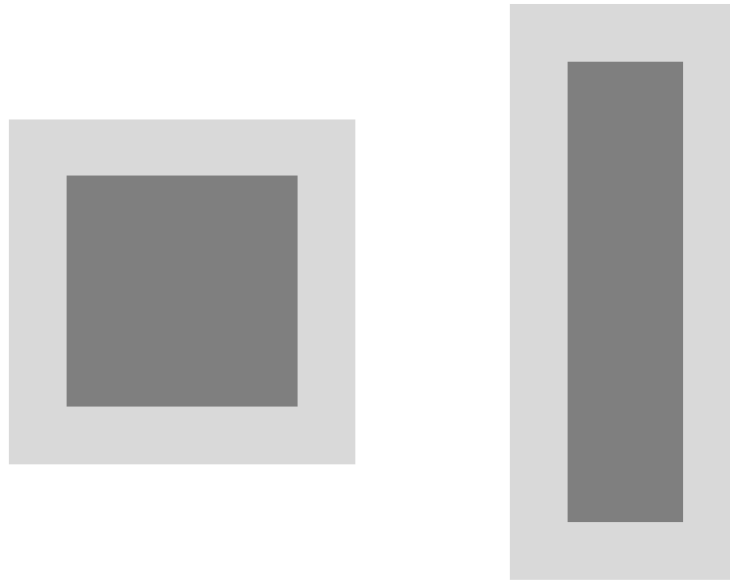


Abbildung 3.4: Zwei Rechtecke der Fläche 16 (dunkelgrau) mit umgebendem Abstand 1 (hellgrau): Links ( $4 \times 4$ ) mit Gesamtfläche 25 ( $5 \times 5$ ), rechts ( $2 \times 8$ ) mit 40 ( $4 \times 10$ ). Der jeweils notwendige Platz für Abstände hängt vom Seitenverhältnis ab.

$4 \times 4$ ) oder länglich (z.B.  $2 \times 8$ ) angeordnet sind; die zusätzliche Fläche für Abstände variiert also erheblich.

Dadurch entsteht ein Zirkelschluss: Das endgültige Layout kann erst berechnet werden, wenn die tatsächlich benötigte Knotenfläche (inklusive Abstände) bekannt ist. Diese Fläche hängt aber wiederum vom Layout ab. Eine triviale Lösung existiert hierfür nicht, weshalb in dieser Arbeit verschiedene Ansätze untersucht werden, um dieses Problem zu lösen (siehe Abschnitt 7).



## VERWANDTE ARBEITEN

---

Im Folgenden werden Forschungsarbeiten vorgestellt, die sich mit Fragestellung beschäftigen, die den beiden zuvor vorgestellten Problemstellungen (siehe Abschnitt 3) ähnlich sind. Zunächst wird ein Überblick über Arbeiten gegeben, die sich mit dem Vergleich oder Vorstellung mehrerer Visualisierungstechniken beschäftigen. Anschließend werden Arbeiten vorgestellt, die sich speziell mit dem Treemap-Problem beschäftigen, also dem Problem von verschwindenden Knoten.

### 4.1 Vergleich von Visualisierungstechniken

Die Arbeit von Pacione et al. (*A comparative evaluation of dynamic visualisation tools* [PRW03]) vergleicht fünf dynamische Visualisierungstools bezüglich ihrer Eignung für das Softwareverständnis und das Reverse Engineering. Die Autoren verfolgen das Ziel, Gründe dafür zu identifizieren, weshalb der Einsatz dieser Werkzeuge außerhalb der Forschung bislang nur begrenzt erfolgt, obwohl sie großes Potenzial für die Analyse komplexer Softwaresysteme bieten. Die Studie macht deutlich, dass keines der untersuchten Tools alle Anforderungen an Softwareverständnis vollständig abdeckt. Insbesondere wird auf Defizite in der Abbildung von Design-Patterns und Hotspot-Analysen hingewiesen, was unter Anderem im Kontext der Evaluation in dieser Arbeit untersucht werden soll.

Einen Überblick im Bereich der dreidimensionalen Softwarevisualisierung gibt die Arbeit von Teyseyre et al. (*An overview of 3D software visualization* [TC08]). Die Autoren präsentieren verschiedene Visualisierungsansätze und Werkzeuge. Dabei werden folgende Kategorien differenziert:

- **Graphbasierte Ansätze:** 3D-Knoten-Kanten-Diagramme mit Fokus auf die Darstellung von Klassenbeziehungen
- **Baumstrukturen:** Dreidimensionale Variationen wie *Cone Trees*, *Information Cubes* (im Grund 3D Treemaps) oder *hierarchical nets* (siehe Abbildung 4.1) zur hierarchischen Darstellung von Softwareelementen
- **Abstrakte Geometrische Darstellungen:** Nutzung von verschiedenen Formen zur Darstellung von verschiedenen Metriken und Strukturen. (Beispiel in Abbildung 4.2)

Sie zeigen auch auf, dass einige Ansätze Metaphern wie Städte oder Landschaften aus der echten Welt nutzen. Interessant ist, dass von den 22 analysierten Tools nur vier für das Management oder Stakeholder ohne spezielle Entwicklungskompetenzen geeignet sind. Von diesen vier visualisiert zudem lediglich eines dieser Werkzeuge tatsächlich den Quellcode. Die anderen drei Tools konzentrieren sich auf die Visualisierung von Anforderungen.

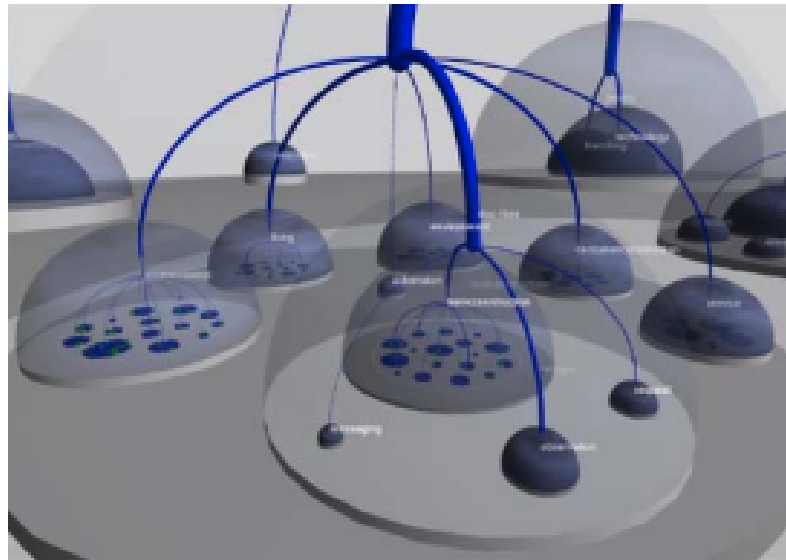


Abbildung 4.1: Beispiel für ein hierarchisches Netz in 3D. Die hierarchische Struktur wird durch Kreise, Halbkugeln und deren Verbindungen dargestellt. [TC08, S. 6]

Zudem wird ein Mangel an empirischen Nutzungsstudien im 3D-Bereich konstatiert, was den Status und die Praxistauglichkeit der Ansätze weiter einschränkt. [TC08] In dieser Arbeit wird daher untersucht, ob die in dieser Arbeit vorgestellten Ansätze auch für Nutzer ohne tiefere technische Kenntnisse geeignet sind.

Auch die Arbeit von Caserta und Zendra (*Visualization of the static aspects of software: A survey* [CZ10]) gibt einen Überblick über etablierte und aktuelle Visualisierungstechniken und ordnet diese unterschiedlichen Analyseebenen zu:

- **Quellcode-Ebene:** Zeilenorientierte Visualisierungen von einzelnen Dateien oder Klassen
- **Klassen-/Mittel-Ebene:** Darstellungen der Strukturen oder Relationen von Dateien oder Klassen
- **Architekturebene:** Globale Übersichten über Organisation und Qualitätsattribute, etwa durch Darstellung von Vererbungen und Aufrufstrukturen

Für die Architekturebene, die im Fokus dieser Arbeit steht, werden insbesondere folgende Visualisierungstypen diskutiert: Tree/Node-Link-Diagramme (wie Bereits in der Grundlagen gezeigt: bei großen Systemen schnell unübersichtlich), Treemaps und deren Varianten (Circular, Sunburst, Voronoi), die hierarchische Strukturen platzsparend repräsentieren und in 2D sowie 3D verfügbar sind. Sie stellen unter anderem folgende Probleme dieser Visualisierungen fest: Schlechte Nutzbarkeit, kognitive Überlastung, Desorientierung in dreidimensionalen Darstellungen sowie eine geringe Verbreitung



Abbildung 4.2: Beispiel für eine abstrakte geometrische Darstellung von Software.  
[TC08, S. 7]

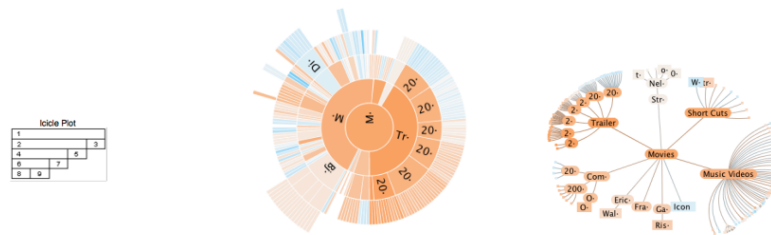


Abbildung 4.3: Beispiele für *Icicle Plots* (links), *Sunburst-Diagramme* (mitte) und *hyperbolische Treelayouts* (rechts). [Kha+12, S. 4]

praxistauglicher Tools, da viele existierende Lösungen im Prototypstatus sind.

Die Übersicht von Khan et al. (*Visualization and evolution of software architectures* [Kha+12]) stellt ebenfalls verschiedene Visualisierungstechniken vor. Sie unterscheiden zwischen hierarchischen und beziehungsorientierten Visualisierungstechniken. Bei den hierarchischen Ansätzen werden platzfüllende Treemaps vorgestellt, die aufgrund ihrer Eigenschaften vor allem für große Hierarchien geeignet sind, wobei Sie hervorheben, dass Einschränkungen hinsichtlich der Visualisierung multipler Metriken und der Anschaulichkeit interner Strukturen bestehen, was in dieser Arbeit angegangen wird. Ergänzend nennen die Autoren auch *Icicle Plots*, *Sunburst-Diagramme* und *hyperbolische Bäume* (siehe Abbildung 4.3).

In *Software visualization tools: survey and analysis* [BK01] von Bassil und Keller führen eine Umfrage durch, in der sie untersuchen, was Nutzern bei Softwarevisualisierungstools wichtig ist. Es zeigte sich, dass insbesondere hierarchische Repräsentationen, Benutzerfreundlichkeit und Skalierbarkeit bei großen Softwaresystemen von den Nutzern als entscheidend bewertet werden. Auch hier ist auffällig, dass speziell Experten befragt werden und sich auch die meisten untersuchten Werkzeuge primär an Experten und weniger an Endanwender wie Softwarekunden oder Management richten.

Eine detaillierte Übersicht verschiedener Treemap-Layout-Algorithmen geben Scheibel et al. in ihrer Arbeit *Survey of treemap layout algorithms* [SLD20]. Die Autoren unterscheiden Treemap-Algorithmen anhand der Art der Aufteilung (packing vs. splitting). Splitting ist die klassische Treemap-Variante, bei der die Fläche geteilt wird. Packing-Ansätze hingegen platzieren die Rechtecke so, dass sie möglichst wenig Platz verschwenden. Beide Ansätze erzeugen leicht verschiedene Layouts (siehe Abbildung 4.4).

Zudem unterscheiden sie verschiedene layouts anhand der Layout-Form (z.,B. rechteckig, kreisförmig, konvex, nicht-konvex) sowie der Referenzraum-Dimension (2D vs. 3D). Für die in dieser Arbeit betrachteten Anwendungsfälle stehen rechteckige 2D-Splitting-Layouts im Vordergrund (siehe Abschnitt 3 und 2.4). Wir stellen fest, dass von 81 analysierten Ansätzen sind 54 rechteckige und 58 splitting-basiert sind, was zeigt, dass dies das präferierte Treemap-Verfahren ist. Neben klassischen Verfahren (Slice-and-Dice, Squarified, Strip) werden auch Polygon-, Voronoi- und Circular-Treemaps sowie spezialisierte Layouts beschrieben, etwa für Aspektverhältnis-Optimierung oder

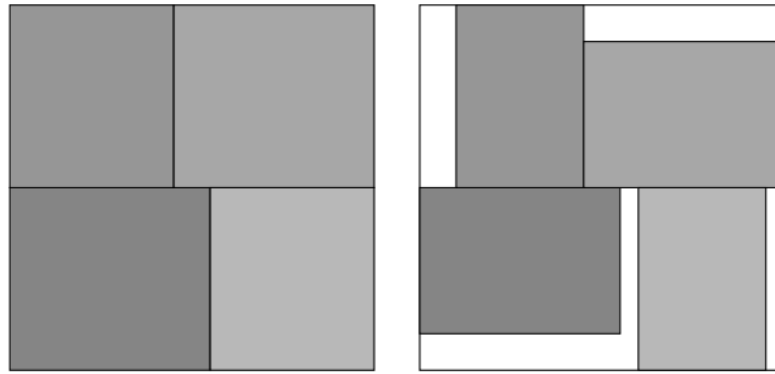


Abbildung 4.4: Beispiel für Layouts die durch Splitting (links) und Packing (rechts) Algorithmen erzeugt wurden. Es ist zu erkennen, dass bei Packing-Algorithmen mehr Platz zwischen den Knoten entsteht. [SLD20, S. 3]

Ordnungserhaltung. Die große Vielfalt verdeutlicht die Notwendigkeit einer gezielten anwendungsspezifischen Auswahl des Algorithmus.

Die analysierten Arbeiten verdeutlichen eine große Vielfalt an Visualisierungsansätzen für Softwaresysteme. Diese Vielfalt unterstreicht die Notwendigkeit eines vergleichenden Ansatzes, um die Stärken und Schwächen unterschiedlicher Techniken systematisch herauszuarbeiten. Zugleich wird in der Literatur fortwährend eine Diskrepanz zwischen den in der Forschung entwickelten Konzepten und deren tatsächlichem Einsatz in der Praxis festgestellt, da viele der vorgestellten Ansätze prototypischer Natur oder stark theoretisch ausgerichtet sind[TC08; CZ10].

Darüber hinaus mangelt es an umfassenden empirischen Studien zur Evaluation dieser Ansätze. In einigen Arbeiten werden neue Visualisierungsmethoden beschrieben, ohne deren Wirksamkeit im Vergleich zu etablierten Methoden systematisch zu untersuchen [TC08; SLD20; Kha+12]. Um die Praxisrelevanz zu erhöhen, berücksichtigt die vorliegende Arbeit auch existierende, in realen Kontexten eingesetzte Werkzeuge und legt einen besonderen Fokus auf die Bedürfnisse von Endnutzern und Stakeholdern, die in bisherigen Arbeiten häufig nur unzureichend adressiert wurden[BK01; PRW03].

## 4.2 Das Treemap-Problem

Lü und Fogarty identifizieren in ihrer Arbeit [LF08] als zentrales Problem von Treemaps die erschwerte Wahrnehmbarkeit der zugrundeliegenden Hierarchiestruktur: “An important limitation of treemaps is the difficulty of discerning the structure of a hierarchy” [LF08, S. 1]. Dieses Problem wird auch in anderen Arbeiten als eine der größten Herausforderungen für algorithmische Treemap-Layouts genannt (siehe Abschnitt 3.1). Verschiedene Ansätze in der Literatur beschäftigen sich mit diesem Problem.

Die Autoren des ursprünglichen Squarified-Treemap-Algorithmus [BHVW00] entwickeln in einer späteren Arbeit das Konzept der *Cushion Treemaps* [VWW99]. Dabei wird jedem Knoten ein Innenraumschatten zugewiesen, welcher durch



Abbildung 4.5: Beispiel für eine Cushion Treemap [VWW99, S. 4].

gezielte Schattierungseffekte die hierarchische Struktur visuell hervorhebt (siehe Abbildung 4.5). Auf diese Weise bleibt das klassische Layout erhalten, während die Wahrnehmung der Hierarchiebeziehungen unterstützt wird.

Ein alternativer Ansatz wird von Demian und Fruchter präsentiert, indem sie die Verwendung unterschiedlich dicker Umrisslinien vorschlagen, um die Zugehörigkeit und Tiefe der Knoten in der Hierarchie hervorzuheben [DFo6]. Ähnliches schlagen auch Kong et al. vor, die die Vor- und Nachteile verschiedener Umriss-Stärken für die Darstellung der Knotenstruktur vorstellen [KHA10] (siehe Abbildung 4.6).

Obwohl diese Ansätze die Erkennbarkeit der Struktur in 2D signifikant verbessern, sind sie für Anwendungen im 2.5D- oder 3D-Kontext wenig geeignet, da Schatten und Outlines bei einer Extrusion ins Dreidimensionale nicht konsistent und flächenübergreifend darstellbar sind. Dies führt dazu, dass insbesondere bei 3D-Treemaps (Extrusion ohne zusätzliche Farbgebung) die hierarchische Struktur nur schwer erkannt werden kann. Daher besteht Bedarf an Layout-Algorithmen, die explizit räumliche Abstände zwischen den Hierarchieebenen erzeugen. Klassische Treemap-Layouts verzichten in der Regel auf solche Abstände, was die Lesbarkeit der Hierarchie in extrudierten Darstellungsformen weiter erschwert [WDo8; SWo1; Hu+14].

Eine explizite Behandlung dieses Problems findet sich in der Arbeit *Cascaded treemaps: examining the visibility and stability of structure in treemaps* [LFo8] von Lü und Fogarty, die im Folgenden detailliert dargestellt wird. Die Arbeit von Lü und Fogarty bildet außerdem eine idee-gebende Grundlage für die in dieser Arbeit vorgenommenen Anpassungen am Squarified-Treemap-Algorithmus.

Im Gegensatz zu verschachtelten (*Nested*) Ansätzen, bei denen Kindknoten streng innerhalb der Elternelemente platziert werden, schlagen Lü und Fogarty eine leichte Versetzung der Kindknoten vor. Diese werden im so-

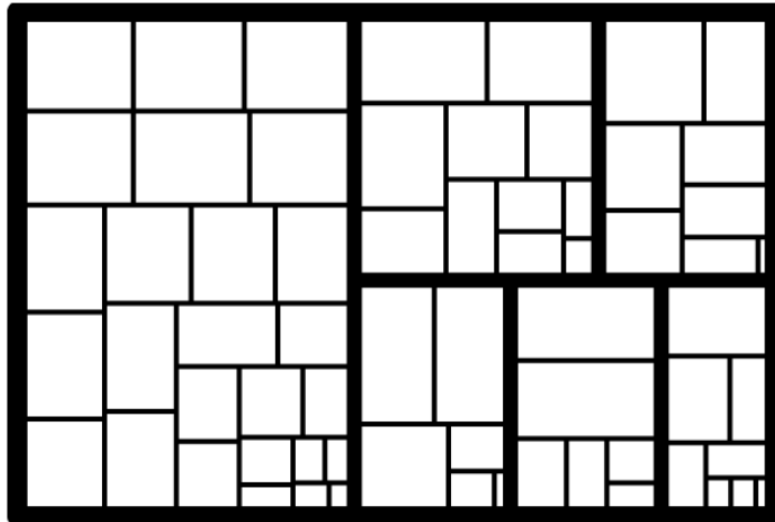


Abbildung 4.6: Beispiel für eine Treemap mit unterschiedlich dicken Outlines zur Verdeutlichung der Hierarchiestruktur der Knoten [KHA10, S. 1].

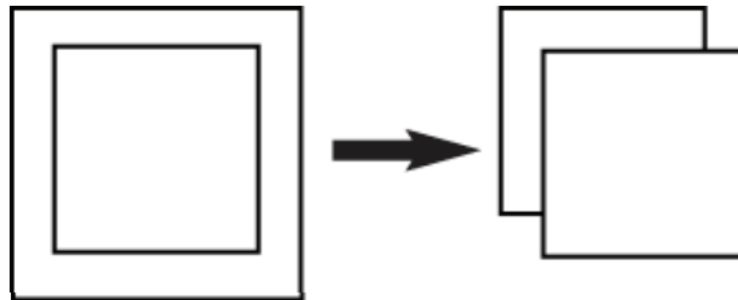


Abbildung 4.7: Links: Nested-Treemap-Ansatz mit vollständig platzierter Verschachtelung. Rechts: Der Cascaded-Treemap-Ansatz nach [LFo8, S. 3] mit versetzten Kindknoten.

genannten *Cascaded Treemap*-Ansatz nicht direkt innerhalb, sondern leicht versetzt über ihren Elternknoten dargestellt (siehe Abbildung 4.7). Dadurch wird ein subtiler 3D-Effekt erzeugt und der Platzverlust im Vergleich zu klassischen Verschachtelungen reduziert.

Ein identifiziertes Problem bei diesem Verfahren (und auch bei klassischen Nested Layouts) besteht darin, dass Knoten vollständig verschwinden können, wenn nicht ausreichend Platz für Abstände und Beschriftungen zur Verfügung steht. Um dieses Problem abzumildern, schlagen Lü und Fogarty einen zweistufigen Prozess vor: Im ersten Schritt erfolgt eine Flächenaufteilung mittels des Squarified-Treemap-Algorithmus [BHVWoo]. Im anschließenden zweiten Schritt werden für die einzelnen Knoten Abstände und Beschriftungsflächen neu berechnet, wobei die Größe, nicht jedoch die Position der Knoten angepasst wird. Dabei kann es, trotz dieses Verfahrens, weiterhin zum Verschwinden von Knoten kommen, wenn nicht genügend Raum für Beschriftung und Abstand vorhanden ist. Einen Pseudocode für die genaue



Implementierung liefern die Autoren nicht, sodass eine Nachvollziehbarkeit und ein quantitativer Vergleich zu alternativen Algorithmen erschwert wird.

Das Verfahren wird wie folgt beschrieben:

The function then computes how much vertical space is needed for offsets [...]. The remaining space is for the content of the tree-map, and so the layout function gives each side of the split the space computed as necessary for [...] offsets as well as a portion of the remaining content space based on the relative weights of nodes on each side of the split. The layout procedure is then ready to recurse on both sides of the split, as it knows how much space will be used by [...] offsets and has ensured that the remaining space is appropriately divided by node weight. [LFo8, S. 6]

Für jeden Knoten wird die verfügbare Fläche jeweils neu berechnet. Dabei wird an jeder Teilungskante, der Kante, an der ein Knoten entweder horizontal oder vertikal aufgeteilt wird, die für Abstände benötigte Fläche bestimmt. Es wird also immer eine Reihen betrachtet. Zunächst erfolgt die Berechnung des PLatzes, der für die Abstände entlang dieser Teilungskante notwendig ist. Im Anschluss wird der verbleibende Raum für die eigentlichen Knoten berechnet, indem die Fläche für die Abstände von der Gesamtfläche subtrahiert wird. Auf diese Weise wird sichergestellt, dass für jeden Knoten ausreichend Platz für die erforderlichen Abstände eingeplant wird und die Knoten entsprechend ihrer Gewichtung skaliert sind.

Allerdings bleibt ein zentrales Problem bestehen. Es kann dazu kommen, dass einzelne Knoten vollständig verschwinden, wenn der für die Abstände erforderliche PLatz größer ist als die für die Knoten verbleibende Fläche. Das grundlegende Problem, das zuvor beschrieben wurde (siehe Abschnitt 3.1), ist zwar dadurch weniger groß, aber nicht vollständig gelöst.

Obwohl kein Quellcode für das beschriebene Verfahren vorliegt, lässt sich basierend auf der publizierten Beschreibung ein weiterer Nachteil dieses Ansatzes erkennen: Bei jeder betrachteten Reihen-Kante wird nur der Raum für die Abstände entlang der Richtung berücksichtigt, die senkrecht zu dieser Kante steht. Abbildung 4.8 veranschaulicht, dass der Algorithmus für den Bereich oberhalb und unterhalb der Trennlinie jeweils die gleiche Menge an Abstandfläche vorsieht, da ausschließlich die senkrechte Richtung zur Kante Berücksichtigung findet. Dies führt dazu, dass die in der Abbildung roten Rechtecke zusammen einen wesentlich größeren Bedarf an Abstand aufweisen würden als das gelbe Rechteck alleine. Der Algorithmus differenziert jedoch nicht zwischen diesen verschiedenen Konstellationen, was eine ineffiziente Allokation von Abstandflächen zur Folge haben kann.

Lü und Fogarty berichten, dass mit ihrem Verfahren keine Verbesserung im Verhältnis von Knoten-Gewicht zu Knotengröße festgestellt werden konnte. Verbesserungen traten lediglich bei sehr kleinen Knoten auf, da konstante Abstände bei kleinen Flächen proportional stärker ins Gewicht fallen als bei größeren. Sie führen dies explizit auf ihren spezifischen Ansatz zurück und regen weiterführende Forschung zu diesem Problem an, was wir unter anderem in dieser Arbeit bezwecken. Darüber hinaus vermuten sie, dass die



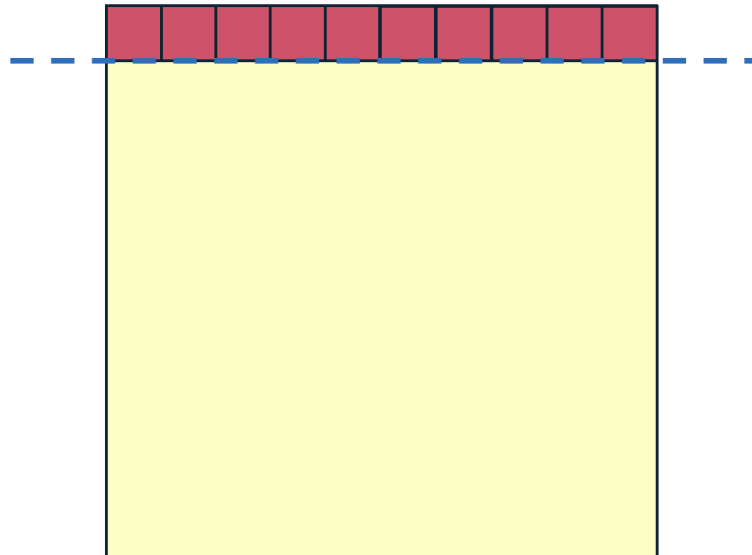


Abbildung 4.8: Illustration einer, für den den Cascaded-Treemap-Algorithmus, ungünstigen Flächenaufteilung. Die Teilungskante ist in blau gekennzeichnet. Senkrecht zu dieser Kante werden die Benötigten Flächen neu berechnet. Für die roten Rechtecke wird zu viel, für das gelbe Rechteck jedoch vergleichsweise wenig Abstand berücksichtigt.

Abweichung zwischen Größen- und Gewichtsverhältnis in tiefen Hierarchien weiter zunimmt.

Das Ziel dieses Abschnitts ist es 2D-Layouts zu finden, die geeignet sind, um extrudiert zu werden und Software-Qualitätsmetriken in Form der in Abschnitt 3 definierten Form in 2.5D zu visualisieren. Alternativ können auch direkt 2.5D Visualisierungen gefunden werden, die geeignet sind, um Software-Qualitätsmetriken wie in der definitern Form zu visualisieren. In diesem Abschnitt stellen wir zunächst die Methodik vor, die wir bei der Recherche verwendet haben. Anschließend stellen wir die Ergebnisse der Recherche vor.

### 5.1 Methodik

Die hier verwendete Methodik orientiert sich an der Methodik der systematischen Literaturrecherche, wie sie in *Procedures for performing systematic reviews*[Kit+04] beschrieben wird, ist aber leicht angepasst, da es vor allem um das finden und einordnen von Layouts geht und nicht direkt um das Bewerten und vergleichen von verschiedenen Arbeiten.

#### 1. Forschungsfrage

Die Problemstellung dieser Arbeit an der hier gearbeitet wird ist bereits definiert:

Lassen sich durch Analyse verwandter Arbeiten und bestehender Tools (zum Treemap Layout) alternative Layouts identifizieren, die eine bessere Grundlage für die Visualisierung von Code-Qualitätsmetriken bieten? (siehe Abschnitt 3)

Die Konkrete Forschungsfrage bzw. das Ziel dieser Rechere leitet sich aus dieser Problemstellung ab: Unsere Recherche hat das Ziel, Layouts zu finden, die geeignet sind, um Software-Qualitätsmetriken in 2.5D zu visualisieren. Wir wollen dabei nicht nur spezifisch nach 2.5D visualierungen suchen, sondern auch nach 2D-Layouts, die geeignet sind, um in 2.5D extrudiert zu werden.

#### 2. Suchstrategie

Wir wollen sowohl wissenschaftliche Arbeiten als auch bestehende Tools und Ansätze finden, die sich mit der Visualisierung von Software-Qualitätsmetriken beschäftigen und analysieren, ob diese Ansätze geeignet sind, um auf unser spezifische Problemstellung übertragen zu werden.

Für die Suche nutzen wir zunächst Google Scholar wir wollen folgende Suche nutzen:

Wir wollen paper suchen, die sich mit der Visualisierung von Software-Qualitätsmetriken beschäftigen. Wir teilen unsere Suche in zwei Teile auf: Der vordere Teil bezieht sich auf die Visualisierung. Und der hintere Teil soll sicher gehen, dass es sich um Software-Qualitätsmetriken handelt, dafür muss auf jeden Fall das Wort software vorkommen und dann entweder "quality" oder "metrics". Beides muss im Paper vorkommen, um in unserer Suche aufzutauchen. Daraus ergibt sich folgende Suchanfrage: ("3D" OR "2.5D" OR "visualization" OR "treemap" OR "hierarchical data") AND (software AND ("quality" OR "metrics"))

Zur Suche nutzen wir die erweiterte Suche von Google Scholar, um möglichst viele relevante Ergebnisse von verschiedenen Journalen und Anbietern zu finden. Das Problem bei der erweiterten Suche von Google Scholar ist, dass sie sehr limitiert ist und keine verschachtelten Suchanfragen unterstützt [Sch23]. Deshalb teilen wir die Suche in zwei Teile auf: "3D" OR "2.5D" OR "visualization" OR "quality" OR "metrics" AND software und "treemap" OR "hierarchical data" OR "quality" OR "metrics" AND software

Um trotzdem die gewünschte Suchanfrage nutzen zu können, um potentiell noch spezifischere und relevantere Ergebnisse zu erhalten, nutzen wir zusätzlich die *Command Search* von IEEE Xplore, die auch verschachtelte boolesche Suchanfragen unterstützt [Lee].

Wir schauen uns jeweils die ersten 20 Ergebnisse an, da dies ein guter Kompromiss zwischen Relevanz und Anzahl der Ergebnisse ist.

Wir suchen nicht nur im Titel, sondern auch im Text, da es nicht immer alles explizit im Titel steht. Als Ziel des Papers, das ist z.B. wichtig für die Wörter quality oder metrics. Auch wenn viele das nicht explizit im Titel oder im abstract haben, wird trotzdem gibt es Paper diese Daten als Grundlage für die Visualisierung verwendet, ohne dies wirklich explizit zu unterscheiden zwischen Qualität und Software selbst. Z.B.: Visualization of the Static Aspects of Software: A Survey

Außerdem suchen wir mit der normalen Google Suche nach existierenden Tools oder anderen Seiten, Ideen die genutzt werden können, um Software-Qualitätsmetriken zu visualisieren. Nutzen wir verschiedenste Suchanfragen.

### 3. Inklusion und Exklusion Kriterien/Paper Auswahl

Auf die Suchergebnisse wenden wir noch folgende Filter an, um unrelevante oder nicht zugängliche Ergebnisse zu exkludieren:

1. Wie bereits gesagt für Google Scholar: software ist ein Muss, der Rest ist optional in Google Scholar, auch wenn wir eigentlich gerne hätten, dass zumindest eines der Wörter "quality" oder "metrics" vorkommt, um sicher zu gehen, dass es sich um Software-Qualitätsmetriken handelt und nicht um Software-Visualisierung im allgemeinen (die Wichtigkeit der Unterscheidung wurde im Grundlagen Abschnitt 2.2 erläutert) deswegen filtern wir die Suchergebnisse auch manuell, um sicher zu gehen, dass es sich um relevante Ergebnisse handelt: z.B.: Software Architecture Visualization: An Evaluation Framework and Its Application [GHMo8]

2. in englisch oder deutsch - aber alles war sowieso auf englisch, aufgrund der Suchbegriffe, außer ein spanisches paper: Software visualization tools and techniques: A systematic review of the literature[Cru+16]

3. Frei zugänglich über alle herkömmlichen anbieter: IEEE, sprignerlink, acm digital library, researchgate,... Das sind die meisten außer zb. Understanding software evolution using a combination of software visualization and software metrics [LD02]

4. Außerdem generell inhaltlich auch passen. zb. Open source software for visualization and quality control of continuous hydrologic and water quality sensor data [Hor+15]

5. wird wirklich eine visualisierung vorgestellt? Wenn in dem paper entweder, mindestens eine 2D layout vorgestellt wird oder wenn mindestens eine 2.5D Visualisierung vorgestellt wird. zb.: Existing model metrics and relations to model quality [MD09]

Für die normale Google suche, verwenden wir verschiedenste heuristische Filter um werbung, Dienstleistungsanbieter und andere irrelevante Ergebnisse zu filtern.

#### *4. auswertung der paper und extraktion der layouts*

Filtern der Layouts, könnten diese für 2.5D geeignet sein? Sammeln und gruppieren der geeigneten verschiedener Layouts Entscheiden für layouts

Wie entscheide ich, ob ein layout geeignet ist: - Es sollte eine Art von Metrik oder Wert darstellen können. - Es sollte eine Art von Layout haben, dass die Struktur der Hierarchie darstellt. - Es sollte ins drei dimensionale extrahierbar sein. - Es soll auch ohne spezielle Einfärbung im 2D funktionieren - eventuell nur die ordner einfärben

#### *5. qualitäts analyse: (Analyse der Layouts und entscheidung welche layouts in dieser arbeit behandelt werden)*

#### *5.2 Durchführung der Suche*

Hier stellen wir zunächst die Ergebnisse der Suche vor nachdem bereits die Filter angewendet wurden.

Erste Google Scholar Suche: 1. Voronoi treemaps for the visualization of software metrics [BDL05] - Voronoi Treemap Layout - sie sagen dass problem ist dass geschwister nicht einfach von anderen benachbarten knoten unterschieden werden können, wir versuchen dass mit den margins zu lösen - als erste probieren sie nicht rechtecke sondern polygone

The problem is provoked by the square-like shape of the rectangles, and because the edges are only horizontally and vertically aligned, whereby the edges of the different objects appear to run into each other. [...] A solution for this problem is the layout of Treemaps based on non-rectangular objects.[BDL05, S. 3]

these polygons are clearly distinguishable because of their irregular shapes, and their aspect ratio converges to one [BDLo5, S. 6]

Aber selbst sie fügen kanten hinzu - und größere he höher außerdem nutzen sie Sättigung um noch mehr die Tiefe und die Hierarchie darzustellen - Kritik: sie wollen treemap geschwisterproblem lösen, schaffen dies aber durch ihren ansatz nicht wirklich und fügen dann doch wieder kanten und farbe hinzu um das zu unterstützen

2. Visualization-based analysis of quality for large-scale software systems [LSPo5] - Zeichnen auch Quader. diese Quader haben immer die selbe grundfläche - metriken werden dargestellt durch 1. die höhe des Quaders, 2. die farbe des Quaders, 3. die rotation (zwischen 0 und 90 Grad) - deren problem ist also nur die platzierung der Quader - KRitik: gefällt mir nicht so gut, vorallem die Roation, außerdem viel platz verschwendet. Sie sagen, dass gedreht schlecht ist. aber 90 Grad sieht nicht so schlimm aus wie 45 grad. dumm -Struktur auf klassen und paket ebene -> das macht es natürlich einfacher, da es generell wenier packete als ordner gibt (packete können auch nochmal in ordner unterteilt sein) - Am ende nutzen sie für die platzierung treemap und Sunburst - Da die fläche fix und diskret ist, erweitern sie sowohl den treemap als auch den sunburst algorithmus - Treemap: beim slicen, wenn der platz zu schmal wird, wird der platz einfach erweitert, für alle knoten, in eine richtung -> sehr viel leerer platz, auch wenn sie versuchen das so gering wie möglich zu halten "burst can still result in the insertion of holes in the representations. In simple illustrations such as in Figure 4, this looks like a serious problem. However, our framework is designed to study systems made of hundreds and thousands of elements. In fact, in the average cases (real softwares) that we tested, it did not prove a problem at all in our visualization." Zeigt, wie wichtig unsere datenanalyse ist, da es wichtig ist die visualisierung für reale daten zu machen und nicht nur simple beispiele vielleicht etwas zu simple gedacht??

"there is still room for improvements, mainly to better exploit the entire display space"[LSPo5, S. 8]

3. Visualization of the static aspects of software: A survey [CZ10] In diesem Paper werden verschiedene Visualisierungen vorgestellt. dieses paper auch schon in verwandten arbeiten erwähnt. Es werden 6 verschiedene Visualisierungen vorgestellt, die auch auf unseren usecase passen.

Sehr gute arbeit, da sie einige visualiasierungen vorstellen. 2D: Treemap, Sunburst, Circle Packing, voronoi

Verschiedene Stadtmethaphern Eine stadt und insel methapher, solar system methapher, welche gleich auch noch genauer beschrieben wird hierarchical net

Zeigen auch, dass das was am meisten verwendet ist stadt methapher und generell treemaps

Ansonsten auch viele andere visualisierungen, die nicht so gut passen, weil es speziell um architektur, evolution oder verbindungen geht.

4. Visual realism for the visualization of software metrics [HVVWo5] Nutzen normale Cushion Treemaps und schattierung und struktur um metrik

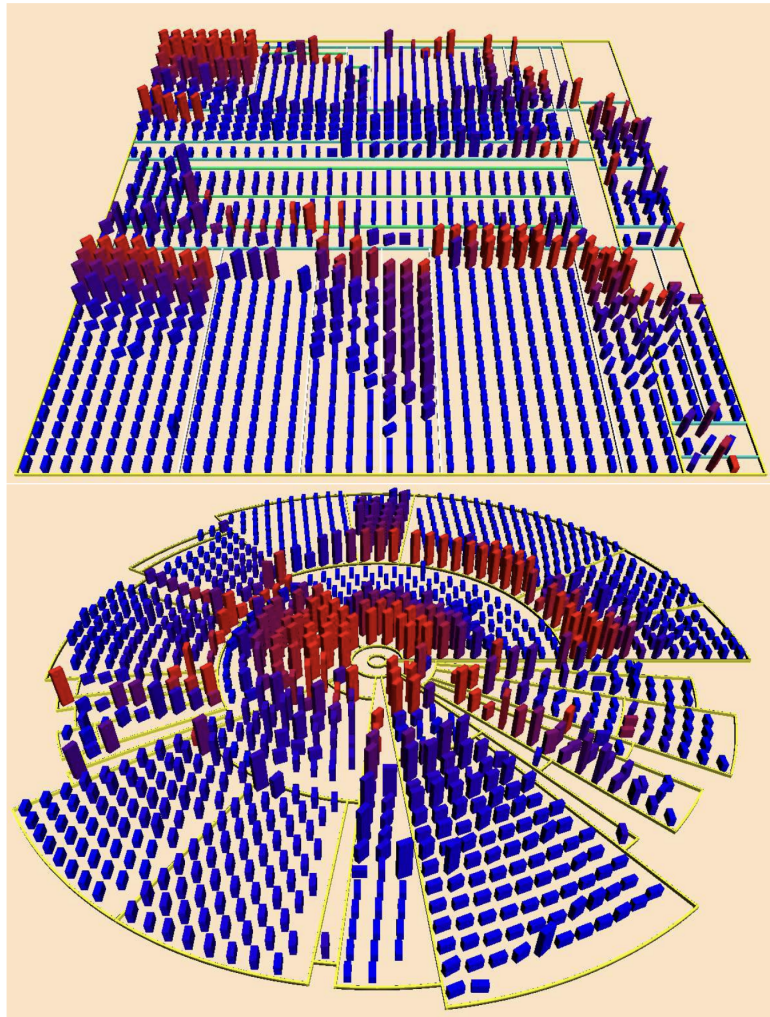


Abbildung 5.1: (Oben) Modifizierte Treemap-Technik und (unten) modifizierte Sunburst-Technik. Beide repräsentieren PCGEN, ein Tool zur Charaktererstellung in RPG (1129 Klassen). [LSP05, S. 5].



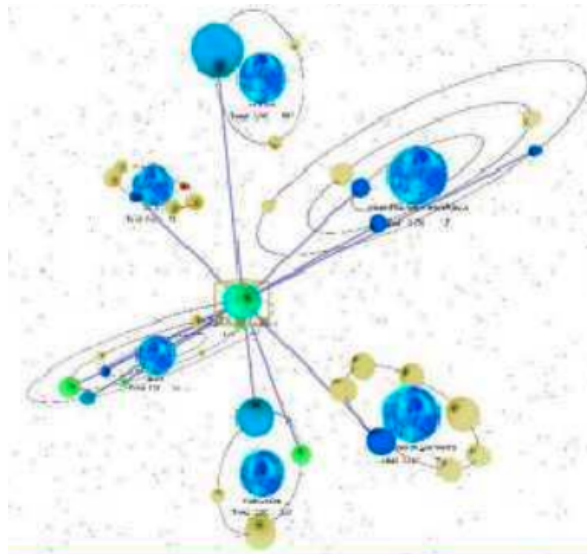


Abbildung 5.2: Solar System Metapher für 3D Visualisierung von objektorientierten Softwaremetriken. Die Farben des Bildes sind invertiert, um die Lesbarkeit im Ausdruck zu verbessern. [GYBo4, S. 4]

werte dar zu stellen nutzen aber auch package struktur, die deutlich einfacher ist als ordner struktur -> nur drei level

5. An overview of 3D software visualization [TCo8] Zeigen alles von 2D, 2.5D und 3D

6. CityVR: Gameful software visualization [Mer+17] - Stellen stadt methapher in 3D vor - fokus auf gamification - layout ist ähnlich wie bei codecity

7. A solar system metaphor for 3D visualisation of object oriented software metrics [GYBo4] Schauen diese metriken an: Lines of Code(LOC) NCSS Number of Children(NOC) DF Weighted Methods (WMC) NCSS Coupling Between Objects(CBO) DF Depth of Inheritance Tree (DIT) DF ganz neue methapher: solar system

Man erkennt, dass es wirklich 3D ist und nicht 2.5D Sie kritisieren, dass die Stadt-Metapher nicht so gut ist, da sie nicht wirklich 3D ist. Und keine Komplexen Zusammenhänge dargestellt werden können. Überdeckung.

Sonnen sind pakete. Planeten sind Klassen, wobei die farbe den typ der klasse angibt. Umlaufbahnen stellen die Vererbungsebenen innerhalb des Pakets dar. Planeten in derselben Umlaufbahn gehören zur gleichen Vererbungshierarchie Die Größe der Planeten ist eine Metrik (die anzahl der zeilen code) Verbindungen zeigen kopplung zwischen Klassen an.

Es werden aber nur relativ kleine Pakete visualisiert, mit nur so ca. 30-40 klassen.

es können nur 2 metriken dargestellt werden, eine auf klassenebene, eine auf verbindungsebene. Man könnte noch die Farbe als metrik nutzen und man könnte nicht auf klassen sondern auf file ebene betrachten (geht ja eigentlich immer) Es wird gefühlt sehr viel leerer platz verschwendet Es gibt doch auch einige überdeckungen, da die umlaufbahnen, teilweise hinter an-

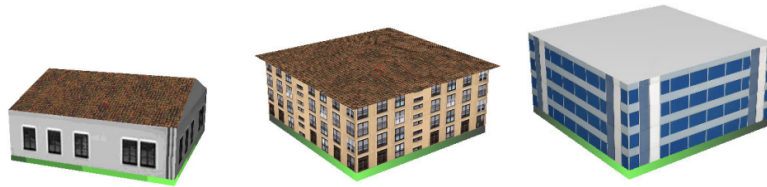


Abbildung 5.3: Ein Beispiel für eine EvoSpaces Visualisierung. Die Gebäude repräsentieren Klassen, je nach Metrik-Wert werden diese in unterschiedliche Kategorien eingeteilt (z.B. bis zu 50 LOC - Wohnhaus, 51-200 LOC - Apartmentblock,...) [ADo7, S. 3].

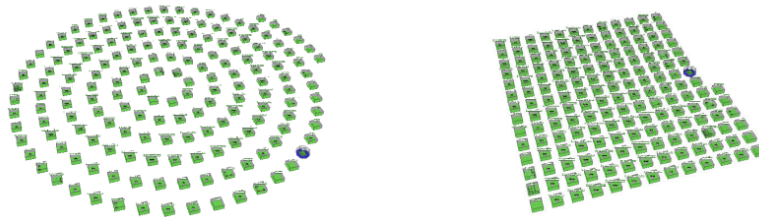


Abbildung 5.4: Die zwei Layouts, die EvoSpaces verwendet. Links: Konzentrisch und Rechts Schachbrettartig. [ADo7, S. 3].

deren planeten hergehen. Man könnte das ganze als 2D Layout verwenden und dann kugelförmig extrudieren, das ich stelle es mir unvorteilhaft vor

8. EvoSpaces: 3D Visualization of Software Architecture. [ADo7] Sie untersuchen verschiedene darstellungen simple darstellungen wie einfache quader oder auch wirkliche häuser metriken werden transparenz, höhe, größe, textur,... dabei können mehrere metriken verwendet werden gebäude sind klassen oder dateien. verbindungen sind röhren bzw. strahle die oben aus einem gebäude herauskommen un dann im bogen in ein anderes gebäude von oben hineingehen

sie stellen aber nur zwei layouts vor, die nicht die struktur zeigen, sondern auch wieder eine Metrik darstellen. zb. im kreis und älteste datei am weitesten innen.

9. 3D representations for software visualization [MFM03] weiterentwicklung von 2d pixel bars ins 3D, weil mehr metriken und besser die verbindungen darstellbar softvis3d läuft auch auf basis von metriken auf datei ebene sehr detailliert, viel kleinteiligere darstellung, da einzelene code zeilen dargestellt werden vorallem für die entwicklung gedacht überdeckung wollen sie mit transparenz lösen layout angeordnet nach zeile für zeile keine hierarchie

10. Software cartography: Thematic software visualization with consistent layout [Kuh+10] Es wird eine topographische Kartenmetapher verwendet Dateien sind Hügel, die Höhe ist eine Metrik (zB. LOC) Das layout wird mit hilfe eines speziellen Multidimensionale Skalierungsalgorithmus (Hit-MDS) erstellt als stress wert der funktion wird die Lexikalische Distanz zwischen den Dateinamen verwendet (ist vielleicht gut wenn man schnell bestimmte dateien finden will, aber nicht so gut für die hierarchie und auch weit weg



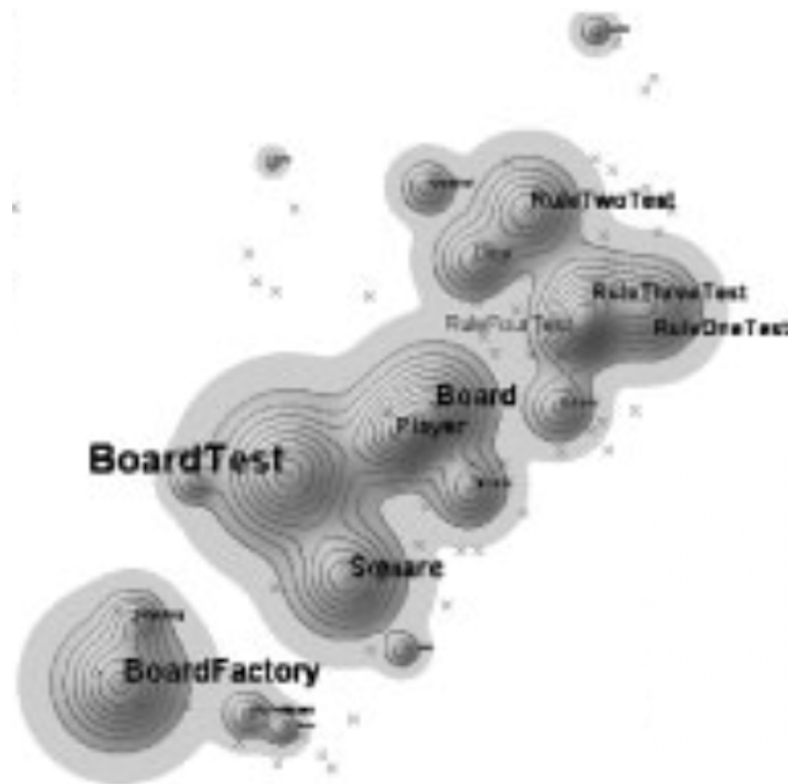


Abbildung 5.5: Software Cartography: Topographische Kartenmetapher für Software Visualisierung. Die Hügel repräsentieren Dateien, die Höhe ist eine Metrik (z.B. LOC). [Kuh+10].

von der software selbst) sie zeigen noch eine erweiterung mit icons auf den bergspitzen, die auch noch eine metrik darstellen können wurde besonders entwickelt mit dem Ziel von software evolution

ich sage, das ist eine interessante grundidee und anders

zweite Google scholar suche: Doppelte Treffer: 4 zB.: Visualization-based analysis of quality for large-scale software systems [LSP05] 1. Exploring Relations within Software Systems Using Treemap Enhanced Hierarchical Graphs [BD05] auf oberer Ebene wird energy model of Noack and Lewerentz genutzt um einen graphen zu malen. dieser graph kann auch mehrere hierarchie ebenen visualisieren, dabei wird farbe als unterstützendes mittel genutzt, um die verschiedenen pakete oder order zu unterscheiden. Jeder knoten hat dann nochmal intern eine Treemap, die die Struktur des knotens zeigt

interaktion mit zoomen ist von nöten. Alles bisher noch in 2D

2. Visualizing Software Metrics in a Software System Hierarchy [Bur15] im grunde eine link treediagram, Metrik stripes - metrische streifen, zeigen über breite und farbe verschiedene metriken an

layout wird wie in Grundlagen gesagt, sehr schnell unübersichtlich.

3. A Stable Greedy Insertion Treemap Algorithm for Software Evolution Visualization [VCT18] stellen nur neuen treemap algorithmus vor, der fokus auf stabilität hat diese wurden erstmals hier vorgestellt [Gor+18]

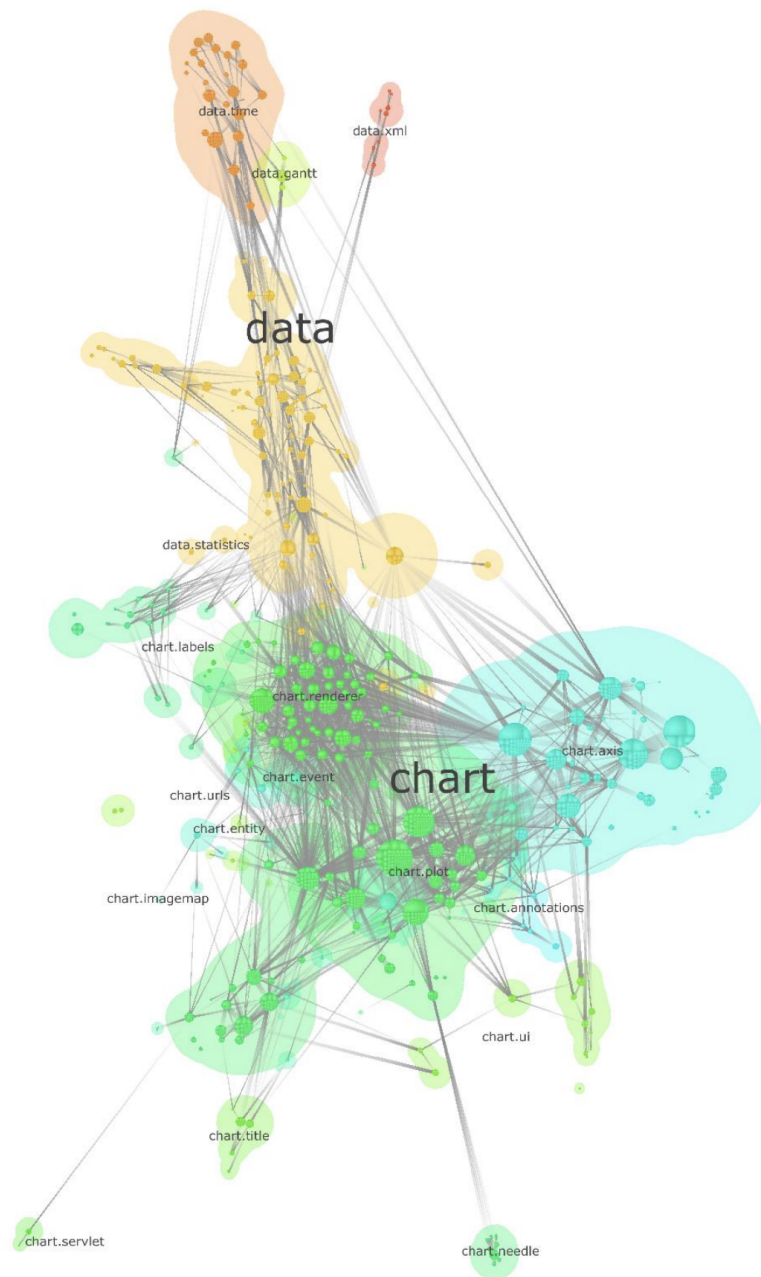


Abbildung 5.6: Exploring Relations within Software Systems Using Treemap Enhanced Hierarchical Graphs. Die obere Ebene ist ein Graph, der die Beziehungen zwischen den Paketen zeigt. Die untere Ebene (in den einzelnen kreisen nur klein zu sehen) ist eine Treemap, die die Struktur des Pakets zeigt. [BD05, S. 5].

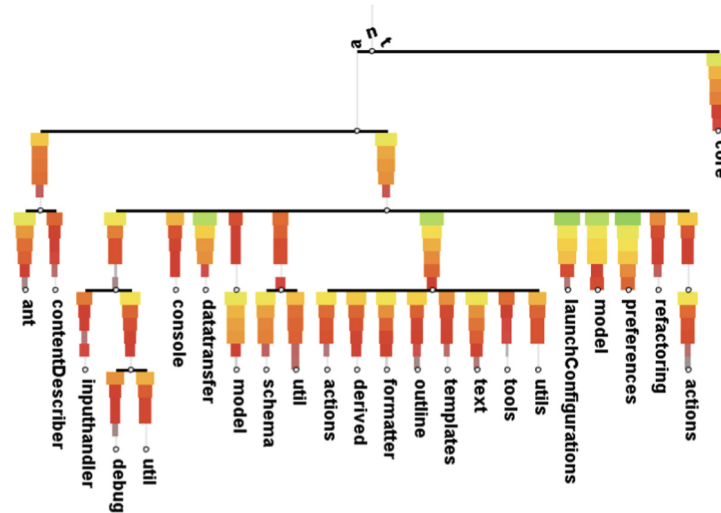


Abbildung 5.7: Im Grunde ein Link-Treediagramm, das mit *Metric Stripes* verschiedene metriken darstellt. [Bur15, S. 740].

4. Visualizing Program Quality – A Topological Taxonomy of Features [AO+19] eigentlich gar nicht für software qualität, aber schlagen die bubble treemap vor

5. Stable and predictable Voronoi treemaps for software quality monitoring [HH17] erweitern voronoi 3 metriken: 1. farbe 2. gröÙe 3. reihenfolge der anordnung/bzw positionierung idee ist als 3. metrik den fully qualified name zu nutzen um so die topologie der hierarchie darzustellen -> Ziel vergleich zwischen verschiedenen versionen

6. Visualization and evolution of software architectures [Kha+12]

IEEE Suche: Doppelt: 3 zB. Visualization of the Static Aspects of Software: A Survey [CZ10] 1. Metrics-based 3D visualization of large object-oriented programs [LS02] auch gröÙe und farbe als metrik

Einzelne Kugeln, Würfel,... im 3d raum, die mit linien verbunden sind, je nach verbindung das layout ist nach einem gleichheitswert der klassen angeordnet, dieser wert kann alles sein, entweder metrik oder platz etc.

2. Visualizing Metric Trends for Software Portfolio Quality Management [Gen+21] Stellen einfach eine Plattform vor, die verschiedene Metriken über zeit visualisiert mit linien plots und balken diagrammen für komplette software - nicht wirklich relevant für uns

3. E-Quality: A graph based object oriented software quality visualization tool [ETB11]

First, we examine a small-sized project to be able to easily illustrate the features of E-Quality. [ETB11, S. 5]

nur vergleich mit folder based navigations methode

packete werden kreisförmig angeordnet Metriken haben auswirkung auf, muster, gröÙe, farbe, form

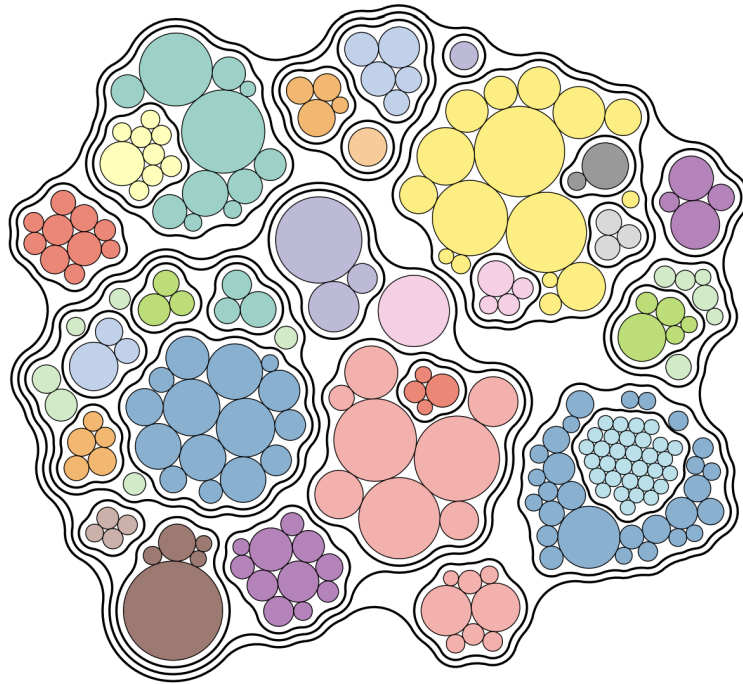


Abbildung 5.8: Beispiel für eine Bubble Treemap [Gor+18, S. 7]

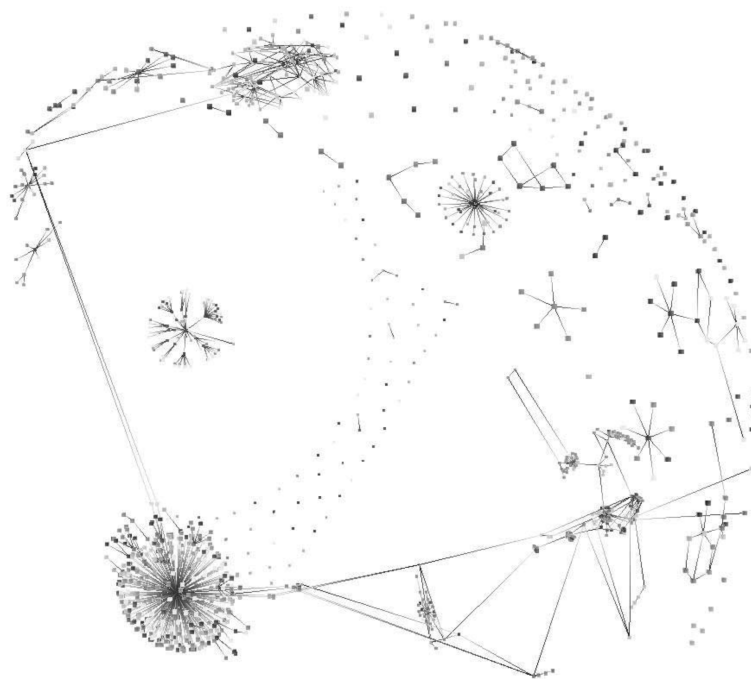


Abbildung 5.9: Die vorgestellte visualisierung zeigt, wie Klassen in 3D gemappt werden anhand von similarity score [LS02, S. 5].

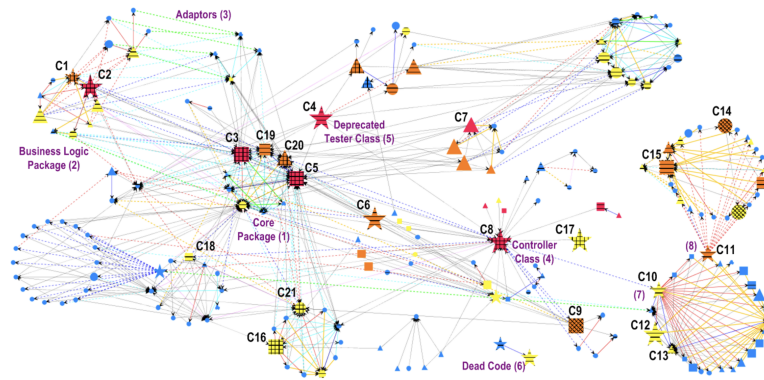


Abbildung 5.10: E-Quality: Ein graph basiertes objektorientiertes Software-Qualitätsvisualisierungstool. [ETB11, S. 5].

4. Visualization of Software Quality Expert Assessment [HB19] auf projekt ebene

5. QScore: An Open Platform for Code Quality Ranking and Visualization [TKS20] stellen eigentlich nur plattform vor visualisieren mit treemaps und sunburst und dependency graph

6. Using the City Metaphor for Visualizing Test-Related Metrics [Bal+16] Stadt methapher in minecraft Gebäude sind sehr detailliert und bilden methoden, attribute, metriken, interfaces, ... alles mögliche ab eher eine spielerei, aber im grunde layout wie code city

7. Visual Analytics of Software Structure and Metrics [Kha+15] Nutzen den city view: von ecity [Kha+13] eCITY: A Tool to Track Software Structural Changes Using an Evolving City

Google Tool suche: Gefilter werden paper, wenn sie kommen. Nur webseiten werden geöffnet Software Quality Visualizer

Software Metric Visualizer/software quality analysis/ software visualisation tool/ dabei auch die filter: 1. software quality vis 2. existing tool 3. keine paper, viele dienstleister raus filtern wie zb. <https://bitsea.de/dienstleistungen/software-qualitaetsanalyse/> -> nur dienstleistungen anbieten dabei gestoßen auf: <https://home.uni-leipzig.de/svis/publications.html> und <https://hpi.de/doellner/publications.html>

grafana.com -> aussortiert, weil generelle library für visualisierungen und dashboards

<https://github.com/MaibornWolff/codecharta>

[www.jarchitect.com](http://www.jarchitect.com) und sonarqube <https://www.cs.rug.nl/svcg/SoftVis/ArchiVis> GETAVIZ gefunden sereen gefunden

code-is-beautiful: letzter commit vor 7 jahren <https://github.com/quantifiedcode/code-is-beautiful> bietet: code city, Sunburst (2d): im grunde ein kreis diagram, welches von innen nach außen die ordner strukturen zeigt. die es können die größe (wie viel des kreises nimmt etwas ein) und die farbe als metric gewählt werden Stack (2d): im grund wie das kreis diagram nur von oben nach unten

Code Radar: <https://github.com/pschild/CodeRadarVisualization> Wie code city speziell auf vergleich von versionen ausgelegt, indem man zwei karten direkt nebeneinander sehen kann letzter commit vor 7 jahren

softvis: vor 2 jahren wie code city

Ndepend: <https://www.ndepend.com/docs/treemap-visualization-of-code-metrics?cx=0150956779873219162952D> Aber interessant: hier werden die kanten dunkel eingefärbt, um die einzelnen elemente visuell zu unterscheiden

<https://community.sap.com/t5/welcome-corner-blog-posts/i-have-a-dream-code-visualization/bc-p/13485189/highlight/true>: hat die idee dass gebäude wirklich echt sein könnten ein gebäude an dem gebaut wird, wurde kürzlich geändert alte gebäude wurden lange nicht mehr verändert oft geänderte objekte sind nah an einem hafen oder so

ein kommentar auf deren seite: "Playing around with these metrics seems like an easy way to identify those objects where a refactoring promises to be most beneficial, because large and complex objects that are changed often tend to introduce bugs and slow down the development process.

It also seems that this visual approach helps to promote topics like clean code within an organisation as the negative impact of such skyscraper classes becomes clearer when you look at these graphics.

I think I will take a Code City Snapshot from time to time to track how the red skyscrapers steadily turn into beautiful, clean and green suburbs.

Even without medieval buildings that can be accessed using VR, this might help a lot to navigate towards a clean code base."

<https://home.uni-leipzig.de/svis/getaviz/index.php?setup=web/Cityauch> sehr sehr nice mit bausteinen verschiedene sachen visualisieren

<https://codescene.com/product>: 2d kreis diagramme

<https://github.com/adamtornhill/code-maat> bietet auch einiges in 2d

Wie ist der Stand der Forschung? overview of 3d software visualisierung: <https://ieeexplore.ieee.org/document/4564449>

interessant: <https://opus-htw-aalen.bsz-bw.de/frontdoor/deliver/index/docId/658/file/ICCSSEE.pdf>

[Mer+18]: A systematic literature review of software visualization evaluation: Wie kann man visualisierungen bewerten? Hier geht es vor allem darum: "help analysts make sense of multivariate data (Merino et al., 2015), to support programmers in comprehending the architecture of systems (Panas et al., 2016), to help researchers analyze version control repositories (Greene et al., 2017), and to aid developers of software product lines"

Quality models are usually defined based on concrete measurements of software metrics (N. Fenton and J. Bieman, Software metrics: a rigorous and practical approach. CRC Press, 2014)

4 metriken werden visualisiert: farbe, position, höhe, breite

sehr interessant. 1. Idee unterschiedliche metriken zu stacken 2. gibt einen generelen qualitäts wert am ende heraus, erstellt aber eine grafik, die die einflüsse verschiedener klassen auf die verschiedenen metriken zeigt und die generelle bedeutung für die allgemein metrik am ende.



Moose - ich würde das eher bewreten für entwickler. Es werden hier verbindungen von klassen dargestellt - abhängigkeiten undco.

VON cascaded: hier könnte man auch nochmal suchen, muss aber auch nicht. So etwas ähnliches auch sagen: A number of other hierarchy visualization techniques have been developed [18, 23, 29, 32], including space-filling visualizations like step trees [6], Voronoi treemaps [2] and generalized tree-maps [34]. Although relevant to hierarchy visualization, we pursue contributions that are sufficiently distinct from such work that we do not dwell on extensive comparisons. [LFo8]

### 5.3 *Dokumentation der Ergebnisse und Vorstellung der Layout-Umsetzung*

In diesem Abschnitt stellen wir die Ergebnisse der Recherche vor und dokumentieren die Layouts, die wir in dieser Arbeit umsetzen wollen.

#### 5.3.1 *Sunburst Layout*

#### 5.3.2 *Circle Packing Layout*

#### 5.3.3 *Noch eins mehr?*

Dieser abschnitt hat 2 ziele 1. eine gute auswahl an code basen finden, die wir sowohl für unsere algorithmus anpassungen als auch für die evaluation am ende verwenden können. 2. wollen wir die Struktur von Softwareprojekten näher untersuchen, um zu verstehen, wie sie aufgebaut sind und welche Eigenschaften sie haben. wie gesagt wir wollen die Struktur näher bestimmen. Wie definiert ist die struktur immer eine baumstruktur. genauso wie in der problemstellung (abschnitt 3) definiert.

### 6.1 *Auswahl der Projekte*

Als Quelle werden wir GitHub verwenden, da es eine große Anzahl an Open-Source-Projekten bietet und die Daten leicht zugänglich sind. Wir gehen nicht davon aus, dass es einen signifikanten Unterschied zwischen Open-Source und Closed-Source Projekten gibt [Rag+05; PSE04], weshalb wir nur auf open source zugreifen. Wir probieren möglichst breit gefächert Projekte zu betrachten, um ein möglichst breites Spektrum an Softwareprojekten abzudecken. Das Ziel bei der Auswahl der Projekte ist es, eine möglichst diverse Auswahl an Projekten zu haben, um später eine Aussage über die Struktur von Softwareprojekten im Allgemeinen treffen zu können und nicht nur über die Struktur der meisten Softwareprojekte. Wir könnten einfach zufällige Projekte von Github auswählen, aber das würde vermutlich zu einer Verzerrung führen und eher kleinere und unbekannte Projekte auswählen. Deshalb suchen wir nach Kriterien, die einen Einfluss auf die Struktur von Softwareprojekten haben könnten.

da es keine quellen gibt, die eine solche Auswahl an Kriterien vorgeben, können wir nur vermutungen anstellen und diese im nachhinein anhand der Daten überprüfen, können aber nicht sicher garantieren, dass wir wirklich alle relevanten Kriterien berücksichtigt haben. aber das ist ja auch keine arbeit über die selektion von Projekten, deshalb geben wir uns damit zufrieden. Zur identifikation von relevanten Kriterien richten wir uns nach den informationen, die die GitHub API uns zur verfügung stellt. Es gibt eine Vielzahl an Metainformationen.

Folgend checken wir erstmal die Kriterien, ob diese überhaupt relevant sind und ob sie einen Einfluss auf die Struktur von Softwareprojekten haben.

#### *Programmiersprache*

Wir werden Projekte in verschiedenen Programmiersprachen betrachten, da die Struktur von Softwareprojekten in verschiedenen Programmiersprachen stark unterschiedlich sein kann - hier fehlt noch ein beweis - ist da überhaupt



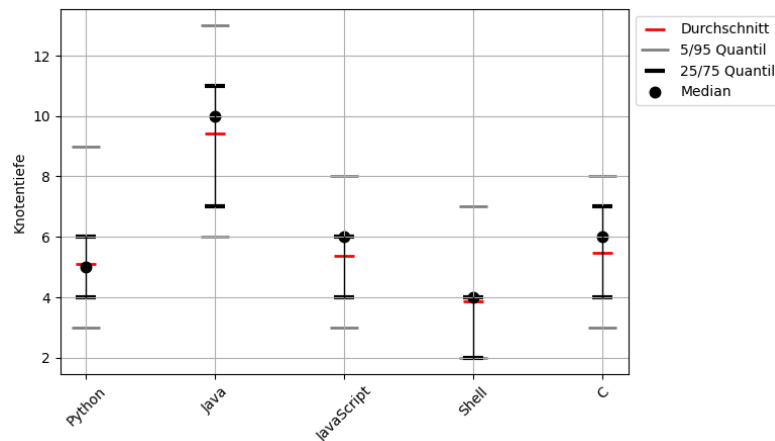


Abbildung 6.1: Die Tiefe variiert stark nach Sprache. Zum Beispiel sind die meisten Knoten (95%) in java deutlich tiefer (tiefe 7) gelgen als die meisten Knoten (75%) in shell (tiefe 4)

so? wenn nicht kann man das auch weglassen. Wir wählen die Programmiersprachen nach ihrer Popularität und Paradigma aus.

- **Python:** Die zweit populärste Sprache [Sofb] mit dynamischer Typisierung, sowohl imperativem als auch objektorientiertem Paradigma.
- **JavaScript:** die populärste Sprache [Sofb] mit dynamischer Typisierung. Die Struktur von JavaScript-Projekten ist besonders geprägt durch Frameworks.
- **Java:** Die dritt populärste Sprache [Sofb] mit statischer Typisierung und objektorientiertem Paradigma. Java-Projekte sind oft sehr strukturiert und folgen bestimmten Konventionen.
- **C:** Eine der ältesten Programmiersprachen, die immer noch weit verbreitet ist (Platz 9 [Sofb]). C-Projekte sind oft sehr nah an der Hardware und haben eine andere Struktur als Projekte in höheren Programmiersprachen.
- **Shell:** Eine Skriptsprache (Platz 8 [Sofb]), die oft für Automatisierung und Systemadministration verwendet wird.

Man sieht aber in beiden Grafiekn, dass die Sprache einen Einfluss auf die Struktur hat, was alles war was wir hiermit zeigen wollten.

### Projektgröße

Die Projektgröße ist offensichtlich der Wichtigste Faktor für die STruktur eines Projekts. Wir gehen davon aus, dass Große Projekte auch dann größere Strukturen (also mehr knoten) haben (stimmt nicht zwingen - wie im folgenden Abschnit 6.1.1 gezeigt wird) Github bietet leider nur die möglichkeit nach größe und nicht nach Anzahl der Dateien zu filtern, was uns eigentlich interessieren würde.

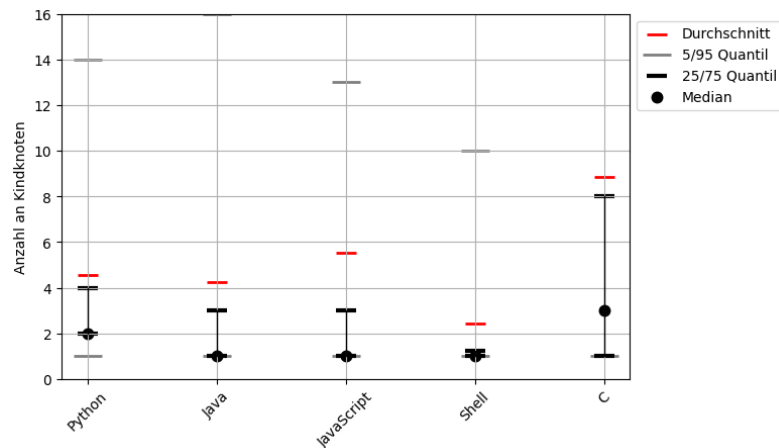


Abbildung 6.2: Die Varieiert zwar im Durchschnitt teilweise zb zwischen sheell und C aber im großen und ganzen sind die Intervalle der Quantile schon überlappend

Wir beweisen kurz, dass die Größe einen Einfluss auf die Struktur hat, indem die Hierarchie-Tiefe in Abhängigkeit von der Größe untersuchen.

Man sieht aber in beiden GRafiekn, dass die größe einen Einfluss auf die Struktur hat, was alles war was wir hiermit zeigen wollten.

#### *Ungenutzte Kategorien*

Man könnte denken, dass auch andere Kategorien wie die Anzahl der Sterne oder die Anzahl der Mitwirkenden eines Projekts einen Einfluss auf die Struktur eines Projekts haben. Wir hatten auch diese vermutung und haben dies nachgeprüft. Um fragen zu vermeiden zeigen wir hier auch die ergebnisse, die gezeigt haben dass diese Kategorien keinen Einfluss haben und wir sie deshalb verworfen haben.

1. Wir gingen davon aus, dass die Popularität eines Projekts einen Einfluss auf die Qualität eines Projekts hat. Je Popularitärer ein Projekt ist, desto "besser" muss es sein, desto besser muss ja auch die Qualität sein. Und wir gehen davon aus, dass die Qualität eines Projekts einen Einfluss auf die Struktur des Projekts hat. Allerdings deutet eine Studie an, dass Popularität nicht in Korrelation mit Software Qualität steht [Saj+14]. Außerdem sind die meisten Projekte die auf GitHub viele Sterne haben eher Sammlungen von Bücher, Kursen oder anderen Ressourcen und nicht wirklich Softwareprojekte [evanli\_github-ranking\_2025], weshalb wir uns entschieden haben, die Popularität nicht als Kriterium zu verwenden, auch wenn es auf den ersten Blick sinnvoll erscheinen mag.

2. Eine größere Anzahl an Mitwirkenden, mehr Änderungen und generell mehr aufmerksamkeit. Es gibt paper, die sagen, dass unterschiedlichen Anzahlen an Mitwirkenden hat einen Einfluss auf die Struktur und Qualität des Codes haben kann [CAC20]. Wir werden nur Projekte bereits ab einer Anzahl von 2 Mitwirkenden betrachten, da Projekte mit nur einem Mitwirkenden oft nicht wirklich repräsentativ für die Struktur von Softwa-

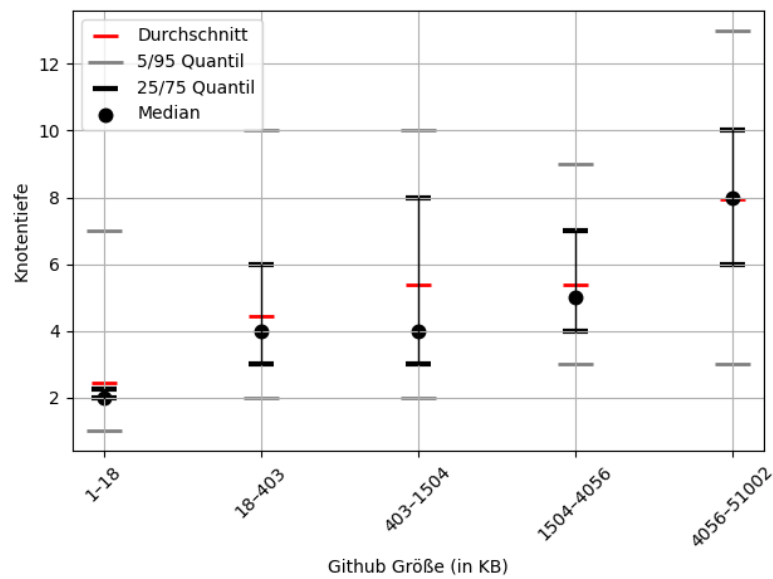


Abbildung 6.3: Die Hierarchie-Tiefe korreliert positiv mit der Größe des Repositories, was zeigt, dass größere Repositories tendenziell eine tiefere Hierarchie haben. Pearson Korrelation von 0.37, zeigt dass die korrelation schon schwach ist, bzw es viele ausreißer oder andere faktoren gibt (wie zum beispiel die sprache)

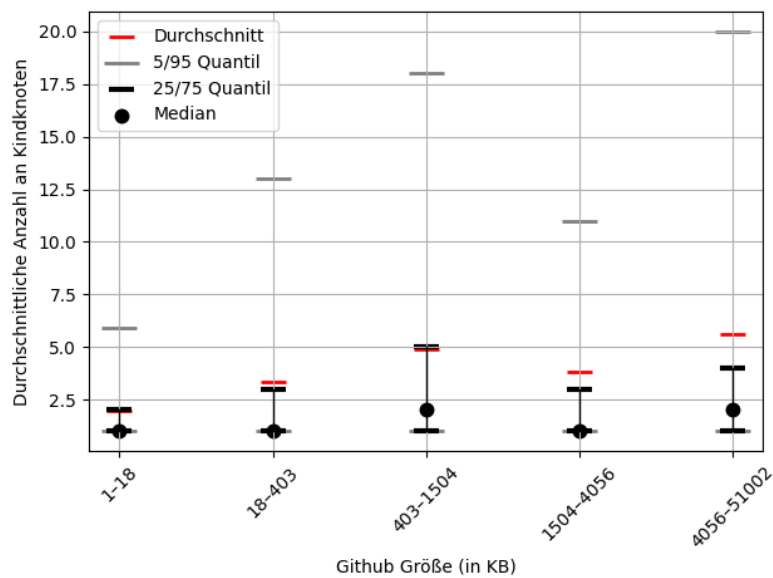


Abbildung 6.4: Die Durchschnittliche Anzahl an Kinder-Knoten allerdings nur leicht positiv mit der Größe des Repositories korreliert, was zeigt, dass größere Repositories tendenziell eine größere Anzahl an Kinder-Knoten haben, aber nicht wirklich ausschlaggebend. (Pearson Korrelation: 0.09)

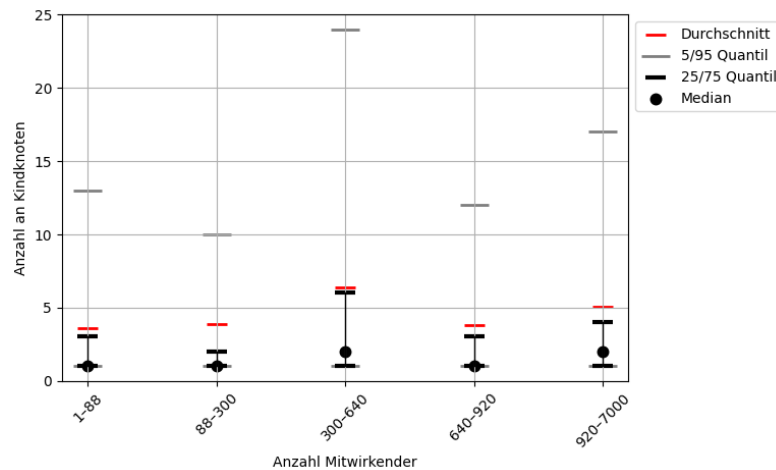


Abbildung 6.5: Die Anzahl der Mitwirkenden auf der x-Achse hat keinen erkennbaren Einfluss auf die Anzahl der Kinder-Knoten eines Knotens.

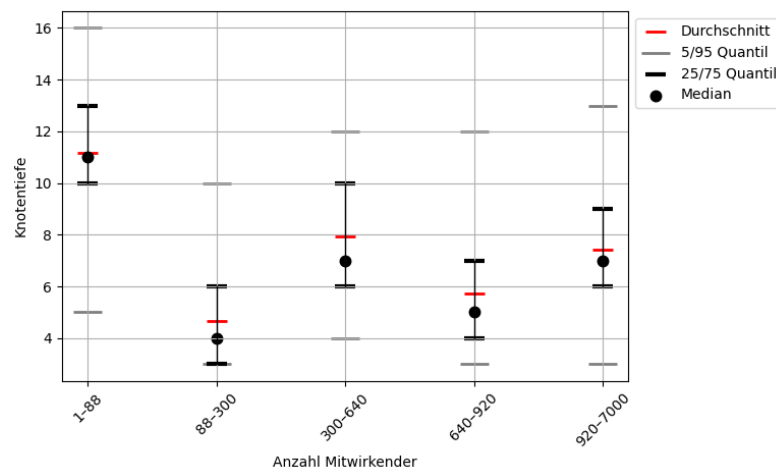


Abbildung 6.6: Die Anzahl der Mitwirkenden auf der x-Achse hat keinen erkennbaren Einfluss auf die Hierarchie-Tiefe eines Knotens.

reprojekten sind. Wir überprüfen, ob die Anzahl der Mitwirkenden einen Einfluss auf die Struktur - konkret die Hierarchie-Tiefe und die Anzahl der Kinder-Knoten - hat.

#### 6.1.1 Die Auswahl der Projekte im Detail

So wie wir wissen jetzt, dass wir nur auf Sprache und Größe achten wollen. Wir haben 5 verschiedene bekannte Sprachen und verschiedene Größen. Wir nehmen eine gute Anzahl an Projekten, so dass jede Kombination repräsentiert ist, aber trotzdem nur so viele wie wir rechnerisch handlen können. In Abbildung 6.7 haben wir die Verteilung der Projektgrößen von 2500 zufällig ausgewählten GitHub-Repositories dargestellt. Es ist zu erkennen, dass es deutlich mehr kleinere Projekte gibt. (Zur Größen-Einordnung: das REact repository, was als ein sehr großes Projekt gilt, hat eine Größe von ca. einem Gigabyte) Wir wollen die Größe von 98 Prozent aller Repositories abdecken. Wir haben uns

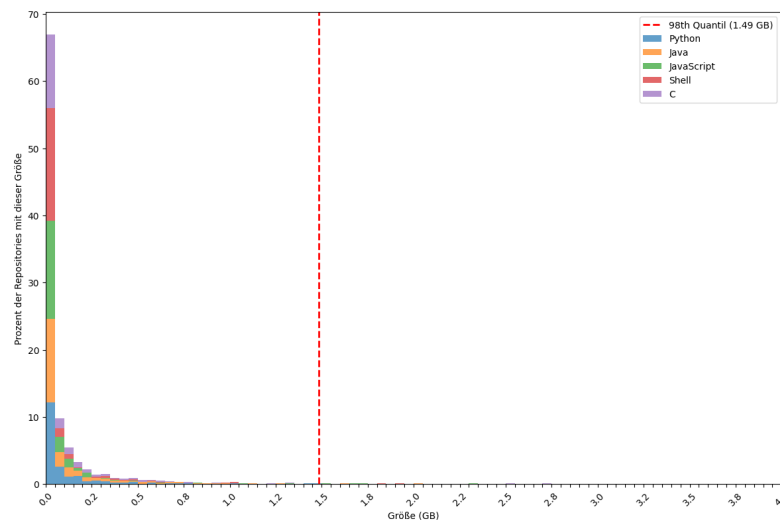


Abbildung 6.7: Das Diagramm zeigt, dass es deutlich mehr kleine Projekte gibt (logarithmisch abnehmend)

für diesen Wert entschieden, da wir eine sehr gute Abdeckung auch großer Projekte haben wollen, aber trotzdem auch nicht zu viele Projekte generell haben wollen. Außerdem ab dieser Größe wird es wahrscheinlich generell sehr schwer sinnvoll zu visualisieren bereits das React Repo kann mit 1GB teilweise unübersichtlich werden, nicht weil der algorithmus oder die visualisierung schlecht ist sondern weil es einfach zu viele Dateien werden irgendwann.

Man könnte argumentieren, warum macht ihr euch die arbeit und nehmt nicht einfach random Projekte? Weil wir spezifisch sicher gehen wollen projekte aller größen in einer linearen verteilung. ansonsten haben wir vorallem kleine Projekte, bei denen sowieso das treemap problem nicht so sehr auftritt, dann würden wir in den daten auch keine wirklichen änderungen sehen können (wahrscheinlich) oder nicht so große. weil eher kleine projekte ??.

Deswegen wählen wir spezifisch Projekte bis zu 1,5GB (98er Quantil) aus. Wir wählen das so, dass wir eine lineare verteilung haben und ca. alle 0,1GB ein Projekt - also haben wir 15 Projekte pro Sprache -> 70 Projekte, die die grundlage für alle analysen und evals dieser ARbeit bieten. Zusätzlich gehen wir sicher, dass Projekte wie *The most comprehensive database of Chinese poetry* oder *A list of funny and tricky JavaScript examples* nicht im datensatz enthalten sind, weil das keine echten Software-Applikationen sind, sondern sammlungen oder andere sachen, für die unsere Visualisierung nicht gedacht ist -> klar auch möglich natürlich, aber nich das Ziel dieser Arbeit.

In Abbildung 6.8 ist zu erkennen, dass wir relativ gleichmäßg die repos bis zu einer Größe von 1,5GB abdecken. Die genau auswahl an repos ist in Anhang A.3 zu finden.

Da die größe nicht 1 zu 1 mit der Anzahl an nodes korreliert. ein order zum beispiel nimmt fast keine größe ein. außerdem sind bei jeder sparche die dateien verschieden groß es gibt unterschiedliche anzahl an dateien die

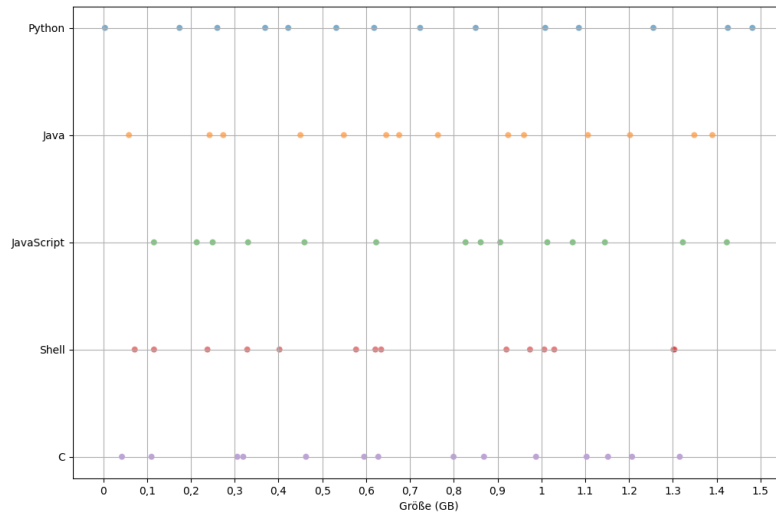


Abbildung 6.8: Die Verteilung der Projektgrößen der ausgewählten Repositories je Sprache.

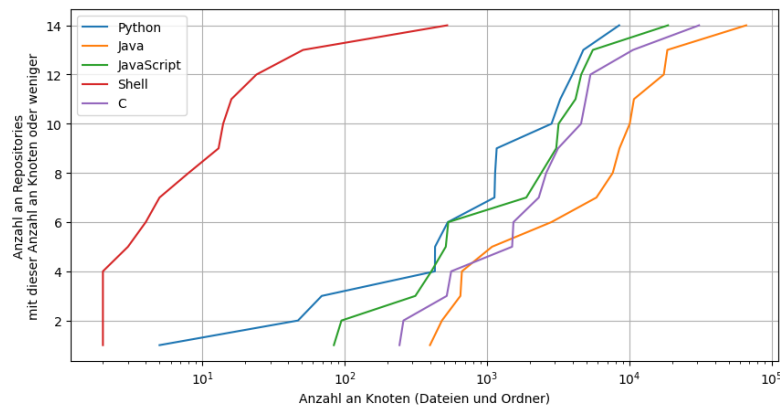


Abbildung 6.9: Die Verteilung der Anzahl an Knoten der ausgewählten Repositories je Sprache.

nicht betrachtet werden wie png, mp4 etc die zur größe aber nichth zur anzahl an nodes beitragen. Es ist zu erkennen, dass dadurch doch wieder mehr kleinere Repos gibt, aber das akzeptieren wir einfach. 6.9 Besonders bei Shell ist zu erkennen, dass die Anzahl an Knoten nicht wirklich linear verteilt ist. Zeigt wie unterschiedlich die repos der verschiedenen Sprachen sind, trotzdem akzeptieren wir das, am ende bildet es die Realität ab.

## 6.2 Struktur von Softwareprojekten

Wie berechnen wir die Struktur der Repositories und wie bekommen wir metrik daten für diese REpos? Wir clonen jedes repo und analysieren die Struktur der Repositories mit dem CodeCharta Analysis CLI Tools [Gmbd]. Dieses Tool analysiert die Struktur der Repositories und berechnet für jede Datei (es werden nur Dateien betrachtet die zur entsprechenden Programmiersprache gehören, also zB. .py für Python) die Metriken, die wir später

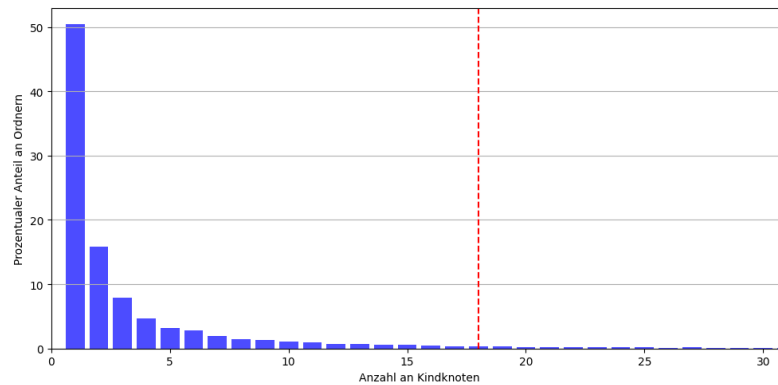


Abbildung 6.10: Die Anzahl der Kind-Knoten pro Ordner. Mehr als 50% der Ordner haben nur ein Kind. Das 95 Prozent Quantil liegt bei 18.

für die Visualisierung verwenden werden. Genutzt wird der unified parser mit diesem Command: Als ergebnis bekommen wir eine JSON-Datei, mit der in Abschnitt 3 definierten Struktur. Ein Beispiel ist auch im anhang (Listing A.1) zu finden.

Im Abschnitt Problemstellung (Abschnitt 3) wurde das Format der Eingabe-Daten formal definiert (siehe Listing 3.1). Die Hierarchische-Struktur selbst ist offen in ihrer Größe und Form also zB. wie viele Kinder hat ein Knoten, wie Tief ist die Verschachtelung etc. Wir versuchen das in diesem Kapitel einzuschreänken, also auf zu zeigen, wie hierarchische strukturen, die aus software analyse auf file-basis laufen aussehen. UM nochmal klarer zu werden: die formale definition steht. aber in der praxis realität werden diese Strukturen aus der Datei-Hierarchie von Software-Projekten entstehen.

Warum wollen wir das tun? Weil wir uns erhoffen, erkenntnisse über die Datei-Struktur von Software-Projekten zu gewinnen, die uns helfen, die algorithmen genau für diese Strukturen zu optimieren und außerdem die Laufzeiten der algorithmen eine bessere einschätzung zu geben.

Im Grundlagen Kapitel wurde bereits erwähnt, dass die Metrik-Daten, die wir durch die Analyse von Code erhalten, spezielle Eigenschaften aufweist (siehe Abschnitt 2.1). Diese Eigenschaften sind wichtig, um die Visualisierung zu optimieren und zu wissen in welchem Rahmen sich die zu visualisierenden Daten bewegen.

numberOfFiles: 3262.34 (avr), 708.50 (median), 7391.42 (std) numberOfFolders: 958.76 (avr), 242.00 (median), 2089.59 (std) numberOfNodes: 4221.10 (avr), 1108.50 (median), 9308.05 (std)

sehr hohe varianz zwischen den einzelnen repos.

in abbildung 6.10 ist zu erkennen, dass die meisten Ordner nur ein Kind haben (was auffällig ist, sie Ordnerketten im folgenden). Das quantil 95 liegt bei 18, was interessant wird im Abschnitt

### 6.2.1 Orderketten

Wir vermuten, dass die hohe Tiefe bei java daher kommt, dass es in Java üblich ist, die Struktur des Codes in Paketen zu organisieren, die häufig

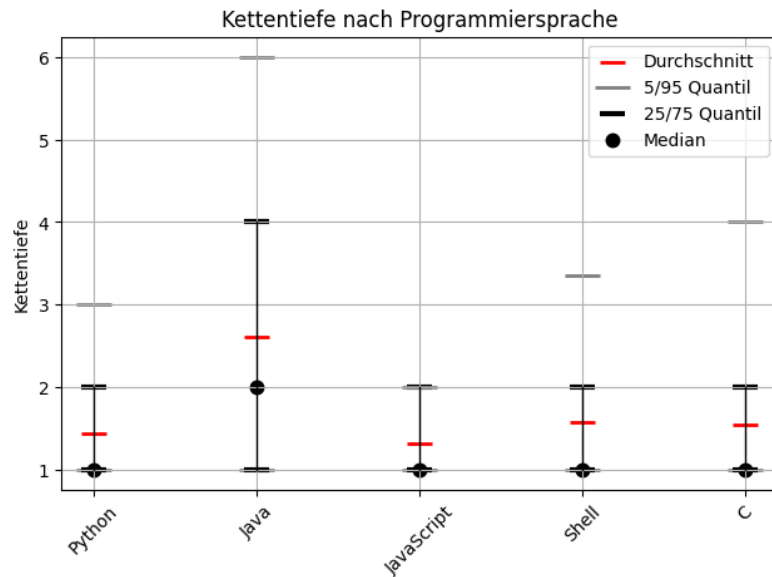


Abbildung 6.11: Die Kettentiefe ist besonders bei Java auffällig variant und auch der Mittelwert ist höher als der bei allen anderen Sprachen.

in mehreren Ebenen verschachtelt sind. Zb: Zum Beispiel im OpenSource Projekt CodeCharta [Gmbb] `codecharta/analysis/analysers/AnalyserInterface/src/main/kotlin/de/maibornwolff/codecharta/analysers/analyserinterface/AnalyserInterface`. Ab dem Ordern *main* bis zum Ordner *analyserinterface* geht die Kette von Ordnern. Da wir uns aber für die generelle vertelung interessieren hier die grafik: Die beiden grafiken zeigen uns, dass es viele Ketten gibt (speziell in Java) und dass viele Ordner in diesen Ketten sind. Außerdem können ketten nicht nur ein order tief sein sondern häufig auch 4 ordner tief. Diese erkenntniss wird in Abschnitt 7.6.1 eine Rolle spielen.

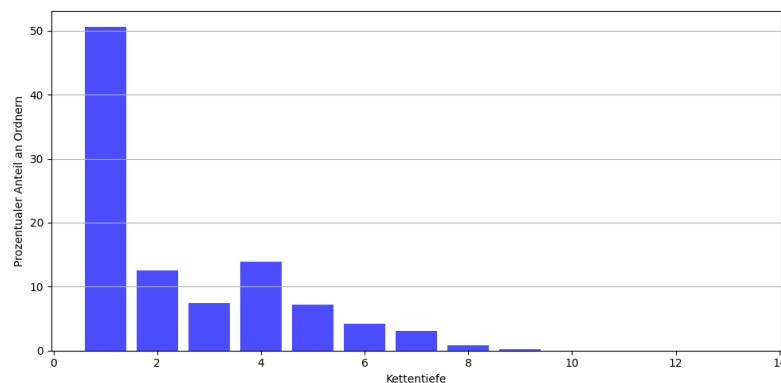


Abbildung 6.12: Die meisten Ketten sind nur 1 Ordner tief (Ordner + Ordner + Datei/Mehrere Dateien/Mehrere Ordner). Auffällig ist, dass es wieder mehr ketten von länge 4 gibt.



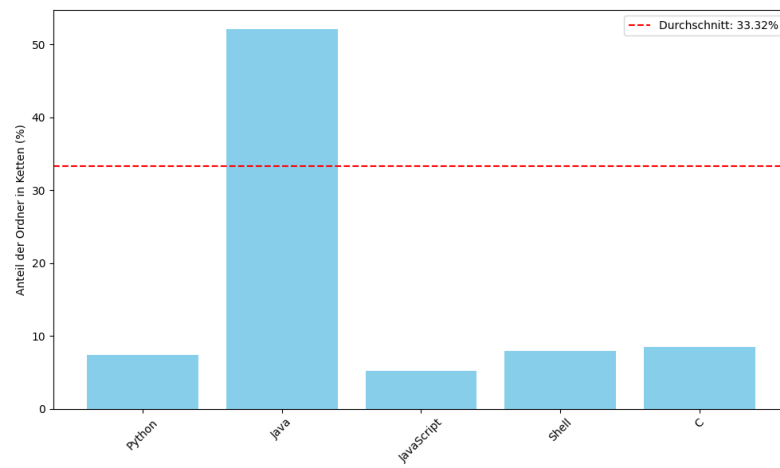


Abbildung 6.13: Die Anzahl der Ordner, die Teil einer Kette sind, ist besonders hoch bei Java. Der Durchschnittswert über alle Sprachen liegt bei ca. 33% und der von Java bei ca. 50%.

## ERWEITERUNG DES SQUARIFY ALGORITHMUS

---

In diesem Abschnitt wird der Squarify Algorithmus [BHVWoo], wie er in Abschnitt 2.4.1 beschrieben wurde, auf verschiedene Weisen erweitert und angepasst, um das Layoutproblem, wie es in Abschnitt 3.1 beschrieben wurde, anzugehen.

### 7.1 Kriterien für Treemap Layouts

Um diese fünf Aspekte zu erreichen, definieren wir sechs Kriterien für das 2D layout, die diese Aspekte messbar machen. Diese Kriterien sollen helfen, die Qualität der Visualisierung zu bewerten und zu vergleichen. Die Kriterien sind:

- **Abstände:** Abstände zwischen den Knoten verbessern die Übersichtlichkeit und die visuelle Komplexität.
- **Knoten sichtbarkeit:** Es sollte keine Knoten geben, die aufgrund von Abständen oder anderen Gründen nicht sichtbar sind. Dieses Ziel spielt speziell auf das in abschnitt 2.4 beschriebene Problem ab.
- **Flächengröße:** Um die Korrelation mit dem Code zu gewährleisten, sollte die Fläche der Knoten proportional zum Metrikwert sein. (Hier ist noch unklar, ob das nur für Blätter gilt oder für alle Knoten)
- **Seitenverhältnis:** Um die visuelle Komplexität zu reduzieren und die Verständlichkeit zu erhöhen, sollte das Seitenverhältnis der Knoten möglichst nahe bei 1:1 liegen. Dies verbessert die Lesbarkeit der Knoten und macht es einfacher, die Informationen zu erfassen.
- **Platznutzung:** Es sollte so wenig Fläche wie möglich ohne Informationsgehalt bleiben. Als Gegenbeispiel kann man die Order-Knoten sehen, wie sie in der CodeCity Arbeit beschrieben wurden, bei denen die Fläche der Knoten nicht proportional zur Anzahl der Zeilen im Code ist, wodurch die Fläche an sich keinen Informationsgehalt mehr hat und außerdem viel leere Fläche entsteht.
- **Zeitaufwand:** Die Generierung des Layouts sollte in einem angemessenen Zeitrahmen erfolgen, um eine schnelle Visualisierung zu ermöglichen. Dies verbessert die Skalierbarkeit und generelle Nutzbarkeit der Visualisierung.
- **Stabilität:** Die Knoten sollten bei Änderungen die Position und Größe beibehalten, um eine stabile Visualisierung zu gewährleisten.

In dieser Arbeit sollen space filling approaches analysiert werden und speziell darauf untersucht werden, wie sie sich eignen für die definierten Anforderungen. Wie gut wird was erfüllt? Wann sollte man was anwenden? Kann eine gute Kombination aus verschiedenen Ansätzen gefunden werden? Bisher wurden im Grundlagenteil vor allem die splitting Algorithmen vorgestellt, aber es gibt natürlich auch andere Ansätze, die verfolgt werden können, um Treemap Layouts zu generieren. Zum Beispiel gibt es bin packing oder Optimierungs Algorithmen. In dieser Arbeit sollen auch diese Ansätze betrachtet werden, um zu sehen, ob sie für die Visualisierung von Codequalitätsmetriken in einer space filling layout approach geeignet sind.

“Especially for the node weights, we assess if the algorithms use the weight values to scale the sizes of leaf nodes. Typically, the size of parent nodes is adjusted to the spatial consumption of the child nodes.”[SLD20, S. 3] - Scheibel et al. sagen also, für sie ist es nicht so wichtig, dass die Größe der Elternknoten proportional zu den Metrikwerten der Knoten ist.

Dies soll getestet werden auf Basis von verschiedenen öffentlichen Repositories, die von kleinen bis großen Codebasen reichen. Als Metrik für die Fläche soll der Einfachheit halber die Anzahl der Zeilen verwendet werden.

auch schauen, wann der Algorithmus besonders gut und wann er schlecht ist - hängt natürlich von Input Daten ab. z.B. ab welcher Tiefe ist er gut/schlecht. Ab wie viel Knoten. Ab wie viel Knoten Unterschied in der Größe

## 7.2 Zweifache Berechnung

Die Grundlegende Idee dieser Erweiterung ist es, dass sich die Fläche der Knoten mit Abstand durch das Layout und die Fläche der Knoten ohne Abstand approximieren lässt. Die Idee ist es also einen ersten Durchlauf zu machen, bei dem das Layout ohne Abstand berechnet wird. Dann werden die Größen der Knoten entsprechend dem Layout angepasst, sodass die Größe der Knoten nun auch den Abstand berücksichtigt. Anschließend wird ein zweiter Durchlauf mit diesen angepassten Größen durchgeführt, um das finale Layout zu berechnen.

Bevor wir uns die Details und Ergebnisse dieser Erweiterung anschauen, wollen wir vorweg nehmen, dass diese Erweiterung natürlich nicht optimal funktionieren kann und das auch klar ist, da sich die Änderung der Größe der Knoten natürlich auch das Layout im Zweiten Durchlauf ändern wird, wodurch die Größen der Knoten wieder nicht korrekt sind. Was ja überhaupt erst das Grundlegende Problem ist (siehe Abschnitt 3). Allerdings ist es ein erster Schritt sich dem Problem zu nähern und zu schauen, ob es sich lohnt in diese Richtung weiter zu forschen.

Der Grundlegende Algorithmus bleibt also (fast) gleich, nur dass zwischen dem ersten und dem zweiten Durchlauf ein zusätzlicher Schritt *Größenanpassung* eingefügt wird. Die Einzige Änderung die vorgenommen werden muss ist, dass Knoten nur mit dem definierten Abstand zwischen dem Elternknoten platziert werden können und generell die Fläche des Elternknotens um den Abstand verkleinert wird. Außerdem ist es nötig nach dem

zweiten Durchlauf die Knoten, deren größenwert ja nun den abstand beinhalten, zu verkleinern, um auch den abstand zwischen den Geschwistern herzustellen. Es ist zu erkennen, dass dadurch der Abstand sowohl zwischen Geschwistern als auch zu den Elternknoten den doppelten wert des definierten Abstands hat, dieses Problem ignorieren wir hier, da man es trivialerweise lösen könnte, indem man immer nur die hälfte des Abstands zwischen Geschwistern und Elternknoten abzieht, was wir hier der einfachheit halber nicht tun. – ODER VIELLEICHT HIER IN DER THESIS DOCH? DANN KÖNNTE ICH MIR DIESEN ABSCHNITT SPAREN, AUCH WENN ES IN DER IMPLEMENTIERUNG AM ENDE ANDERS IST

Wir stellen verschiedene Ansätze vor, was sowohl die Größenanpassung als auch die Anpassung der Knoten nach dem zweiten Durchlauf angeht. Die Algorithmen funktionieren allerdings alle nach ähnlichem Prinzip: Es wird zunächst für jeden Knoten die Fläche die der Abstand in diesem Layout benötigen würde addiert, indem die Fläche aus der neuen Länge (alte Länge + 2 mal den Abstand) und der neuen Breite (alte Breite + 2 mal den Abstand) berechnet wird. Zusätzlich wird für Elternknoten die Flächenvergrößerung aller Kinderknoten addiert. An dieser Stelle ist allerdings nicht klar, wie sich die Flächenvergrößerung der Kinderknoten auf die Fläche der Elternknoten auswirkt, da diese Änderung selbst von der Anordnung der Kinderknoten abhängt. Wir testen verschiedene Ansätze, um die Flächenänderung der Elternknoten in Abhängigkeit zu der Flächenänderung der Kinderknoten zu approximieren.

Nach dem zweiten Durchlauf wird nun die Fläche der Knoten so reduziert, dass sowohl der Abstand zwischen Geschwistern als auch der Abstand zu den Elternknoten den gewünschten Wert hat. Dies ist straight forward und wird hier nicht weiter erläutert. Anzumerken ist aber das dieser Schritt speziell abhängt von der Art der Größenanpassung, die im ersten Schritt durchgeführt wurde.

### 7.2.1 *annäherungs wert*

Hier beschreiben, welche phi wert genutzt wurde und warum. Und das dann aus dem grundlagen teil raus nehmen

### 7.2.2 *Größenanpassung*

Dies ist die einfachste naive Version der Größenanpassung, der Algorithmus zeigt aber gut die zuvor beschriebenen Probleme auf. Die Fläche der Knoten wird um den Abstand in beiden Richtungen vergrößert. Zusätzlich wird die Fläche der Elternknoten um die Fläche der Kinderknoten vergrößert.

dies ist ähnoich zu dem cascaded ansatz. wir zeigen genau auf, warum dieser Ansatz große Schwächen hat. bzw Am ehesten ist dieser Ansatz wahrscheinlich mit dem Absulten-Update Ansatz mit Scaling und festen Knoten zu vergleichen, nur mit dem Umterschied, dass die kompletten abstände betrachtet werden und diese betrachtung nur einmalig nach dem ersten squarify Layout Schritt durchgeführt wird (was auch in dem casced paper an-



Abbildung 7.1: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt 2.4.1 mit einem Abstand von 0 und der einfachen Größenanpassung auf der händisch erstellen map (siehe Anhang).

gemerkt wurde, dass das einmalige berechnen der Abstände eine mögliche Optimierung ihres Ansatzes wäre [LFo8, S. 6]).

---

**Algorithm 1** Größenanpassung

---

```

1: function INCREASEVALUESIMPLE(node: SquarifyNode, margin: number)
2:   childrenValueIncrease  $\leftarrow$  0
3:   if node.children then
4:     for child in node.children do
5:       childrenValueIncrease += increaseValuesSimple(child, margin)
6:     end for
7:   end if
8:   valueIncrease  $\leftarrow$  width * margin * 2 + length * margin * 2 + margin *
     margin * 4 + childrenValueIncrease
9:   node.value += valueIncrease
10:  return valueIncrease
11: end function

```

---

Das Problem des Algorithmus ist am besten an einem Beispiel zu verdeutlichen. Wir visualisieren den ZWITEN AnHANG - auch wieder eine händisch erstellte Map, um das Problem zu verdeutlichen. In Abbildung 7.1 ist das Layout mit einem Abstand von 0 zu sehen.

In Abbildung 7.2 ist das Layout mit einem Abstand von 1 zu sehen. Es ist zu erkennen, dass die Knoten auf der linken Seite des Layouts sich außerhalb ihrer Elternknoten erstrecken. Warum passiert das? Knoten 10 ist im zweiten

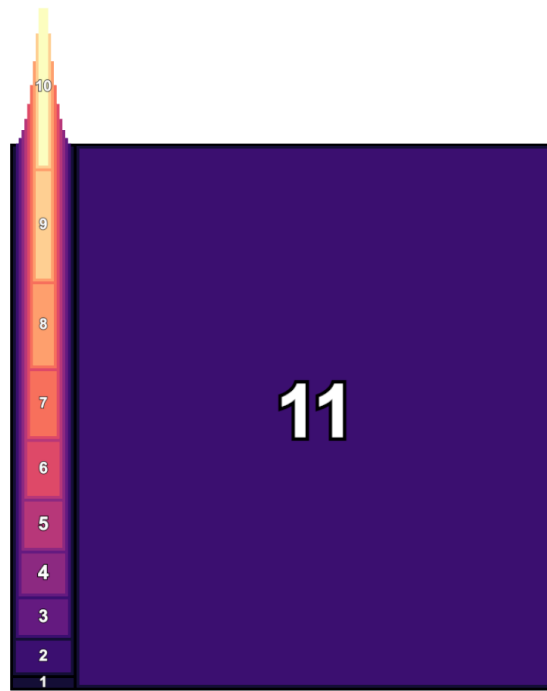


Abbildung 7.2: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt 2.4.1 mit einem Abstand von 1 und der einfachen Größenanpassung auf der händisch erstellen map (siehe Anhang).

Layout-Schritt deutlich schmaler als im ersten Layout-Schritt. Dadurch wird die Fläche die durch den Abstand eingenommen wird größer als angenommen, weshalb die Fläche des Knotens nach abzug des Abstands im letzten Schritt kleiner ist, als gewünscht. Dementsprechend ist auch die Fläche der Knoten unten links größer als gewünscht, da das Layout der Knoten im zweiten Layout-Schritt quadratischer wird. Knoten 5, der Knoten, der am Quadratischsten ist, wird also am größten erscheinen. Obwohl beide den selben wert haben ist Knoten 5 ca. 1,2 mal größer als Knoten 10) In diesem Beispiel erscheint der unterschied kaum merklich, aber es gibt ihn trotzdem und in anderen fällen kann dieser Unterschied merklich werden. Viel signifikanter ist aber der Effekt, dass Elternknoten ebenfalls immer schmaler werden, wodurch die fläche, die der innerere abstand einnimmt, ebenfalls größer wird und dass sogar immer mehr von ebene zu ebene, wenn man runter geht. Dadurch wird die Fläche für die Kindknoten immer kleiner, was dazu führt, dass Knoten teilweise über ihre Elternknoten hinauswachsen.

Dieser Effekt kann einfach behoben werden, wie wir in Abschnitt 7.3 sehen werden.

Bei der berechnung zuvor wurde einfach der margin fläche eines Elternknotens berechnet, auf basis der alten Fläche, ohne die Flächenädnerung durch die Kinderknoten zu berücksichtigen. dadurch wird die resultierende Fläche der Elternknoten zu klein sein, da die Fläche der Kinderknoten nicht berücksichtigt wird. Wir stellen folgend eine zweite Version der Größenanpassung vor, die versucht die Fläche der Elternknoten durch die Fläche der

Kinderknoten zu approximieren. Wir werden dann beide Versionen vergleichen und schauen, welche besser funktioniert. Die Idee bei dieser Erweiterung ist es die Änderung der Kinder schon vor dem Hinzufügen der Abstandsfläche zu berücksichtigen. Dafür muss die relative Flächenänderung durch die Kindknoten berechnet werden. und damit dann die Seitenlängen anpassen und dann die margins hinzufügen, um die neue Fläche zu erhalten. Der Algorithmus wird im folgenden als Pseudocode dargestellt (siehe Algorithmus 2).

---

**Algorithm 2** Relative Größenanpassung
 

---

```

1: function INCREASEVALUES(node: SquarifyNode, margin: number)
2:   width  $\leftarrow$  node.x1 - node.x0
3:   length  $\leftarrow$  node.y1 - node.y0
4:   if node.isLeaf then
5:     valueIncrease  $\leftarrow$  width * margin + length * margin + margin *
      margin * 2
6:     node.value += valueIncrease
7:     return valueIncrease
8:   end if
9:   childrenValueIncrease  $\leftarrow$  0
10:  if node.children then
11:    for child in node.children do
12:      childrenValueIncrease += increaseValuesRelative2(child, mar-
        gin)
13:    end for
14:  end if
15:  ratioChildrenValueIncrease  $\leftarrow$  (node.value + childrenValueIncrease)
    / node.value
16:  valueIncrease  $\leftarrow$  Math.sqrt(ratioChildrenValueIncrease) * width *
    margin * 2 + Math.sqrt(ratioChildrenValueIncrease) * length * margin
    * 2 + margin * margin * 4 + childrenValueIncrease
17:  node.value += valueIncrease
18:  return valueIncrease
19: end function

```

---

Problem: Der Algorithmus in dieser Form zeigt einige Probleme auf. Die Flächenvergrößerung der Kindknoten sagt nichts darüber aus, in welche Richtung sich die Fläche ändert. Es wird davon ausgegangen, dass sich die Fläche gleichmäßig in beide Richtungen ändert (siehe Zeile 13 und 14 in Algorithmus ZEILEN ANPASSEN 2). es kommt also zu ähnlichen Problem wie davor. es können knoten sowohl zu viel platz als auch zu wenig Platz bekommen, jenachdem ob Sie im zweiten schritt quadratischer oder schmaler werden. Siehe Abbildung 7.3 für ein Beispiel.

Evaluation: mit diesen Werten:

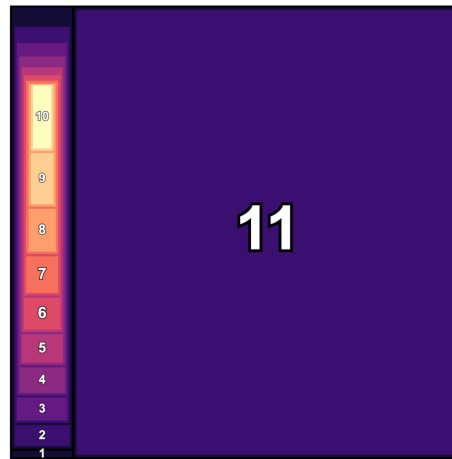


Abbildung 7.3: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt 2.4.1 mit einem Abstand von 1 und der relativen Größenanpassung auf der händisch erstellen map (siehe Anhang).

### 7.3 *Scaling der Knoten*

Wenn die Fläche innerhalb von Elternknoten immer kleiner wird, kann es passieren, dass Knoten über ihre Elternknoten hinauswachsen, wie in Abbildung 7.2 zu sehen ist. Es kann genauso passieren, dass die Fläche innerhalb der Knoten größer wird, wie in Abbildung 7.3 auf der linken Seite zu erkennen ist. Dieser Effekt kann trivialerweise behoben werden, indem der zweite Layout-Schritt angepasst wird, sodass die Knoten immer auf die Fläche des Elternknotens skaliert werden. Vor jedem Squarify-Schritt wird dafür die wirklich zur Verfügung stehende Fläche des Elternknotens berechnet und die Kindknoten entsprechend dieser Änderung skaliert, sodass sie genau in die Fläche des Elternknotens passen.

Der Nachteil dieser Methode ist in Abbildung 7.4 zu erkennen. Die Knoten werden dadurch natürlich nicht mehr proportional zu ihren Werten sein. Knoten 10 hat zum Beispiel ein Verhältnis von ca. 0.5 zu seinem Wert, während Knoten 6 ein Verhältnis von ca. 1.4 zu seinem Wert hat.

Je genauer die Größenanpassung der Knoten ist, desto geringer fällt natürlich dieser Effekt aus. Siehe im Vergleich dazu Abbildung 7.5, da die relative Größenanpassung deutlich genauer ist, ist der Effekt hier auch deutlich geringer. Knoten 10 hat Größe 40 und Knoten 6 hat Größe 41, was fast ähnlich groß ist.

Persönlich würde ich sagen, dass dieser Effekt für das Herunter skalieren der Knoten gut ist, da sonst eine grundlegende Eigenschaft der Darstellung verletzt wird, aber für das Hoch skalieren der Knoten könnte man auch sagen, dass es wichtiger ist die Fläche der Knoten proportional zu ihren Werten zu halten, als die *verschwendete* Fläche aufzufüllen. Diese jetzt hier sehr subjektive Einschätzung wird aber nochmal genauer beleuchtet in der Evaluation und Vergleich.



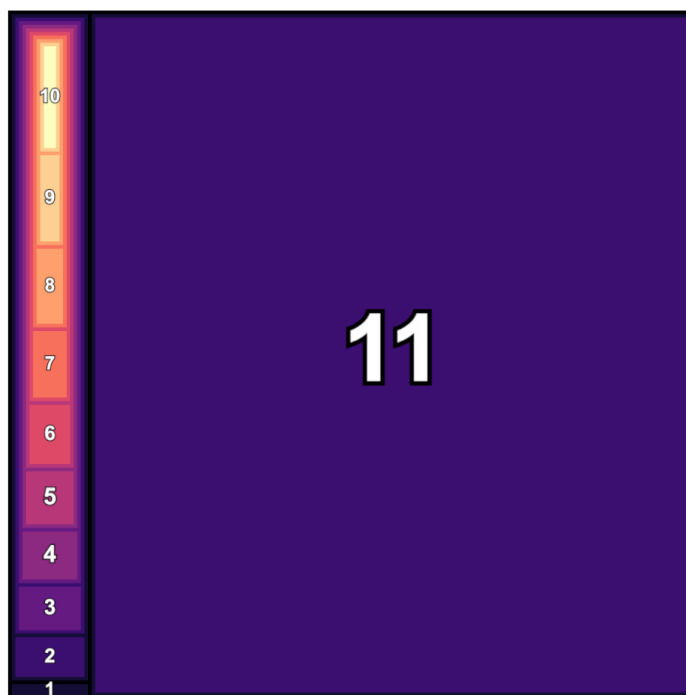


Abbildung 7.4: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt 2.4.1 mit einem Abstand von 1 und der einfachen Größenanpassung auf der händisch erstellen map (siehe Anhang) und der Skalierung der Knoten.

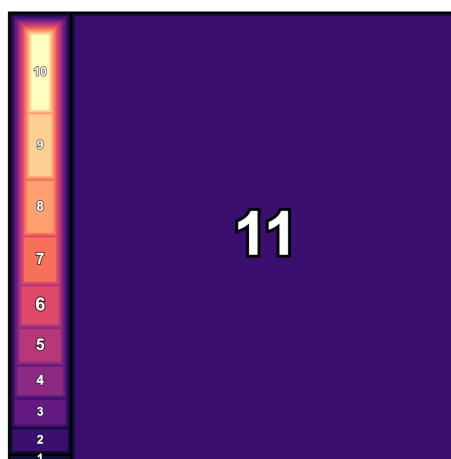


Abbildung 7.5: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt 2.4.1 mit einem Abstand von 1 und der relativen Größenanpassung auf der händisch erstellen map (siehe Anhang) und der Skalierung der Knoten.

#### 7.4 Reihenfolge der Knoten im ersten Layoutschritt

Die Reihenfolge in der die Knoten in das Layout eingefügt werden, hat einen großen Einfluss auf das Layout. [JS91] haben in ihrem Paper herausgefunden, dass die Ergebnisse am besten sind, wenn die Knoten in der Reihenfolge der Größe eingefügt werden. Also der Größte zuerst. Außerdem haben wir in Abschnitt 6.2 gezeigt, dass 95% der Ordner maximal 18 Kindknoten haben. Das Sortieren aller Knoten in einem Ordner nach Größe ist also gar nicht mal so Zeitaufwendig, wie man denken könnte, wenn man tausende von Knoten hat, weil immer nur ein kleines Set von Knoten sortiert werden muss. Kleines Beispiel: Wenn wir von einem typischen Sortieralgorithmus mit einer Laufzeit von  $\mathcal{O}(n \log n)$  (zB. Quicksort), einer Anzahl von 4221 Knoten (Durchschnittswert der Anzahl der Knoten siehe Abschnitt 6.2) und 18 Knoten pro Ordner. Wenn wir alle Knoten auf einmal sortieren wollen:

$$T_1 = n \cdot \log_2(n) = 4221 \cdot \log_2(4221)$$

$$\log_2(4221) \approx 12.04$$

$$T_1 \approx 4221 \cdot 12.04 \approx 50821$$

Wenn wir die Knoten in 18er Bundlen sortieren wollen:

$$T_2 = m \cdot (k \cdot \log_2(k)) = 235 \cdot (18 \cdot \log_2(18))$$

$$\log_2(18) \approx 4.17$$

$$T_2 \approx 235 \cdot (18 \cdot 4.17) = 235 \cdot 75.06 \approx 17639$$

Der Algorithmus mit Sortierung braucht im Schnitt ca. 5.5ms und mit Sortierung ca. 6.5ms. also ca 1ms mehr (Median 0.4ms).

##### 7.4.1 Reihenfolge der Knoten nach erstem Layoutschritt

Durch die Updates der Größen der Knoten nach dem ersten Layoutschritt, kann es passieren, dass die Größenreihenfolge der Knoten sich ändert. Das als zum Beispiel Knoten A im ersten Layoutschritt größer ist als Knoten B, aber nach dem Update der Größen von Knoten A kleiner ist als Knoten B. Das kann dazu führen, dass im folgenden Layoutschritt, die Knoten in einer

anderen Reihenfolge platziert werden als im ersten Layoutschritt. Das ist erstmal gut für die Seitenverhältnisse der Knoten (wenn wir davon ausgehen, dass die Reihenfolge nach Größe sortiert optimal ist), allerdings kann es dazu führen, dass Knoten an ganz anderen Positionen platziert werden, als im ersten Layoutschritt. Das kann dazu führen, dass sich die Form ändert und das kann dazu führen, dass die Kinder anders platziert werden etc. wodurch sich die benötigten Abstände ändern, wodurch die vorausberechnete Fläche der Knoten nicht mehr passt und Knoten entweder verschwinden oder die Wert-Fläche-Relation schlechter wird. Das kann natürlich genauso passieren, wenn die Knoten in der selben Reihenfolge platziert werden, wie im ersten Layoutschritt. Es ist nicht klar ersichtlich, was hier am Ende des Tages am besten ist. Deswegen implementieren wir 3 verschiedene Ansätze:

- **Reihenfolge der Knoten nach Größe:** Die Knoten werden in der Reihenfolge ihrer Größe platziert. Wir vermuten, dass dadurch die besten Seitenverhältnisse entstehen, aber die schlechtesten Wert-Fläche-Relationen.
- **Reihenfolge der Knoten beibehalten:** Die Knoten werden in der Reihenfolge platziert, in der sie im ersten Layoutschritt platziert wurden. Wir vermuten, dass dadurch im Großen und Ganzen die Platzierung der Knoten gleich bleibt, außer es gibt wirklich große Flächenupdates. Wir vermuten, dass dies ein guter Mittelweg zwischen dem ersten und dem jetzt folgenden Ansatz ist.
- **Platzierung der Knoten beibehalten:** Nach dem ersten Layoutschritt wird sich genau die Aufteilung der Knoten in Reihen und die Platzierung der Knoten in den Reihen gemerkt. Wir vermuten, dass dadurch die Form der Knoten im Großen und Ganzen ähnlich zum ersten Schritt bleibt, wodurch die Wert-Fläche-Relationen besser bleibt, aber die Seitenverhältnisse schlechter werden könnten, die die optimale Reihenfolge der Knoten (aufgrund der neuen Werte) nicht mehr berücksichtigt wird.
- 

Es ist wie gesagt nicht ganz klar, was genau die Auswirkungen sind. Deswegen nur Vermutungen, die wir jetzt testen werden.

## 7.5 Mehrfache Berechnung

Die grundlegende Idee dieser Erweiterung ist es, das Layout mehrfach zu berechnen und dabei die Fläche der Knoten immer weiter anzupassen. Warum glauben wir, dass das gut ist. Nach dem ersten Update-Schritt ist die Vorhersage der neuen Fläche ja oft nicht so gut, und es kann sich noch einiges in der Reihenfolge der Knoten ändern oder die Form der Knoten. Dadurch ist nach dem zweiten Layout-Schritt die Platzierung ja anders, dadurch stimmt die vorhergesagte Fläche der Knoten nicht mehr. Dann könnte man nochmal

her gehene und die fläche neu berechnen und nochmal den layoutschrift machen.

Grund dafür: Grund dagegen:

#### 7.5.1 *Iterative Abstandsvergrößerung*

In bild updateValues.png ist zu erkennen, dass kp. margin 10 noP 3

Die Grundlegende Idee dieser Erweiterung ist es, das wenn das Layout mehrfach berechnet wird, es besser ist, den Abstand zwischen den Knoten iterativ von Schritt zu Schritt zu vergrößern, anstatt ihn von Anfang an auf den finalen Wert fest zu setzen. Warum gehen wir von dieser Annahme aus? Wenn man im ersten Schritt den Abstand auf den finalen Wert setzt, dann werden die Vorhersagen für die neue Fläche der Knoten im zweiten Schritt nicht exakt sein, das ist ja klar. Umso kleiner die Abstände sind, umso kleiner sind auch die Abweichungen dieser Vorhersagen. Dadurch sollten die Wert-Fläche-Relationen im zweiten Schritt besser sein. Die Annahme ist jetzt, dass wenn man jedes mal den Abstand um einen kleinen Wert vergrößert, dann sind in jedem Schritt die Vorhersagen für die Fläche der Knoten genauer, wodurch die Wert-Fläche-Relationen besser werden. Als Gegenargument könnte man bringen, dass dadurch, dass jedes mal die Abstände größer werden auch jedes mal das Optimale Layout (man kann das optimale layout nicht berechnen, aber wir gehen mal von einem theoretischen optimalen Layout aus) auch in jedem layout schritt anders sein wird. Dadurch wird erst im letzten Layoutschritt das "wahreoptimale Layout angestrebt, wodurch die vorherigen schritte unnötig waren.

Wir überprüfen jetzt die annahme und schauen, welche begründung richtiger ist/welcher effekt mehr gewicht hat. hier ist es auch interessant den effekt in kombination mit der Reihenfolge der Knoten zu testen, weil

### 7.6 *Änderungen im Layout*

Folgend werden Änderungen beschrieben, die sich sichtbar auf die Visualisierung auswirken und nicht nur auf Platzierung, Größe, Form,... etc der Knoten. Diese Änderungen zielen trotzdem vor allem darauf ab, das Treemap-Layout-Problem zu lösen bzw. abzuschwächen, machen dies aber vor allem durch visuelle Anpassungen und nicht durch komplexe algorithmische Anpassungen.

#### 7.6.1 *Optimierung von Ordnerketten*

Bei der Datenanalyse ist aufgefallen, dass es oft vorkommt, dass Ordner nur ein Kind haben bzw. dass es Ketten von Ordnern gibt, die nur ein Ordner als Kind haben. Häufig sind diese Ketten auch nicht nur einen Ordner tief, was gar nicht so viel Abstand bzw platz verbrauchen würde, doch es gibt auch einige ketten die 4 ordner tief sind, was dann schon wieder (mehr) als viel mal so viel platz für abstände auf braucht durch die Verschachtelung. obwohl im Grunde diese Schachtelung keine wirklichen Strukturin-

formationen enthält. Viele IDEs wie zum Beispiel auch IntelliJ IDEA oder VSCode verbergen diese Ordnerstruktur zum Beispiel auch automatisch, in dem solche Ordnerketten nur als ein Ordner mit entsprechendem zusammengeführten Namen (in dem Fall *main/kotlin/de/maibornwolff/codecharta/analysers/-analyserinterface*) angezeigt wird. Diese Optimierung kann auch für unsere Visualisierung übernommen werden, indem solche Ordnerketten zu einem Ordner zusammengefasst werden. Es ergeben sich folgende Auswirkungen auf die Ziele der Visualisierung:

- **Informationsgehalt und effiziente Nutzung des Platzes:** Wird verbessert, da weniger Platz für Abstände zwischen den Ordnern benötigt wird und weniger Knoten im Layout sind.
- **Niedrige visuelle Komplexität und hohe Verständlichkeit:** Wird verbessert, da weniger Knoten im Layout sind und tiefe Verschachtelungen vermieden werden.
- **Skalierbarkeit:** Wird verbessert, da weniger Knoten im Layout sind und die Berechnung des Layouts beschleunigt wird.
- **Korrelation mit dem Code:** wird verschlechtert, da die Ordnerstruktur nicht mehr 1:1 mit der Code-Struktur übereinstimmt. Die *wahre* Tiefe der Ordnerstruktur ist nicht mehr ersichtlich. Allerdings ist die Tiefe der Ordnerstruktur selbst in eigentlich gar nicht so relevant, die Tiefe wird eher als Mittel benutzt um die Ordnerstruktur darzustellen. Ob eine Datei jetzt 5 oder 10 tief liegt, macht für die Wahrnehmung der Struktur keinen großen Unterschied.
- **Stabilität gegenüber Änderungen:** Wird verbessert, da durch weniger Abstände die Differenz zwischen Knotenfläche und Knotenwert geringer werden, wodurch weniger Umstrukturierung bei der Platzierung der Knoten nötig ist. Außerdem wirken sich trivialerweise Änderungen an Ordnerketten nicht mehr auf die gesamte Struktur aus und verändern somit auch die Visualisierung nicht, was positiv für die Stabilität ist (aber wie gesagt negativ für die Korrelation mit dem Code).

Wir würden sagen, dass die Vorteile dieser Änderung die Nachteile stark überwiegen.

Im Schnitt werden ca. 175 Ordner pro Codebasis gefaltet (12 Median). Bei ca. 958 Knoten (Median 242) sind das ca. 18% (bzw. 5% Median) der Knoten, die durch die Faltung verschwinden. Also schon einige.

Folgend kommen die Daten:

Die Daten zeigen in allen Aspekten eine Verbesserung durch das Falten von Ordnerketten. Wir sprechen uns also dafür aus, dass Ordnerketten immer gefaltet werden sollten, von den messbaren Kriterien her sehen wir keine Gründe, warum das nicht gemacht werden sollte. Das einzige Ziel, was dagegen spricht, ist die Korrelation mit dem Code, die dadurch verschlechtert wird. Alles in allem sprechen wir uns also dafür aus und werden dies fix für unseren Algorithmus implementieren.

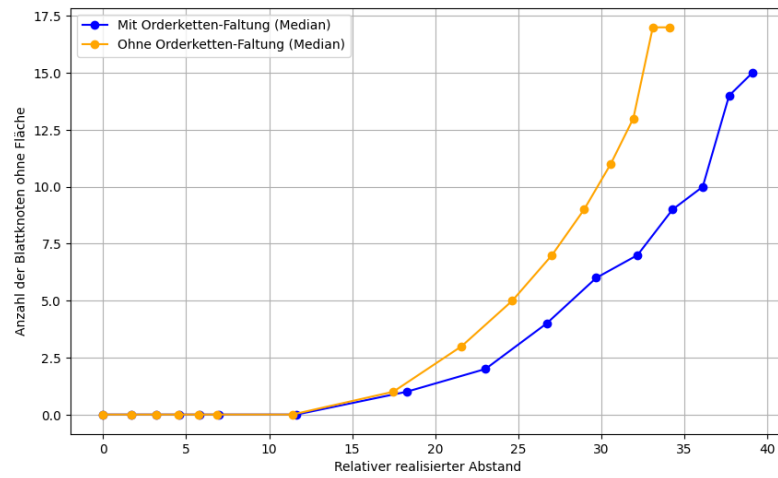


Abbildung 7.6: Die Anzahl der Knoten ohne Fläche geplottet gegenüber des Abstandes. Es ist zu erkennen, dass das Falten von Ordnerketten die das verschwinden von Knoten reduziert.

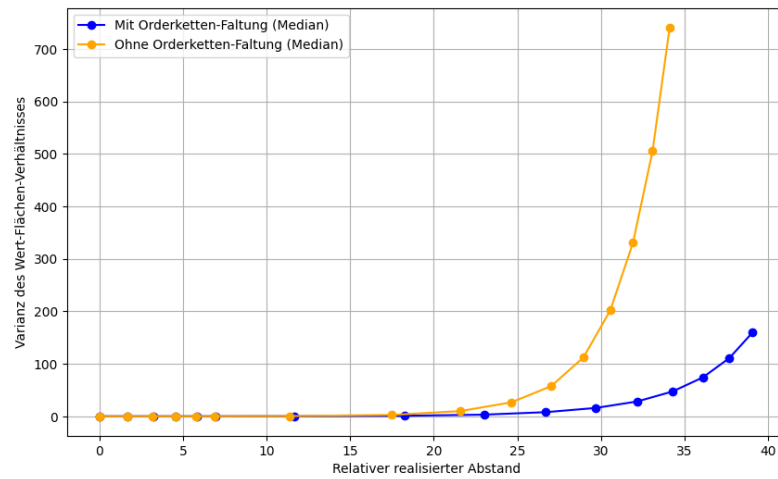


Abbildung 7.7: Das Wert-Fläche-Verhältnis der Knoten geplottet gegenüber des Abstandes. Es ist zu erkennen, dass das Falten von Ordnerketten das Wert-Fläche-Verhältnis der Knoten verbessert.

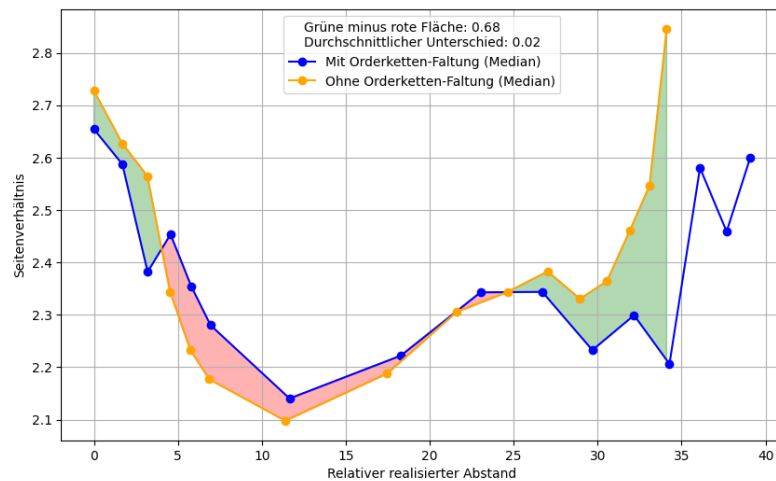


Abbildung 7.8: Das Seitenverhältnis der Knoten geplottet gegenüber des Abstandes. Es ist nicht klar zu erkennen, ob das Falten von Ordnerketten das Seitenverhältnis der Knoten verbessert oder verschlechtert, deshalb sind die Bereiche des positiven effekts grün markiert und die Bereiche des negativen Effekts rot markiert. Es zeigt sich ein leicht positiver Effekt besonders bei kleinen und großen Abständen.

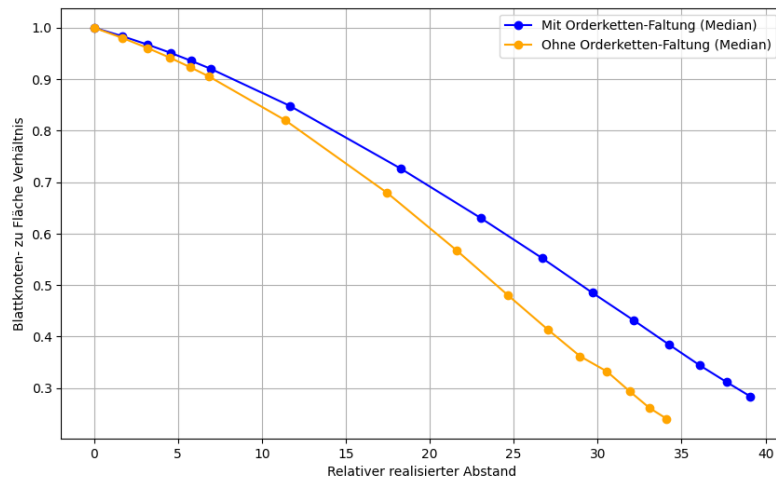


Abbildung 7.9: Die Fläche der Blattknoten in Verhältnis zu der gesamt Fläche geplottet gegenüber des Abstandes. Es ist zu erkennen, dass das Falten von Ordnerketten die Fläche der Blattknoten erhöht. Das heißt, es wird weniger Platz benötigt für Abstände und Ordner, also gleiche Informationen auf weniger Fläche, bedeutet besserer Informationsgehalt.

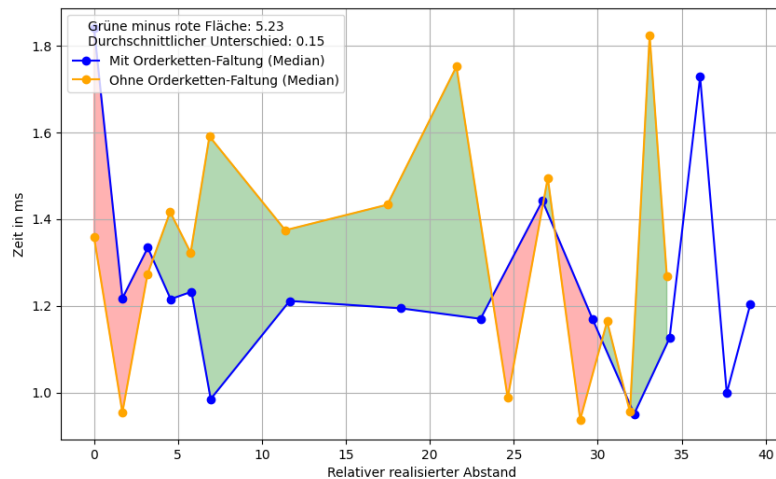


Abbildung 7.10: Die Rechenzeit geplottet gegenüber des Abstandes. Es ist nicht klar zu erkennen, ob das Falten von Ordnerketten die Rechenzeit verbessert oder verschlechtert, deshalb sind die Bereiche des positiven Effekts grün markiert und die Bereiche des negativen Effekts rot markiert. Die Schwankungen sind zwischen ungefähr 1 und 2 millisekunden, was nicht wirklich ausschlaggebend ist. Trotzdem ist ein leichter performance gewinn zu erkennen beim falten, was wahrscheinlich einfach daran liegt, dass es weniger Knoten gibt.

### 7.6.2 Beschriftung der Knoten

Die Beschriftung der Knoten ist ein wichtiger Aspekt der Visualisierung, da sie es dem Nutzer ermöglicht, die Knoten zu identifizieren und zu verstehen, welche Informationen sie repräsentieren. Es gibt verschiedene Ansätze, um die Beschriftung der Knoten zu gestalten. Jeder Ansatz hat seine Vor- und Nachteile. Ein häufig verwendeter Ansatz ist wie zum Beispiel auch beim in Abschnitt VerwandteArbeiten 4 vorgestellten Cascaded Treemap Ansatz [LFo8]. Ordner erhalten eine zusätzliche horizontale Fläche, entweder unten oder oben, die den Namen des Ordners enthält. Dies nimmt allerdings viel Platz ein und erzeugt die selben Schwierigkeiten für den Algorithmus, wie die Abstände zwischen den Knoten. Außerdem kann es passieren, dass die Beschriftung zu lang für die Breite der Knotens ist, wodurch die Beschriftung entweder abgeschnitten wird oder über den Knoten hinauswächst. Hao Lü and James Fogarty schlagen vor dynamisch zu entscheiden, ob die Beschriftung der Knoten angezeigt werden soll oder nicht [LFo8]. Wenn das Hinzufügen der Beschriftung den Knoten dazu führen würde, dass gar kein Platz mehr für den Knoten selbst übrig bleibt, wird die Beschriftung nicht angezeigt. Der Vorteil dieses Ansatzes ist, dass sicher gegangen wird, dass durch die Beschriftung kein Knoten verschwindet. Der Nachteil ist, dass selbst wenn die Änderung der Knotenfläche zum Knotenwert durch das Hinzufügen der Beschriftung riesig wird (wenn die Fläche also super klein wird), die Beschriftung trotzdem hinzugefügt wird.

Wir schlagen einen statischen ansatz vor, bei dem nur die Beschriftungen der Top N order angezeigt werden. Die Wahl von N ist nun die Stellschraube



für die nicht für jede Codebasis gleich ist. Es ist Stellschraube um zwischen Informationsgehalt und visuelle Komplexität zu balancieren.

- Je höher  $N$  ist, desto mehr Informationen (also Ordnernamen) werden angezeigt, was den Informationsgehalt erhöht.
- Je niedriger  $N$  ist, desto weniger Informationen werden angezeigt, was die visuelle Komplexität reduziert, viele Beschriftungen können schnell ablenken und die Visualisierung unübersichtlich machen.
- Außerdem wird die Korrelation mit dem Code verbessert, da ein Übertragen von Visualisierung zur Codebasis einfacher wird mit höherem  $N$ .
- Skalierbarkeit hängt nicht ab von  $N$ , da  $N$  ja nicht mitwächst, sondern statisch ist, ob noch mehr Order und noch mehr Tiefe hinzu kommt, macht keinen wirklichen Unterschied.
- Stabilität gegen Änderungen wenn überhaupt sinkt es, weil Änderung vom Namen z.B. auf einmal sichtbar wird (das ist natürlich gewollt, trotzdem in unserer Messung hier erstmal negativ - wenn auch vernachlässigbar) - eventuell werden dadurch in zwei Versionen die selben Knoten, die jetzt anders heißen, nicht mehr so einfach als gleich identifiziert, obwohl diese die selbe Form haben und ohne die Beschriftung vielleicht anhand der Form als gleich identifiziert werden würden - zugegeben sehr theoretisch und wahrscheinlich nicht relevant aber trotzdem erwähnenswert.

Mit diesem Ansatz hat man natürlich immernoch die gleichen Probleme wie eingangs beschrieben, allerdings sind diese abhängig von dem Wert  $N$  und außerdem werden diese minimiert, da wirklich nur die Top Knoten beschriftet werden, wodurch die Fläche der unteren Knoten trotzdem besser vorhergesagt werden kann und sich dieser Fehler nicht nach oben propagiert.

### 7.6.3 Abstände zwischen Geschwister Knoten

Abstände zwischen Geschwister Knoten sind eine wichtige Entscheidung. Im ursprünglichen Squarify Algorithmus Paper [BHVWoo] wird der Nested Ansatz mit Abständen vorgestellt, jedoch ohne Abstände zwischen Geschwistern. Viele Tools, verwenden jedoch Abstände auch zwischen Geschwisterknoten [WLo7; Gmbc; Gmba]. Was sind die Vor- und Nachteile von Abständen zwischen Geschwistern?

- **Informationsgehalt und effiziente Nutzung des Platzes:** Wird verschlechtert, da jeder Knoten mit Geschwistern mehr Platz benötigt.
- **Niedrige visuelle Komplexität und hohe Verständlichkeit:** Wird verbessert, da die Knoten besser voneinander getrennt werden und somit die Struktur besser erkennbar ist. Allerdings muss das nicht der Fall sein, man könnte zum Beispiel die Kanten der Knoten mit einer anderen Farbe einfärben, um die Knoten besser voneinander zu trennen.

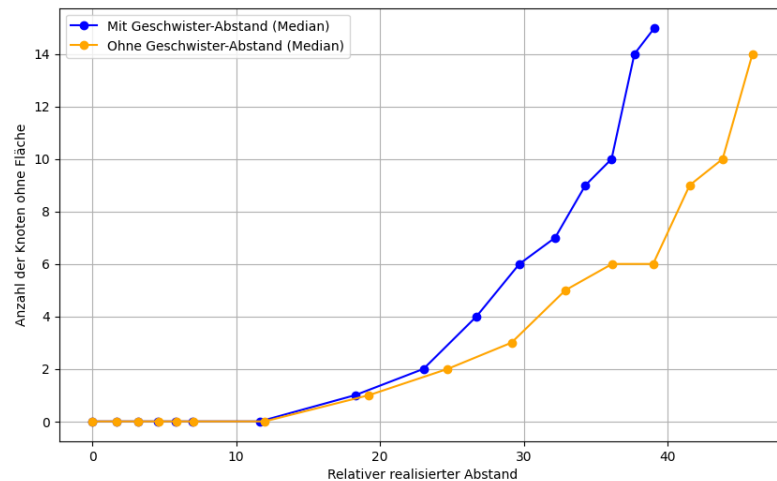


Abbildung 7.11: Die Anzahl der Knoten ohne Fläche geplottet gegenüber des Abstandes.

- **Skalierbarkeit:** Eigentlich wenig auswirkung auf skalierbarkeit, weder die laufzeit ändert sich signifikant (in unsrem algorithmus werden die abstände erzeugt, indem am ende alle knoten um den abstand verkleinert werden - also im grunde kommt laufzeit von  $N$  hinzu, wo  $N$  die Anzahl der Knoten ist, das ist aber nicht ausschlaggebend). Also wenn wird die Skalierbarkeit minimal schlechter.
- **Korrelation mit dem Code:** Keine auswirkung
- **Stabilität gegenüber Änderungen:** Auch nicht wirklich

Die Anpassung, die Knoten mit Kanten zu versehen, natürlich stark abhängig von der gewählten Farbgebung (auch Schattierung, Struktur oder Transparenz). Wie in der Problemstellung definiert (siehe Abschnitt 3) sollte das Layout so gestaltet sein, dass es möglich ist, weitere Metriken durch Farbgebung zu visualisieren. Die Kanten widersprechen dieser Einschränkung nur in Ausnahme fällen. Die meisten Visualisierungen in der Praxis verwenden einfache Einfärbungen [WLo7; Gmbc; Gmba], was bedeutet, dass man einfach Schwarz oder eine Komplementärfarbe als Kantenfarbe nutzen kann, ohne dass es zu Konflikten mit der Farbgebung kommt. Die einzige Ausnahme, die mir bekannt ist, ist wäre die Nutzung von Linien, Schachbrettmuster oder ähnlichen Strukturen, durch die eine Kante nicht mehr als solche erkennbar wäre. Dies könnte zum Beispiel bei dem Ansatz aus dem paper *E-Quality* von Ural Erdemir et al. [ETB11] der Fall sein, da hier zum Beispiel ein Gittermuster verwendet wird, um einen Mangel an Kohäsion zu visualisieren, eine Kante, selbst in anderer Farbe, würde hier eher stören oder in dem Muster untergehen.

## 7.7 Fazit

Immernoch straight forward, es gibt aber noch Probleme

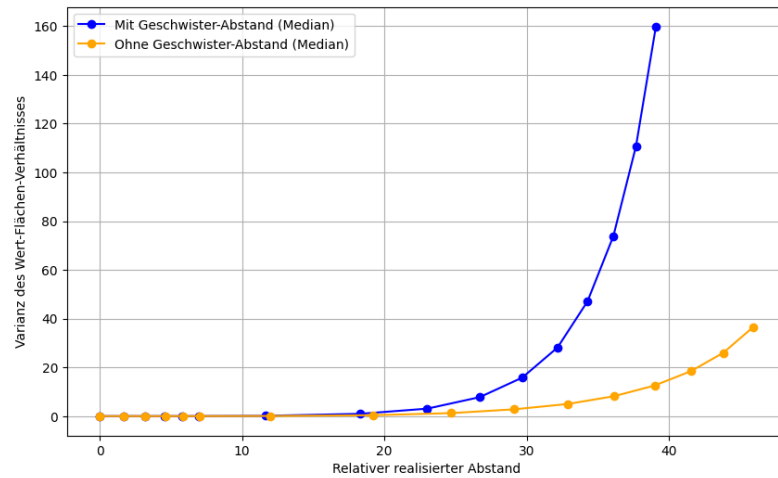


Abbildung 7.12: Das Wert-Fläche-Verhältnis der Knoten geplottet gegenüber des Abstandes. Es ist zu erkennen, dass die Abstände zwischen Geschwistern das Wert-Fläche-Verhältnis der Knoten verbessert.

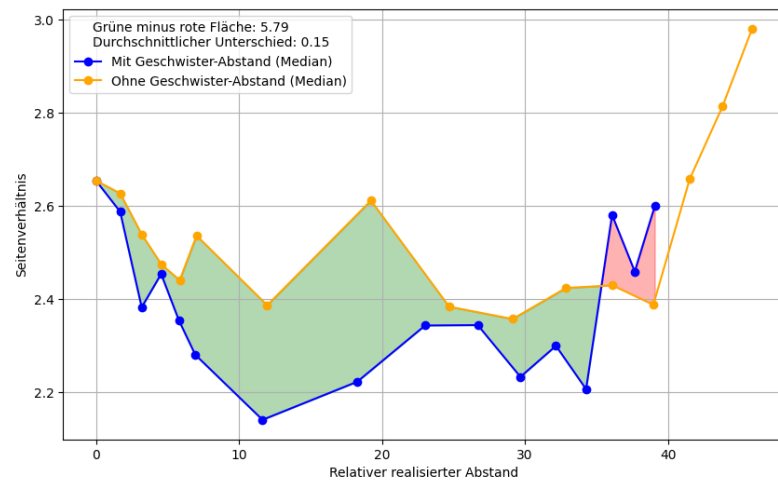


Abbildung 7.13: Das Seitenverhältnis der Knoten geplottet gegenüber des Abstandes. Es ist zu erkennen, dass die Abstände zwischen Geschwistern das Seitenverhältnis der Knoten verbessert.

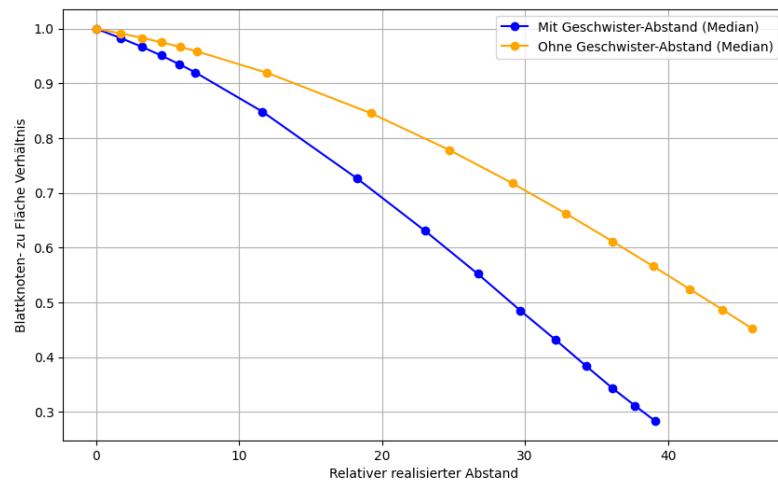


Abbildung 7.14: Die Fläche der Blattknoten in Verhältnis zu der gesamt Fläche geplottet gegenüber des Abstandes. Es ist zu erkennen, dass die Abstände zwischen Geschwistern die Fläche der Blattknoten erhöht. Das heißt, es wird weniger Platz benötigt für Abstände und Ordner, also gleiche Informationen auf weniger Fläche, bedeutet besserer Informationsgehalt.

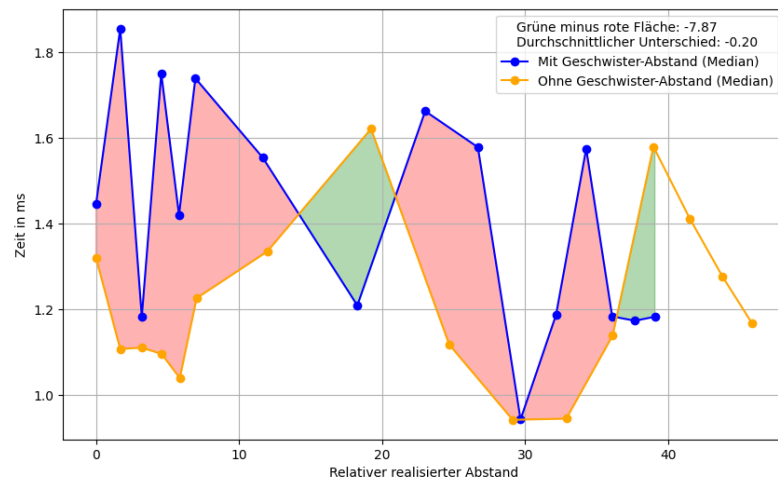


Abbildung 7.15: Die Rechenzeit geplottet gegenüber des Abstandes. Es ist zu erkennen, dass die Abstände zwischen Geschwistern die Rechenzeit nicht signifikant beeinflussen.

Warum besonders den squarify algorithmus betrachtet und nicht zum beipsiel pivot oder circle?? -> weil diese Andere Ziele haben -> noch mehr begründen anhand der definierten Ziele

## EVALUATION

---

Evaluation anhand eines echten beispiels mit verschiedenen Layouts. In diesem Abschnitt werden 1. die verschiedenen Treemap-Algorithmen verglichen und anhand von konkreten Kriterien evaluiert, um den besten Algorithmus für die Visualisierung von Codequalitätsmetriken zu finden. 2. Werden alle vorgestellten Layouts anhand eines konkreten Beispiels aus der Praxis evaluiert. Dies wird gemacht, indem eine echte, bereits durchgeführte, Software-Analyse nochmal mit den verschiedenen Layouts visualisiert wird. Dann wird verschiedenen unerfahrenen Personen diese verschiedenen Visualisierungen gezeigt und gefragt, wie sie die Software-Qualität einschätzen würden. Wenn das möglichst nahe an die Experten Einschätzung kommt, so wie es in der Analyse durchgeführt wurde, dann ist das Layout gut geeignet.

## FAZIT

---

In dieser arbeit konnte natürlich ein eine endliche Anzahl an Ansätzen und Ideen untersucht werden. In Zukunft ist bestimmt noch viel kreativität und Forschungspotential in diesem Bereich. Außerdem vielleicht sogar komplett neue Ansätze, die nicht auf der Stadt-Metapher basieren sondern vielleicht auf planeten oder anderen Metaphern, auch wenn zu vermuten ist, dass dabei die übersichlichekeit und einfachheit leiden könnte. Die in diser arbeit untersuchten Ansätze sind alle sehr einfach mit basic formen und daher übersichtlich.

Es ist natürlich klar, dass der treempa ansatz optmiert wurde und die anderen Ansätze nicht, weshalb der treemap ansatz besere ergebnisse liefert. Es bleibt zu erforschen, ob sich die anderen Ansätze auch so optimieren lassen, dass sie bessere Ergebnisse liefern.

Außerdem ist bei der literatur recherche noch interessant auch komplett über den tellerrand von software hinaus zu schauen und andere hierarchical vis zu betrachten, die nicht direkt mit software zu tun haben, um eventuell neue gut viz zu finden. Oder sogar ganz anders mit Kreativen ideen findungs ansätzen komplett neue Ansätze zu finden. Oder sich neue methaphern zu überlegen und nicht nur stadt, inseln, weltall,....

Außerdem ist es interannt in der umfrage die tools den nutzern direkt zur verfügung zu stellen. In dieser arbeit wurden tools nur mit statischen bildern vorgestllt, um sich konkret nur auf die Visualisierung zu konzentrieren und faktoren wie nutzerfreundlichkeit, performance und interaktion nicht mit zu betrachten, wie bereits in problemstellung erwähnt. In zukunft wäre es zum beispiel interessant verschiedene interaktionsmöglichkeiten mit den layouts, wie zoomen, filtern, suchen, navigieren, etc zu untersuchen und zu vergleichen

Außerdem kann es passieren, dass die Beschriftung zu lang für die breite der Knotens ist, wodurch die Beschriftung entweder abgeschnitten wird oder über den Knoten hinauswächst."das angehen, mit zum beispiel rechteck breiter machen oder so, anders paltzieren, PIPAPO

Außerdem eventuell dynamische Beschriftung implementieren

Auch interessant wenn die struktur oder die metriken nicht auf file, sondern zum beispiel auf klassen basis sind. schauen, ob das einen unterschied macht.

Bewusst, dass gewisse wissenschaftliche standards nicht zu 100 Przent eingehalten wurden Zb. hat nur eine person am code gearbeitet, es wurde speziell der algorithmus so lange implementiert, bis die ergebnisse korrekt aussahen, das ist keine 100 prozentige garantie, dass zb die evaluation wirklich korrekt und fehlerfrei ist. interessant wäre nochmal eine arbeit, die den code überprüft. aber bei software engineering immer so, dass es bugs und

fehler gibt, nie auszuschließen. aber in dieser arbeit größte vorsicht und gewissenhaftigkeit - auch wenn das nicht wissenschaftlich ist. Deswegen stellen wir den kompletten quellcode zur verfügung, damit andere die Ergebnisse überprüfen können. Vorallem wichtig, dass man sich dessen bewusst ist, auch wenn man nichts dagegen tun kann



Teil II

APPENDIX

DATEIEN

---

Alle Dateien sind auch digital unter diesem [Link \[Meh25a\]](#) zu finden.

```
{
  "nodes": [
    {
      "name": "root",
      "type": "Folder",
      "attributes": {},
      "children": [
        {
          "name": "leaf 0",
          "type": "File",
          "attributes": {
            "rloc": 10,
            "functions": 5,
            "mcc": 100
          }
        },
        {
          "name": "small_parent_1",
          "type": "Folder",
          "attributes": {},
          "children": [
            {
              "name": "leaf 1",
              "type": "File",
              "attributes": {
                "rloc": 30,
                "functions": 100,
                "mcc": 100
              }
            },
            {
              "name": "small_parent_2",
              "type": "Folder",
              "attributes": {},
              "children": [
                {
                  "name": "leaf 2",
                  "type": "File",
                  "attributes": {
```

```

        "rloc": 30,
        "functions": 10,
        "mcc": 100
    }
},
{
    "name": "small_parent_3",
    "type": "Folder",
    "attributes": {},
    "children": [
        {
            "name": "leaf 3",
            "type": "File",
            "attributes": {
                "rloc": 30,
                "functions": 10,
                "mcc": 100
            }
        }
    ]
}
]
}
]
},
{
    "name": "huge_parent",
    "type": "Folder",
    "attributes": {},
    "children": [
        {
            "name": "huge leaf",
            "type": "File",
            "attributes": {
                "rloc": 3000,
                "functions": 100,
                "mcc": 100
            }
        }
    ]
}
]
}
]
}
]
}

```

Listing A.1: Händisch erzeugte map

```

{
  "nodes": [
    {
      "name": "root",
      "type": "Folder",
      "attributes": {},
      "children": [
        {
          "name": "leaf 0",
          "type": "File",
          "attributes": {
            "rloc": 10,
            "functions": 5,
            "mcc": 100
          }
        },
        {
          "name": "small_parent_1",
          "type": "Folder",
          "attributes": {},
          "children": [
            {
              "name": "leaf 1",
              "type": "File",
              "attributes": {
                "rloc": 30,
                "functions": 100,
                "mcc": 100
              }
            }
          ],
          "name": "small_parent_2",
          "type": "Folder",
          "attributes": {},
          "children": [
            {
              "name": "leaf 2",
              "type": "File",
              "attributes": {
                "rloc": 30,
                "functions": 10,
                "mcc": 100
              }
            }
          ],
          "name": "small_parent_3",

```

```

"type": "Folder",
"attributes": {},
"children": [
  {
    "name": "leaf 3",
    "type": "File",
    "attributes": {
      "rloc": 30,
      "functions": 10,
      "mcc": 100
    }
  },
  {
    "name": "small_parent_4",
    "type": "Folder",
    "attributes": {},
    "children": [
      {
        "name": "leaf 4",
        "type": "File",
        "attributes": {
          "rloc": 30,
          "functions": 10,
          "mcc": 100
        }
      }
    ],
    {
      "name": "small_parent_5",
      "type": "Folder",
      "attributes": {},
      "children": [
        {
          "name": "leaf 5",
          "type": "File",
          "attributes": {
            "rloc": 30,
            "functions": 10,
            "mcc": 100
          }
        }
      ],
      {
        "name": "small_parent_6",
        "type": "Folder",
        "attributes": {},
        "children": [
          {

```

```

"name": "leaf 6",
"type": "File",
"attributes": {
  "rloc": 30,
  "functions": 10,
  "mcc": 100
}
},
{
  "name": "small_parent_7",
  "type": "Folder",
  "attributes": {},
  "children": [
    {
      "name": "leaf 7",
      "type": "File",
      "attributes": {
        "rloc": 30,
        "functions": 10,
        "mcc": 100
      }
    }
  ],
  {
    "name": "small_parent_8",
    "type": "Folder",
    "attributes": {},
    "children": [
      {
        "name": "leaf 8",
        "type": "File",
        "attributes": {
          "rloc": 30,
          "functions": 10,
          "mcc": 100
        }
      }
    ],
    {
      "name":
        "small_parent_9",
      "type": "Folder",
      "attributes": {},
      "children": [
        {
          "name": "leaf 9",
          "type": "File",
          "attributes": {

```



```

{
  "link": "https://github.com/nvbn/thefuck",
  "description": "Magnificent app which corrects your
    previous console command."
},
{
  "link": "https://github.com/scikit-learn/scikit-learn",
  "description": "scikit-learn: machine learning in Python"
},
{
  "link": "https://github.com/ansible/ansible",
  "description": "Ansible is a radically simple IT
    automation platform that makes your applications and
    systems easier to deploy and maintain. Automate
    everything from code deployment to network
    configuration to cloud management, in a language that
    approaches plain English, using SSH, with no agents
    to install on remote systems.
    https://docs.ansible.com."
},
{
  "link": "https://github.com/CorentinJ/Real-Time-Voice-Cloning",
  "description": "Clone a voice in 5 seconds to generate
    arbitrary speech in real-time"
},
{
  "link": "https://github.com/apache/airflow",
  "description": "Apache Airflow - A platform to
    programmatically author, schedule, and monitor
    workflows"
},
{
  "link": "https://github.com/harry0703/MoneyPrinterTurbo",
  "description": "Generate short videos with one click using
    AI LLM."
},
{
  "link": "https://github.com/freqtrade/freqtrade",
  "description": "Free, open source crypto trading bot"
},
{
  "link": "https://github.com/streamlit/streamlit",
  "description": "Streamlit A faster way to build and share
    data apps."
},
{

```



```

    "link":"https://github.com/mlflow/mlflow",
    "description":"The open source developer platform to
        build AI/LLM applications and models with confidence.
        Enhance your AI applications with end-to-end
        tracking, observability, and evaluations, all in one
        integrated platform."
},
{
    "link":"https://github.com/commaai/openpilot",
    "description":"openpilot is an operating system for
        robotics. Currently, it upgrades the driver
        assistance system on 300+ supported cars."
},
{
    "link":"https://github.com/pytorch/pytorch",
    "description":"Tensors and Dynamic neural networks in
        Python with strong GPU acceleration"
},
{
    "link":"https://github.com/pytorch/vision",
    "description":"Datasets, Transforms and Models specific
        to Computer Vision"
},
{
    "link":"https://github.com/frappe/erpnext",
    "description":"Free and Open Source Enterprise Resource
        Planning (ERP)"
},
{
    "link":"https://github.com/StevenBlack/hosts",
    "description":"Consolidating and extending hosts files
        from several well-curated sources. Optionally pick
        extensions for porn, social media, and other
        categories."
},
{
    "link":"https://github.com/macrozheng/mall",
    "description":"mallBoot+MyBatis"
},
{
    "link":"https://github.com/zxing/zxing",
    "description":"ZXing (Crossingbarcode scanning library
        for Java, Android"
},
{
    "link":"https://github.com/scwang90/SmartRefreshLayout",

```

```

    "description": "Footer"
  },
  {
    "link": "https://github.com/oracle/graal",
    "description": "GraalVM compiles Java applications into
      native executables that start instantly, scale fast,
      and use fewer compute resources "
  },
  {
    "link": "https://github.com/keycloak/keycloak",
    "description": "Open Source Identity and Access Management
      For Modern Applications and Services"
  },
  {
    "link": "https://github.com/doocs/leetcode",
    "description": "solutions in any programming language |
      LeetCodeOffer2 6 "
  },
  {
    "link": "https://github.com/neo4j/neo4j",
    "description": "Graphs for Everyone"
  },
  {
    "link": "https://github.com/deeplearning4j/deeplearning4j",
    "description": "Suite of tools for deploying and training
      deep learning models using the JVM. Highlights
      include model import for keras, tensorflow, and
      onnx/pytorch, a modular and tiny c++ library for
      running math code and a java based math library on
      top of the core c++ library. Also includes samediff:
      a pytorch/tensorflow like library for running deep
      learn..."
  },
  {
    "link": "https://github.com/OpenAPITools/openapi-generator",
    "description": "OpenAPI Generator allows generation of API
      client libraries (SDK generation), server stubs,
      documentation and configuration automatically given
      an OpenAPI Spec (v2, v3)"
  },
  {
    "link": "https://github.com/SonarSource/sonarqube",
    "description": "Continuous Inspection"
  },
  {
    "link": "https://github.com/libgdx/libgdx",

```

```

        "description": "Desktop/Android/HTML5/iOS Java game
                        development framework"
    },
    {
        "link": "https://github.com/flyway/flyway",
        "description": "Flyway by Redgate Database Migrations Made
                        Easy."
    },
    {
        "link": "https://github.com/openjdk/jdk",
        "description": "JDK main-line development
                        https://openjdk.org/projects/jdk"
    },
    {
        "link": "https://github.com/arduino/Arduino",
        "description": "Arduino IDE 1.x"
    },
    {
        "link": "https://github.com/sveltejs/svelte",
        "description": "web development for the rest of us"
    },
    {
        "link": "https://github.com/danielmiessler/Fabric",
        "description": "Fabric is an open-source framework for
                        augmenting humans using AI. It provides a modular
                        system for solving specific problems using a
                        crowdsourced set of AI prompts that can be used
                        anywhere."
    },
    {
        "link": "https://github.com/open-webui/open-webui",
        "description": "User-friendly AI Interface (Supports
                        Ollama, OpenAI API, ...)"
    },
    {
        "link": "https://github.com/atom/atom",
        "description": ":atom: The hackable text editor"
    },
    {
        "link": "https://github.com/phaserjs/phaser",
        "description": "Phaser is a fun, free and fast 2D game
                        framework for making HTML5 games for desktop and
                        mobile web browsers, supporting Canvas and WebGL
                        rendering."
    },
    {

```

```

    "link": "https://github.com/swagger-api/swagger-ui",
    "description": "Swagger UI is a collection of HTML,
        JavaScript, and CSS assets that dynamically generate
        beautiful documentation from a Swagger-compliant API."
  },
  {
    "link": "https://github.com/handsontable/handsontable",
    "description": "JavaScript Data Grid / Data Table with a
        Spreadsheet Look & Feel. Works with React, Angular,
        and Vue. Supported by the Handsontable team "
  },
  {
    "link": "https://github.com/zhaoolee/ChromeAppHeroes",
    "description": " ChromePluginHeroes, Write a Chinese
        manual for the excellent Chrome plugin, let the
        Chrome plugin heroes benefit the human~ 01"
  },
  {
    "link": "https://github.com/aframevr/aframe",
    "description": ":a: Web framework for building virtual
        reality experiences."
  },
  {
    "link": "https://github.com/4ian/GDevelop",
    "description": "Open-source, cross-platform
        2D/3D/multiplayer game engine designed for everyone."
  },
  {
    "link": "https://github.com/facebook/react",
    "description": "The library for web and native user
        interfaces."
  },
  {
    "link": "https://github.com/gatsbyjs/gatsby",
    "description": "The best React-based framework with
        performance, scalability and security built in."
  },
  {
    "link": "https://github.com/nodejs/node",
    "description": "Node.js JavaScript runtime "
  },
  {
    "link": "https://github.com/ToolJet/ToolJet",
    "description": "Low-code platform for building business
        applications. Connect to databases, cloud storages,
        GraphQL, API endpoints, Airtable, Google sheets,

```

```

        OpenAI, etc and build apps using drag and drop
        application builder. Built using
        JavaScript/TypeScript. "
    },
    {
        "link":"https://github.com/romkatv/powerlevel10k",
        "description":"A Zsh theme"
    },
    {
        "link":"https://github.com/testssl/testssl.sh",
        "description":"Testing TLS/SSL encryption anywhere on any
        port "
    },
    {
        "link":"https://github.com/linux-surface/linux-surface",
        "description":"Linux Kernel for Surface Devices"
    },
    {
        "link":"https://github.com/ONLYOFFICE/DocumentServer",
        "description":"ONLYOFFICE Docs is a free collaborative
        online office suite comprising viewers and editors
        for texts, spreadsheets and presentations, forms and
        PDF, fully compatible with Office Open XML formats:
        .docx, .xlsx, .pptx and enabling collaborative
        editing in real time."
    },
    {
        "link":"https://github.com/facebookarchive/caffe2",
        "description":"Caffe2 is a lightweight, modular, and
        scalable deep learning framework."
    },
    {
        "link":"https://github.com/mitchellkrogza/
        nginx-ultimate-bad-bot-blocker",
        "description":"Nginx Block Bad Bots, Spam Referrer
        Blocker, Vulnerability Scanners, User-Agents,
        Malware, Adware, Ransomware, Malicious Sites, with
        anti-DDOS, Wordpress Theme Detector Blocking and
        Fail2Ban Jail for Repeat Offenders"
    },
    {
        "link":"https://github.com/dtcooper/raspotify",
        "description":"A Spotify Connect client that mostly Just
        Works"
    },
    {

```

```

    "link": "https://github.com/armbian/build",
    "description": "Armbian Linux build framework generates
                    custom Debian or Ubuntu image for x86, aarch64,
                    riscv64 & armhf"
  },
  {
    "link": "https://github.com/filsv/iOSDeviceSupport",
    "description": "Xcode iPhoneOS (iOS) DeviceSupport files
                    (6.0 - 17.0)"
  },
  {
    "link": "https://github.com/prasanthrangan/hyprdots",
    "description": "// Aesthetic, dynamic and minimal dots for
                    Arch hyprland"
  },
  {
    "link": "https://github.com/hoverbike1/TOTK-Mods-collection",
    "description": "Mod repo for Tears of The Kingdom (TOTK)
                    for Switch and Switch Emulation"
  },
  {
    "link": "https://github.com/HyDE-Project/HyDE",
    "description": "HyDE, your Development Environment "
  },
  {
    "link": "https://github.com/yonggekkk/x-ui-yg",
    "description": "x-uiXhttpargoPsiphonVPN30sing-boxclash-meta"
  },
  {
    "link": "https://github.com/fuzhengwei/CodeGuide",
    "description": ":books: Java Java()"
  },
  {
    "link": "https://github.com/pbatard/rufus",
    "description": "The Reliable USB Formatting Utility"
  },
  {
    "link": "https://github.com/mpv-player/mpv",
    "description": "Command line media player"
  },
  {
    "link": "https://github.com/jart/cosmopolitan",
    "description": "build-once run-anywhere c library"
  },
  {
    "link": "https://github.com/coolsnowwolf/lede",

```

```

      "description": "Lean's LEDE source"
    },
    {
      "link": "https://github.com/wazuh/wazuh",
      "description": "Wazuh - The Open Source Security Platform.
        Unified XDR and SIEM protection for endpoints and
        cloud workloads."
    },
    {
      "link": "https://github.com/videolan/vlc",
      "description": "VLC media player - All pull requests are
        ignored, please use MRs on
        https://code.videolan.org/videolan/vlc"
    },
    {
      "link": "https://github.com/reactos/reactos",
      "description": "A free Windows-compatible Operating System"
    },
    {
      "link": "https://github.com/TelegramMessenger/Telegram-iOS",
      "description": "Telegram-iOS"
    },
    {
      "link": "https://github.com/yugabyte/yugabyte-db",
      "description": "YugabyteDB - the cloud native distributed
        SQL database for mission-critical applications."
    },
    {
      "link": "https://github.com/OnionUI/Onion",
      "description": "OS overhaul for Miyoo Mini and Mini+"
    },
    {
      "link": "https://github.com/wine-mirror/wine",
      "description": null
    },
    {
      "link": "https://github.com/wireshark/wireshark",
      "description": "Read-only mirror of Wireshark's Git
        repository at https://gitlab.com/wireshark/wireshark.
        GitHub won't let us disable pull requests. THEY WILL
        BE IGNORED HERE Upload them at GitLab instead."
    },
    {
      "link": "https://github.com/darktable-org/darktable",
      "description": "darktable is an open source photography
        workflow application and raw developer"
    }
  ]
}

```

```
    },  
    {  
      "link": "https://github.com/hanwckf/rt-n56u",  
      "description": "Padavan"  
    }  
  ]
```

Listing A.3: Die zur Analyse ausgewählten GitHub-Repositories



## LITERATUR

---

- [Sofa] [Online; besucht 17-Juni-2025]. 2019. URL: <https://www.studysmarter.de/studium/informatik-studium/softwareentwicklung/softwaremetriken/>.
- [Sofb] [Online; besucht 22-Juni-2025]. 2022. URL: <https://octoverse.github.com/2022/state-of-open-source>.
- [Iso] 2024. URL: <https://www.iso25000.com/index.php/en/iso-25000-standards/iso-25010>.
- [Wor] [Online; besucht 17-Juni-2025]. 2025. URL: <https://www.gitcler.com/four\%5Fworst\%5Fsoftware\%5Fmetrics\%5Fagitating\%5Fdevelopers>.
- [D3ta] [Online; besucht 19-Mai-2025]. 2025. URL: <https://d3js.org/d3-hierarchy/treemap>.
- [D3tb] [Online; besucht 27-Juli-2025]. 2025. URL: <https://github.com/d3/d3-hierarchy/blob/main/src/treemap/squarify.js>.
- [Iee] [Online; besucht 29-Juli-2025]. 2025. URL: <https://ieeexplore.ieee.org/Xplorehelp/searching-ieee-xplore/search-examples>.
- [AO+19] Islam Al Omari, Razan Al Omoush, Haneen Innab und A Elhassan. "Visualizing Program Quality-A Topological Taxonomy of Features". In: *2019 2nd International Conference on new Trends in Computing Sciences (ICTCS)*. IEEE. 2019, S. 1–10.
- [ADo7] Sazzadul Alam und Philippe Dugerdil. "EvoSpaces: 3D Visualization of Software Architecture." In: *SEKE*. Bd. 7. Citeseer. 2007, S. 500.
- [Bal+16] Gergo Balogh, Tamás Gergely, Árpád Beszédes und Tibor Gyimóthy. "Using the City Metaphor for Visualizing Test-Related Metrics". In: *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. Bd. 2. 2016, S. 17–20. DOI: [10.1109/SANER.2016.48](https://doi.org/10.1109/SANER.2016.48).
- [BD17] Sebastian Baltes und Stephan Diehl. "Sketches and Diagrams in Practice". In: *CoRR* abs/1706.09172 (2017). arXiv: [1706.09172](https://arxiv.org/abs/1706.09172). URL: <http://arxiv.org/abs/1706.09172>.
- [BD05] Michael Balzer und Oliver Deussen. "Exploring relations within software systems using treemap enhanced hierarchical graphs". In: *3rd IEEE international workshop on visualizing software for understanding and analysis*. IEEE. 2005, S. 1–6.

- [BDL05] Michael Balzer, Oliver Deussen und Claus Lewerentz. "Voronoi treemaps for the visualization of software metrics". In: *Proceedings of the 2005 ACM symposium on Software visualization*. 2005, S. 165–172.
- [BK01] S. Bassil und R.K. Keller. "Software visualization tools: survey and analysis". In: *Proceedings 9th International Workshop on Program Comprehension. IWPC 2001*. 2001, S. 7–17. DOI: [10.1109/WPC.2001.921708](https://doi.org/10.1109/WPC.2001.921708).
- [BHVW00] Mark Bruls, Kees Huizing und Jarke J Van Wijk. "Squarified treemaps". In: *Data Visualization 2000: Proceedings of the Joint EUROGRAPHICS and IEEE TCVG Symposium on Visualization in Amsterdam, The Netherlands*. Springer. 2000, S. 33–42.
- [Bur15] Michael Burch. "Visualizing software metrics in a software system hierarchy". In: *International Symposium on Visual Computing*. Springer. 2015, S. 733–744.
- [CAC20] Andrea Capiluppi, Nemitari Ajenka und Steve Counsell. "The effect of multiple developers on structural attributes: A Study based on java software". In: *Journal of Systems and Software* 167 (2020), S. 110593. ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2020.110593>. URL: <https://www.sciencedirect.com/science/article/pii/S016412122030073X>.
- [CZ10] Pierre Caserta und Olivier Zendra. "Visualization of the static aspects of software: A survey". In: *IEEE transactions on visualization and computer graphics* 17.7 (2010), S. 913–933.
- [Cru+16] Adriana Cruz, Camila Bastos, Paulo Afonso und Heitor Costa. "Software visualization tools and techniques: A systematic review of the literature". In: *2016 35th International Conference of the Chilean Computer Science Society (SCCC)*. IEEE. 2016, S. 1–12.
- [DFo6] Peter Demian und Renate Fruchter. "Finding and Understanding Reusable Designs from Large Hierarchical Repositories". In: *Information Visualization* 5.1 (2006), S. 28–46. DOI: [10.1057/palgrave.ivs.9500114](https://doi.org/10.1057/palgrave.ivs.9500114). eprint: <https://doi.org/10.1057/palgrave.ivs.9500114>. URL: <https://doi.org/10.1057/palgrave.ivs.9500114>.
- [Die] "Dynamic Program Visualization". In: *Software Visualization: Visualizing the Structure, Behaviour, and Evolution of Software*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, S. 79–127. ISBN: 978-3-540-46505-8. DOI: [10.1007/978-3-540-46505-8\\_4](https://doi.org/10.1007/978-3-540-46505-8_4). URL: [https://doi.org/10.1007/978-3-540-46505-8\\_4](https://doi.org/10.1007/978-3-540-46505-8_4).
- [EH98] Institute of Electrical und Electronics Engineers (Hrsg.) *IEEE Std 1061-1998: IEEE Standard for a Software Quality Metrics Methodology*. Kapitel 2. Definitions, S. 2. New York: IEEE, 1998. ISBN: 1-55937-529-9.

- [ETB11] Ural Erdemir, Umut Tekin und Feza Buzluca. “E-Quality: A graph based object oriented software quality visualization tool”. In: *2011 6th International Workshop on Visualizing Software for Understanding and Analysis (VISSOFT)*. 2011, S. 1–8. DOI: [10.1109/VISSOFT.2011.6069454](https://doi.org/10.1109/VISSOFT.2011.6069454).
- [GHMo8] Keith Gallagher, Andrew Hatch und Malcolm Munro. “Software architecture visualization: An evaluation framework and its application”. In: *IEEE Transactions on Software Engineering* 34.2 (2008), S. 260–270.
- [Gen+21] Patric Genfer, Johann Grabner, Christina Zoffi, Mario Bernhart und Thomas Grechenig. “Visualizing Metric Trends for Software Portfolio Quality Management”. In: *2021 Working Conference on Software Visualization (VISSOFT)*. 2021, S. 88–99. DOI: [10.1109/VISSOFT52517.2021.00018](https://doi.org/10.1109/VISSOFT52517.2021.00018).
- [Gmba] Generative AI Seerene GmbH. *Sereene Website*. [Online; besucht 29-Juli-2025]. URL: <https://www.seerene.com/de/>.
- [Gmbb] MaibornWolff GmbH. *CodeCharta GitHub*. [Online; besucht 04-August-2025]. URL: <https://github.com/MaibornWolff/codecharta>.
- [Gmbc] MaibornWolff GmbH. *CodeCharta Web Demo*. [Online; besucht 29-Juli-2025]. URL: <https://codecharta.com/visualization/app/index.html>.
- [Gmbd] MaibornWolff GmbH. *CodeCharta Wiki Analysis*. [Online; besucht 31-Juli-2025]. URL: <https://codecharta.com/docs/analysis/codecharta-shell>.
- [Gor+18] Jochen Gortler, Christoph Schulz, Daniel Weiskopf und Oliver Deussen. “Bubble Treemaps for Uncertainty Visualization”. In: *IEEE Transactions on Visualization and Computer Graphics* 24.1 (2018), S. 719–728. DOI: [10.1109/TVCG.2017.2743959](https://doi.org/10.1109/TVCG.2017.2743959).
- [GYBo4] Hamish Graham, Hong Yul Yang und Rebecca Berrigan. “A solar system metaphor for 3D visualisation of object oriented software metrics”. In: *Proceedings of the 2004 Australasian Symposium on Information Visualisation-Volume 35*. 2004, S. 53–59.
- [HF94] Tracy Hall und Norman Fenton. “Implementing software metrics—the critical success factors”. In: *Software Quality Journal* 3 (1994), S. 195–208.
- [HH17] Rinse van Hees und Jurriaan Hage. “Stable and predictable Voronoi treemaps for software quality monitoring”. In: *Information and Software Technology* 87 (2017), S. 242–258.
- [HVVWo5] Danny Holten, Roel Vliegen und Jarke J Van Wijk. “Visual realism for the visualization of software metrics”. In: *3rd IEEE International Workshop on Visualizing Software for Understanding and Analysis*. IEEE. 2005, S. 1–6.

- [Hor+15] Jeffery S Horsburgh, Stephanie L Reeder, Amber Spackman Jones und Jacob Meline. "Open source software for visualization and quality control of continuous hydrologic and water quality sensor data". In: *Environmental Modelling & Software* 70 (2015), S. 32–44.
- [HB19] Yuriy Hrytsiuk und Orest Bilas. "Visualization of Software Quality Expert Assessment". In: *2019 IEEE 14th International Conference on Computer Sciences and Information Technologies (CSIT)*. Bd. 2. 2019, S. 156–160. DOI: [10.1109/STC-CSIT.2019.8929769](https://doi.org/10.1109/STC-CSIT.2019.8929769).
- [Hu+14] Haiyun Hu, Yi Chen, Yuangang Zhen und Ruijun Liu. "A squarified and ordered treemap layout algorithm". In: *Journal of Computer-Aided Design & Computer Graphics* 26.10 (2014), S. 1703–1710.
- [Joh93] Brian Scott Johnson. *Treemaps: Visualizing hierarchical and categorical data*. University of Maryland, College Park, 1993.
- [JS91] Brian Johnson und Ben Shneiderman. *Tree-maps: A space filling approach to the visualization of hierarchical information structures*. Techn. Ber. UM Computer Science Department; CS-TR-2657, 1991.
- [Kha+12] Taimur Khan, Henning Barthel, Achim Ebert und Peter Liggesmeyer. "Visualization and evolution of software architectures". In: *Visualization of Large and Unstructured Data Sets: Applications in Geospatial Planning, Modeling and Engineering-Proceedings of IRTG 1131 Workshop 2011*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik. 2012, S. 25–42.
- [Kha+13] Taimur Khan, Henning Barthel, Achim Ebert und Peter Liggesmeyer. "eCITY: A Tool to Track Software Structural Changes Using an Evolving City". In: *2013 IEEE International Conference on Software Maintenance*. 2013, S. 492–495. DOI: [10.1109/ICSM.2013.80](https://doi.org/10.1109/ICSM.2013.80).
- [Kha+15] Taimur Khan, Henning Barthel, Achim Ebert und Peter Liggesmeyer. "Visual analytics of software structure and metrics". In: *2015 IEEE 3rd Working Conference on Software Visualization (VIS-SOFT)*. 2015, S. 16–25. DOI: [10.1109/VISSOFT.2015.7332411](https://doi.org/10.1109/VISSOFT.2015.7332411).
- [Kit+04] Barbara Kitchenham u. a. "Procedures for performing systematic reviews". In: *Keele, UK, Keele University* 33.2004 (2004), S. 1–26.
- [KHA10] Nicholas Kong, Jeffrey Heer und Maneesh Agrawala. "Perceptual Guidelines for Creating Rectangular Treemaps". In: *IEEE Trans. Visualization & Comp. Graphics (Proc. InfoVis)* (2010). URL: <http://vis.stanford.edu/papers/perception-treemaps>.

- [Kuh+10] Adrian Kuhn, David Erni, Peter Loretan und Oscar Nierstrasz. "Software cartography: Thematic software visualization with consistent layout". In: *Journal of Software Maintenance and Evolution: Research and Practice* 22.3 (2010), S. 191–210.
- [LSP05] Guillaume Langelier, Houari Sahraoui und Pierre Poulin. "Visualization-based analysis of quality for large-scale software systems". In: *Proceedings of the 20th IEEE/ACM international Conference on Automated software engineering*. 2005, S. 214–223.
- [LD02] Michele Lanza und Stéphane Ducasse. "Understanding software evolution using a combination of software visualization and software metrics". In: *LMO'02: Langages et Modèles à Objet*. 2002.
- [LS02] C. Lewerentz und F. Simon. "Metrics-based 3D visualization of large object-oriented programs". In: *Proceedings First International Workshop on Visualizing Software for Understanding and Analysis*. 2002, S. 70–77. DOI: [10.1109/VISSOF.2002.1019796](https://doi.org/10.1109/VISSOF.2002.1019796).
- [LF08] Hao Lü und James Fogarty. "Cascaded treemaps: examining the visibility and stability of structure in treemaps". In: *Proceedings of graphics interface 2008*. 2008, S. 259–266.
- [Lu+17] Liangfu Lu, Shiliang Fan, Maolin Huang, Weidong Huang und Ruolan Yang. "Golden Rectangle Treemap". In: *Journal of Physics: Conference Series* 787.1 (Jan. 2017), S. 012007. DOI: [10.1088/1742-6596/787/1/012007](https://doi.org/10.1088/1742-6596/787/1/012007). URL: <https://dx.doi.org/10.1088/1742-6596/787/1/012007>.
- [Lud] Jochen Ludewig. *Wie gut ist die Software?* URL: <https://elib.uni-stuttgart.de/server/api/core/bitstreams/d40bec3f-ba78-495f-9c44-4e54db9daa30/content>.
- [MFM03] Andrian Marcus, Louis Feng und Jonathan I Maletic. "3D representations for software visualization". In: *Proceedings of the 2003 ACM symposium on Software visualization*. SoftVis '03. San Diego, California: Association for Computing Machinery, 2003, 27–ff. ISBN: 1581136420. DOI: [10.1145/774833.774837](https://doi.org/10.1145/774833.774837). URL: <https://doi.org/10.1145/774833.774837>.
- [Meh25a] Benedikt Mehl. *Anhang zu Masterarbeit*. 2025. URL: <https://github.com/BenediktMehl/master-thesis/tree/main/anhang>.
- [Meh25b] Benedikt Mehl. *Squarify Algorithm Example Video*. 2025. URL: <https://github.com/BenediktMehl/master-thesis/blob/main/images/squarify%5Fexample.gif>.
- [Mer+18] L. Merino, M. Ghafari, C. Anslow und O. Nierstrasz. "A systematic literature review of software visualization evaluation". In: *Journal of Systems and Software* 144 (Juni 2018), S. 165–180. ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2018.06.027>. URL: <https://www.sciencedirect.com/science/article/pii/S0164121218301237>.

- [Mer+17] Leonel Merino, Mohammad Ghafari, Craig Anslow und Oscar Nierstrasz. "CityVR: Gameful software visualization". In: *2017 IEEE International conference on software maintenance and evolution (ICSME)*. IEEE. 2017, S. 633–637.
- [MD09] Parastoo Mohagheghi und Vegard Dehlen. "Existing model metrics and relations to model quality". In: *2009 ICSE Workshop on Software Quality*. IEEE. 2009, S. 39–45.
- [PRW03] Michael J. Pacione, Marc Roper und Murray Wood. "A comparative evaluation of dynamic visualisation tools". English. In: *Proceedings of the 10th Working Conference on Reverse Engineering (WCRE 2003)*. Hrsg. von M. Lanza, S. Ducasse und M. Pinzger. Requires Template change to Chapter in Book/Report/-Conference proceeding : Proceedings of the 10th Working Conference on Reverse Engineering (WCRE 2003); 10th Working Conference on Reverse Engineering, WCRE ; Conference date: 13-11-2003 Through 16-11-2003. 2003, S. 80–89. DOI: [10.1109/WCRE.2003.1287239](https://doi.org/10.1109/WCRE.2003.1287239).
- [PSE04] J.W. Paulson, G. Succi und A. Eberlein. "An empirical study of open-source and closed-source software products". In: *IEEE Transactions on Software Engineering* 30.4 (2004), S. 246–256. DOI: [10.1109/TSE.2004.1274044](https://doi.org/10.1109/TSE.2004.1274044).
- [Rag+05] S. Raghunathan, A. Prasad, B.K. Mishra und Hsihui Chang. "Open source versus closed source: software quality in monopoly and competitive markets". In: *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 35.6 (2005), S. 903–918. DOI: [10.1109/TSMCA.2005.853493](https://doi.org/10.1109/TSMCA.2005.853493).
- [Rei95] Steven P. Reiss. "An Engine for the 3D Visualization of Program Information". In: *Journal of Visual Languages & Computing* 6.3 (1995), S. 299–323. ISSN: 1045-926X. DOI: <https://doi.org/10.1006/jvlc.1995.1017>. URL: <https://www.sciencedirect.com/science/article/pii/S1045926X85710178>.
- [Saj+14] Hitesh Sajnani, Vaibhav Saini, Joel Ossher und Cristina V. Lopes. "Is Popularity a Measure of Quality? An Analysis of Maven Components". In: *2014 IEEE International Conference on Software Maintenance and Evolution*. 2014, S. 231–240. DOI: [10.1109/ICSME.2014.45](https://doi.org/10.1109/ICSME.2014.45).
- [SLD20] Willy Scheibel, Daniel Limberger und Jürgen Döllner. "Survey of treemap layout algorithms". In: *Proceedings of the 13th international symposium on visual information communication and interaction*. 2020, S. 1–9.
- [Sch19] Gast Carina Schmitz. *Wenn Software auf Qualität (s-Metriken) trifft – Teil 2 - MEDtech Ingenieur GmbH*. [Online; besucht 17-Juni-2025]. Jan. 2019. URL: <https://medtech-ingenieur.de/wenn-software-auf-qualitaet-s-metriken-trifft-teil-2/>.



- [Sch23] Google Scholar. *Academia Stack Exchange*. [Online; besucht 29-Juli-2025]. Apr. 2023. URL: <https://academia.stackexchange.com/questions/195737/google-scholar-mixes-all-or-operators-to-one-single-or-condition>.
- [SW01] Ben Shneiderman und Martin Wattenberg. "Ordered treemap layouts". In: *IEEE Symposium on Information Visualization, 2001. INFOVIS 2001*. IEEE. 2001, S. 73–78.
- [TC08] Alfredo R Teyseyre und Marcelo R Campo. "An overview of 3D software visualization". In: *IEEE transactions on visualization and computer graphics* 15.1 (2008), S. 87–105.
- [TKS20] Vishvajeet Thakur, Marouane Kessentini und Tushar Sharma. "QScored: An Open Platform for Code Quality Ranking and Visualization". In: *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2020, S. 818–821. DOI: [10.1109/ICSME46990.2020.00101](https://doi.org/10.1109/ICSME46990.2020.00101).
- [VWW99] J.J. Van Wijk und H. Van de Wetering. "Cushion treemaps: visualization of hierarchical information". In: *Proceedings 1999 IEEE Symposium on Information Visualization (InfoVis'99)*. 1999, S. 73–78. DOI: [10.1109/INFVIS.1999.801860](https://doi.org/10.1109/INFVIS.1999.801860).
- [VCT18] Eduardo Faccin Vernier, João Luiz Dihl Comba und Alexandru Cristian Telea. "A stable greedy insertion treemap algorithm for software evolution visualization". In: *2018 31st SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI)*. IEEE. 2018, S. 158–165.
- [VK17] Jeffrey Voas und Rick Kuhn. "What Happened to Software Metrics?" In: *Computer* 50.5 (Mai 2017), 88–98. DOI: <https://doi.org/10.1109/mc.2017.144>. URL: <https://tsapps.nist.gov/publication/get%5Fpdf.cfm?pub%5Fid=922615>.
- [WLo7] Richard Wettel und Michele Lanza. "Visualizing Software Systems as Cities". In: *2007 4th IEEE International Workshop on Visualizing Software for Understanding and Analysis*. 2007, S. 92–99. DOI: [10.1109/VISSOF.2007.4290706](https://doi.org/10.1109/VISSOF.2007.4290706).
- [WP04] Autoren der Wikimedia-Projekte. *Beschreibung von Eigenschaften einer Software durch Zahlen*. [Online; besucht 17-Juni-2025]. Feb. 2004. URL: <https://de.wikipedia.org/wiki/Softwaremetrik>.
- [WP09] Autoren der Wikimedia-Projekte. *mit deutlichen Einschränkungen räumlich erscheinen oder dreidimensional sein*. [Online; besucht 09-August-2025]. Nov. 2009. URL: <https://de.wikipedia.org/wiki/2%C2%BDD#:~:text=2%C2%BDD%20oder%202%C5D%20englisch,oder%20drei%20Dimensionen%20zu%20besitzen..>

- [Wit18] Frank Witte. “Metriken für die Softwarequalität”. In: *Metriken für das Testreporting: Analyse und Reporting für wirkungsvolles Testmanagement*. Wiesbaden: Springer Fachmedien Wiesbaden, 2018, S. 63–70. ISBN: 978-3-658-19845-9. DOI: [10.1007/978-3-658-19845-9\\_8](https://doi.org/10.1007/978-3-658-19845-9_8). URL: [https://doi.org/10.1007/978-3-658-19845-9\\_%5F8](https://doi.org/10.1007/978-3-658-19845-9_%5F8).
- [WDo8] Jo Wood und Jason Dykes. “Spatially ordered treemaps”. In: *IEEE transactions on visualization and computer graphics* 14.6 (2008), S. 1348–1355.
- [YM98] P. Young und M. Munro. “Visualising software in virtual reality”. In: *Proceedings. 6th International Workshop on Program Comprehension. IWPC’98 (Cat. No.98TB100242)*. 1998, S. 19–26. DOI: [10.1109/WPC.1998.693276](https://doi.org/10.1109/WPC.1998.693276).